BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU ỨNG DỤNG AI TẠO SINH TRONG KIỂM THỬ XÂM NHẬP

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:
Nguyễn Văn Khải - MSSV: AT190128

Người hướng dẫn:
TS. Vũ Thị Vân
Khoa An toàn thông tin - Học viện Kỹ thuật mật mã

Hà Nội, 2026

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU ỨNG DỤNG AI TẠO SINH TRONG KIỂM THỬ XÂM NHẬP

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:
Nguyễn Văn Khải - MSSV: AT190128

Người hướng dẫn:
TS. Vũ Thị Vân
Khoa An toàn thông tin - Học viện Kỹ thuật mật mã

Hà Nội, 2026

MỤC LỤC

Danh mục kí hiệu và viết tắt ..... **iii**
Danh mục hình ảnh ..... **v**
Danh mục bảng ..... **vi**
Lời cảm ơn ..... **vii**
Lời nói đầu ..... **viii**
Chương 1. Tổng quan về kiểm thử xâm nhập và AI tạo sinh **1**
1.1. Tổng quan về kiểm thử xâm nhập 1
1.1.1. Khái niệm và phân loại kiểm thử xâm nhập *1*
1.1.2. Quy trình kiểm thử xâm nhập *2*
1.1.3. Các khung tiêu chuẩn và phương pháp kiểm thử xâm nhập *3*
1.2. Tổng quan về an toàn ứng dụng web và các lỗ hổng phổ biến 4
1.2.1. Tổng quan về OWASP Top 10 ..... *4*
1.2.2. Một số lỗ hổng bảo mật ứng dụng web phổ biến *5*
1.3. Tổng quan về AI tạo sinh và mô hình ngôn ngữ lớn 7
1.3.1. Khái niệm AI tạo sinh và đặc trưng nổi bật *7*
1.3.2. Kiến trúc Transformer và cơ chế hoạt động của LLM *8*
1.3.3. Prompt Engineering và Chain-of-Thought *10*
1.3.4. Cơ chế ReAct trong mô hình ngôn ngữ lớn *12*
1.3.5. Agent AI và cơ chế Tool Calling trong mô hình ngôn ngữ lớn *13*
1.4. Ứng dụng AI tạo sinh trong kiểm thử xâm nhập 16
1.4.1. Hạn chế của kiểm thử xâm nhập truyền thống *16*
1.4.2. Các nghiên cứu và hệ thống AI trong kiểm thử xâm nhập *17*
1.4.3. So sánh và đánh giá các giải pháp hiện có *20*
1.4.4. Thách thức và xu hướng phát triển *22*
1.5. Kết luận chương 1 24
Chương 2. Phân tích và thiết kế hệ thống AI Pentest Agent **26**
2.1. Mô tả bài toán 26
2.2. Các yêu cầu chức năng và yêu cầu phi chức năng 26
2.2.1. Yêu cầu chức năng *26*
2.2.2. Yêu cầu phi chức năng *27*
2.3. Thiết kế kiến trúc hệ thống 28
2.3.1. Kiến trúc tổng quát hệ thống *28*
2.3.2. Kiến trúc các thành phần của hệ thống *31*
2.4. Thiết kế luồng xử lý và cơ chế hoạt động 33
2.4.1. Quy trình hoạt động của hệ thống *33*
2.4.2. Cơ chế quản lý ngữ cảnh và trạng thái phiên *36*
2.4.3. Cơ chế xử lý và thực thi công cụ *40*
2.5. Thiết kế chi tiết các mô-đun chức năng hệ thống 42
2.5.1. Mô-đun điều phối và lập kế hoạch tác vụ *42*
2.5.2. Mô-đun quản lý tri thức và kỹ năng kiểm thử *47*
2.5.3. Mô-đun quản lý và thực thi công cụ *53*
2.5.4. Mô-đun quản lý phiên và bộ nhớ *56*

2.5.5. Mô-đun quản lý trạng thái và kết quả kiểm thử *60*
2.6. Kết luận chương 2 69
Chương 3. Xây dựng và đánh giá hệ thống KAgent 71
3.1. Môi trường và công nghệ sử dụng 71
3.1.1. Môi trường triển khai *71*
3.1.2. Công nghệ và thư viện sử dụng *72*
3.2. Hiện thực và triển khai hệ thống 73
3.2.1. Tổ chức mã nguồn và khởi tạo hệ thống *73*
3.2.2. Hiện thực các chức năng chính *75*
3.2.3. Giao diện và tương tác với hệ thống *80*
3.2.4. Cấu hình và triển khai hệ thống *82*
3.3. Thực nghiệm và đánh giá kết quả 83
3.3.1. Thực nghiệm kiểm thử xâm nhập ứng dụng web *84*
3.3.2. Thực nghiệm tích hợp Burp Suite với KAgent *93*
3.4. Hạn chế và hướng phát triển 99
3.4.1. Hạn chế của hệ thống *99*
3.4.2. Định hướng phát triển hệ thống *101*
3.5. Kết luận chương 3 103
Kết luận 104
Tài liệu tham khảo 105
Phụ lục 107

DANH MỤC KÍ HIỆU VÀ VIẾT TẮT

Từ viết tắt / Ký hiệu	Tiếng Anh	Tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge	Khung tri thức về chiến thuật, kỹ thuật và kiến thức phổ biến về tấn công
CAPEC	Common Attack Pattern Enumeration and Classification	Danh mục phân loại các mẫu tấn công phổ biến
CLI	Command Line Interface	Giao diện dòng lệnh
CoT	Chain-of-Thought	Chuỗi suy luận
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu liên trang
CVE	Common Vulnerabilities and Exposures	Danh mục các lỗ hổng và điểm phơi nhiễm phổ biến
CWE	Common Weakness Enumeration	Danh mục phân loại các điểm yếu phổ biến
DOM	Document Object Model	Mô hình đối tượng tài liệu
GPT	Generative Pre-trained Transformer	Mô hình Transformer tạo sinh được tiền huấn luyện
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
IDOR	Insecure Direct Object References	Tham chiếu đối tượng trực tiếp không an toàn
IP	Internet Protocol	Giao thức Internet
JSON	JavaScript Object Notation	Định dạng biểu diễn dữ liệu dạng văn bản
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MITRE	MITRE	Tổ chức nghiên cứu và phát triển phi lợi nhuận của Hoa Kỳ

Từ viết tắt / Ký hiệu	Tiếng Anh	Tiếng Việt
NIST	National Institute of Standards and Technology	Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ
NVD	National Vulnerability Database	Cơ sở dữ liệu lỗ hổng quốc gia
OWASP	Open Worldwide Application Security Project	Dự án ứng dụng web và phần mềm mở trên toàn cầu
PTES	Penetration Testing Execution Standard	Chuẩn thực hiện kiểm thử xâm nhập
RAG	Retrieval-Augmented Generation	Sinh tăng cường bằng truy xuất
ReAct	Reasoning and Acting	Cơ chế kết hợp suy luận và hành động
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
SQLi	SQL Injection	Tấn công chèn câu lệnh SQL
SSRF	Server-Side Request Forgery	Giả mạo yêu cầu phía máy chủ
TUI	Text User Interface	Giao diện người dùng dạng văn bản
WSTG	Web Security Testing Guide	Hướng dẫn kiểm thử an toàn ứng dụng web
XSS	Cross-Site Scripting	Tấn công chèn mã kịch bản liên trang

DANH MỤC HÌNH ẢNH

Hình 1.1. Kiến trúc tổng quát của Transformer 9
Hình 1.2. So sánh các phương pháp Prompting và cơ chế ReAct 13
Hình 1.3. Kiến trúc tổng quát của hệ thống LLM-based Agent 14
Hình 2.1. Kiến trúc tổng quát hệ thống 29
Hình 2.2. Sơ đồ tổng quan quy trình vận hành của hệ thống KAgent 33
Hình 2.3. Quy trình xử lý một lượt tương tác của KAgent 35
Hình 2.4. Quy trình xây dựng ngữ cảnh cho một lượt tương tác KAgent 38
Hình 2.5. Cơ chế thu gọn ngữ cảnh và khôi phục phiên của KAgent 39
Hình 2.6. Cơ chế xử lý và thực thi một yêu cầu gọi công cụ của KAgent 40
Hình 2.7. Cơ chế điều phối vòng lặp Agent 42
Hình 2.8. Quy trình lập kế hoạch của Decision Planner 45
Hình 2.9. Cơ chế học và tích lũy tri thức 49
Hình 2.10. Quy trình lựa chọn và nạp kỹ năng 52
Hình 2.11. Quy trình xử lý Coverage Tool 66
Hình 2.12. Quy trình xử lý của Confirm Finding 69
Hình 3.1. Giao diện chọn nhà cung cấp và mô hình LLM 76
Hình 3.2. Nạp kỹ năng và thực thi công cụ trên KAgent 77
Hình 3.3. Liên kết dữ liệu xác thực và cập nhật kết quả trong KAgent 79
Hình 3.4. Giao diện thành phần mở rộng KAgent trong Burp Suite 80
Hình 3.5. Giao diện chính của KAgent 81
Hình 3.6. Hộp thoại xác nhận quyền thực thi công cụ 82
Hình 3.7. Quy trình kiểm thử của KAgent 85
Hình 3.8. Các Skill được nạp trong quá trình kiểm thử 86
Hình 3.9. KAgent yêu cầu xác định phương án xử lý các đầu vào B2B 88
Hình 3.10. Điều chỉnh phép thử SQLi theo phản hồi của ứng dụng 90
Hình 3.11. Lưu hai Finding SQL Injection sau xác minh 91
Hình 3.12. Quy trình phối hợp giữa Burp Suite và KAgent 95
Hình 3.13. Chuyển yêu cầu tìm kiếm từ Burp Repeater tới KAgent 95
Hình 3.14. Nạp Skill XSS và triển khai xác minh tham số search 96
Hình 3.15. Kết quả xác minh và thông báo lưu Finding XSS 97
Hình 3.16. Báo cáo Finding XSS tại tham số search 98

DANH MỤC BẢNG

Bảng 1.1. So sánh các phương pháp luận kiểm thử xâm nhập 3
Bảng 1.2. Các nhóm rủi ro trong OWASP Top 10:2021 5
Bảng 1.3. So sánh các hướng ứng dụng AI trong kiểm thử xâm nhập 20
Bảng 2.1. Các yêu cầu chức năng của hệ thống 26
Bảng 2.2. Các yêu cầu phi chức năng của hệ thống 27
Bảng 2.3. Cấu trúc trạng thái và các nguồn dữ liệu hỗ trợ của KAgent 37
Bảng 2.4. Các thành phần của DecisionPlan 44
Bảng 2.5. Các nhóm thông tin chính của IntelligenceScenario 48
Bảng 2.6. Các nhóm metadata chính của kỹ năng kiểm thử 50
Bảng 2.7. Các thành phần chính của công cụ 53
Bảng 2.8. Phân nhóm công cụ theo mục đích sử dụng 55
Bảng 2.9. Các thành phần chính của SessionFile 57
Bảng 2.10. Các nhóm thông tin chính của SessionMemory 58
Bảng 2.11. Các thuộc tính chính của cấu trúc MemoryFact 59
Bảng 2.12. Các thành phần dữ liệu chính của WorkflowState 61
Bảng 2.13. Các trạng thái xử lý của đối tượng nghi vấn (Candidate) 62
Bảng 2.14. Các kết luận xác thực của ValidationResult 64
Bảng 2.15. Các thành phần dữ liệu chính của CoverageEntry 65
Bảng 2.16. Các thành phần dữ liệu chính của Finding 67
Bảng 3.1. Môi trường triển khai hệ thống KAgent 72
Bảng 3.2. Các công nghệ và thư viện chính sử dụng trong KAgent 72
Bảng 3.3. Các nhóm mã nguồn chính của KAgent 74
Bảng 3.4. Các nhóm cấu hình phục vụ khởi chạy KAgent 83
Bảng 3.5. Sử dụng kỹ năng và công cụ trong thực nghiệm 87
Bảng 3.6. Tổng hợp kết quả qua các giai đoạn thực nghiệm 91
Bảng 3.7. Kết quả xác minh SQL Injection tại ba Candidate 92
Bảng 3.8. Các phép thử và kết quả phân tích tại tham số search 97

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án tốt nghiệp, tôi đã nhận được sự hướng dẫn tận tình của TS. Vũ Thị Vân - Giảng viên Khoa An toàn thông tin, Học viện Kỹ thuật Mật mã. Tôi xin được bày tỏ lòng biết ơn sâu sắc tới cô vì những chỉ dẫn quý báu và sự hỗ trợ nhiệt tình trong suốt thời gian nghiên cứu và hoàn thiện đề tài!

SINH VIÊN THỰC HIỆN ĐỀ TÀI

Nguyễn Văn Khải

LỜI NÓI ĐẦU

Trong bối cảnh các hệ thống thông tin, ứng dụng web và dịch vụ trực tuyến ngày càng được triển khai rộng rãi, yêu cầu bảo đảm an toàn cho các hệ thống trước khi đưa vào vận hành ngày càng trở nên quan trọng. Các lỗ hổng bảo mật trong ứng dụng có thể bị khai thác để truy cập trái phép, thay đổi hoặc đánh cắp dữ liệu, từ đó gây ảnh hưởng đến tính an toàn và hoạt động của hệ thống. Kiểm thử xâm nhập (penetration testing) là một phương pháp quan trọng nhằm phát hiện và đánh giá các điểm yếu bảo mật thông qua việc mô phỏng các kỹ thuật tấn công trong phạm vi được cho phép. Tuy nhiên, quá trình kiểm thử vẫn phụ thuộc đáng kể vào kiến thức và kinh nghiệm của kiểm thử viên. Các công việc như thu thập thông tin, phân tích kết quả, lập kế hoạch, lựa chọn công cụ và xác định bước kiểm thử tiếp theo thường phải được thực hiện qua nhiều vòng xử lý, làm gia tăng thời gian và yêu cầu về nguồn lực.

Cùng với sự phát triển của trí tuệ nhân tạo tạo sinh (Generative AI), đặc biệt là các mô hình ngôn ngữ lớn (Large Language Model - LLM), nhiều nghiên cứu đã bắt đầu khai thác khả năng của AI trong việc hỗ trợ các hoạt động kiểm thử xâm nhập. LLM có khả năng xử lý và tổng hợp thông tin, lập kế hoạch, sinh lệnh hoặc hành động và tương tác với các công cụ bên ngoài. Trên cơ sở đó, nhiều hướng nghiên cứu đã được đề xuất nhằm xây dựng các tác nhân AI có khả năng hỗ trợ một phần hoặc một số giai đoạn của quy trình kiểm thử xâm nhập. Tuy nhiên, để LLM có thể tham gia hiệu quả vào một quy trình kiểm thử gồm nhiều bước liên tiếp, vẫn cần giải quyết các vấn đề như duy trì ngữ cảnh, quản lý trạng thái phiên, lựa chọn và điều phối công cụ, kiểm soát hành động, xử lý kết quả và xác định bước thực hiện tiếp theo. Những vấn đề này ảnh hưởng trực tiếp đến khả năng hoạt động ổn định và mức độ tự động hóa của AI Pentest Agent.

Từ thực tiễn kiểm thử xâm nhập và xu hướng nghiên cứu ứng dụng AI tạo sinh nêu trên, việc nghiên cứu cơ chế kết hợp mô hình ngôn ngữ với các công cụ kiểm thử là cần thiết. Đề tài "Nghiên cứu ứng dụng AI tạo sinh trong kiểm thử xâm nhập" được thực hiện với mục tiêu nghiên cứu và xây dựng một nguyên mẫu AI Pentest Agent theo kiến trúc tác nhân đơn (Single-Agent), sử dụng LLM để hỗ trợ một phần quy trình kiểm thử xâm nhập ứng dụng web. Đồ án tập trung giải quyết các vấn đề chính gồm cơ chế tương tác giữa Agent và LLM, quản lý ngữ cảnh và trạng thái phiên, cơ chế suy luận và thực hiện hành động, lựa chọn và điều phối công cụ, xử lý kết quả trả về và duy trì chu trình suy luận - hành động - quan sát. Hệ thống

được xây dựng và thực nghiệm trong môi trường kiểm thử được kiểm soát, tập trung đánh giá khả năng hỗ trợ kiểm thử đối với một số lỗ hổng web tiêu biểu như SQL Injection (SQLi) và Cross-Site Scripting (XSS).

Đồ án được tổ chức thành 03 chương:

Chương 1: Tổng quan về kiểm thử xâm nhập và AI tạo sinh. Chương này trình bày các kiến thức cơ bản về kiểm thử xâm nhập, AI tạo sinh và các hướng ứng dụng AI trong kiểm thử xâm nhập.

Chương 2: Phân tích và thiết kế hệ thống AI Pentest Agent. Chương này trình bày bài toán, yêu cầu, kiến trúc và cơ chế hoạt động của hệ thống.

Chương 3: Xây dựng, thực nghiệm và đánh giá hệ thống AI Pentest Agent. Chương này trình bày quá trình xây dựng, thực nghiệm và đánh giá hệ thống, đồng thời đề xuất hướng phát triển.

SINH VIÊN THỰC HIỆN ĐỀ TÀI

Nguyễn Văn Khải

CHƯƠNG 1. TỔNG QUAN VỀ KIỂM THỬ XÂM NHẬP VÀ AI TẠO SINH

1.1. Tổng quan về kiểm thử xâm nhập

1.1.1. Khái niệm và phân loại kiểm thử xâm nhập

Kiểm thử xâm nhập (Penetration Testing) là một hoạt động đánh giá an toàn thông tin mang tính chủ động, trong đó các chuyên gia bảo mật được ủy quyền mô phỏng các cuộc tấn công trong môi trường được kiểm soát nhằm xác định, xác minh khả năng khai thác và đánh giá mức độ ảnh hưởng của các lỗ hổng bảo mật tồn tại trong hạ tầng công nghệ thông tin của tổ chức. Hoạt động này có thể được thực hiện trên nhiều đối tượng khác nhau như hệ thống, mạng, ứng dụng và thiết bị để đánh giá mức độ an toàn cũng như khả năng chống chịu trước các cuộc tấn công.

Thông qua việc mô phỏng các chiến thuật, kỹ thuật và quy trình tấn công (Tactics, Techniques, and Procedures - TTPs) mà các tác nhân độc hại thường sử dụng, kiểm thử xâm nhập giúp tổ chức đánh giá hiệu quả của các biện pháp bảo vệ hiện có, xác định và ưu tiên khắc phục các lỗ hổng bảo mật, đồng thời hỗ trợ đáp ứng các yêu cầu về tiêu chuẩn, quy định an toàn thông tin và nâng cao mức độ bảo mật tổng thể của hệ thống.

Dựa trên mức độ thông tin được cung cấp về hệ thống mục tiêu, kiểm thử xâm nhập được chia thành ba hình thức chính: kiểm thử hộp đen (Black-box Testing), kiểm thử hộp xám (Gray-box Testing) và kiểm thử hộp trắng (White-box Testing)

a) Kiểm thử hộp đen

Chuyên gia kiểm thử không được cung cấp bất kỳ thông tin nào về hệ thống mục tiêu trước khi tiến hành kiểm thử. Mọi hoạt động đánh giá được thực hiện từ góc nhìn của một tác nhân bên ngoài nhằm mô phỏng các cuộc tấn công thực tế và đánh giá khả năng phòng vệ của hệ thống.

b) Kiểm thử hộp xám

Chuyên gia kiểm thử được cung cấp một phần thông tin về hệ thống, chẳng hạn như tài khoản người dùng, tài liệu thiết kế ở mức cơ bản hoặc sơ đồ kiến trúc. Việc có sẵn một số thông tin giúp quá trình kiểm thử hiệu quả hơn nhưng vẫn phản ánh tương đối sát các kịch bản tấn công trong thực tế.

c) Kiểm thử hộp trắng

Chuyên gia kiểm thử được cung cấp đầy đủ thông tin về hệ thống, bao gồm mã nguồn, kiến trúc, tài liệu thiết kế và cấu hình. Điều này cho phép quá trình kiểm

thử được thực hiện toàn diện hơn, giúp phát hiện cả những lỗ hổng khó nhận biết bằng các phương pháp kiểm thử từ bên ngoài.

1.1.2. Quy trình kiểm thử xâm nhập

Quy trình kiểm thử xâm nhập giúp tổ chức các hoạt động đánh giá theo trình tự thống nhất, từ xác định phạm vi đến tổng hợp kết quả. Trong đó, Penetration Testing Execution Standard (PTES) cung cấp phương pháp luận bao quát vòng đời của một cuộc kiểm thử, làm cơ sở để lựa chọn và phối hợp các hoạt động phù hợp với mục tiêu đánh giá. [1]

Theo PTES, quy trình kiểm thử xâm nhập được chia thành bảy giai đoạn chính, bao gồm: Trao đổi và chuẩn bị trước kiểm thử (Pre-engagement Interactions), Thu thập thông tin (Intelligence Gathering), Mô hình hóa mối đe dọa (Threat Modeling), Phân tích lỗ hổng (Vulnerability Analysis), Khai thác lỗ hổng (Exploitation), Hậu khai thác (Post Exploitation) và Báo cáo kết quả (Reporting).

a) Trao đổi và chuẩn bị trước kiểm thử

Nhóm kiểm thử và khách hàng thống nhất mục tiêu, phạm vi, phương pháp, thời gian và các quy tắc thực hiện (Rules of Engagement). Giai đoạn này cũng xác định quyền hạn, trách nhiệm và cách xử lý tình huống phát sinh, tạo cơ sở để các hoạt động tiếp theo diễn ra trong phạm vi được cấp phép và hạn chế tác động ngoài mong muốn.

b) Thu thập thông tin

Thông tin về mục tiêu được thu thập bằng các phương pháp thụ động hoặc chủ động, gồm tên miền, địa chỉ IP, dịch vụ, công nghệ và các thành phần liên quan. Kết quả thu thập giúp người kiểm thử hình dung cấu trúc hệ thống, nhận diện các điểm cần xem xét và chuẩn bị dữ liệu cho quá trình phân tích tiếp theo.

c) Mô hình hóa mối đe dọa

Dựa trên thông tin đã thu thập, người kiểm thử xác định các tài sản quan trọng, mối đe dọa tiềm ẩn và bề mặt tấn công. Các kịch bản tấn công được xây dựng và ưu tiên theo mức độ rủi ro, qua đó định hướng việc phân tích lỗ hổng và lựa chọn đối tượng cần kiểm tra sâu hơn.

d) Phân tích lỗ hổng

Giai đoạn này sử dụng công cụ hỗ trợ kết hợp với kiểm tra thủ công để nhận diện các điểm yếu có thể tồn tại trong hệ thống. Những dấu hiệu phát hiện được tiếp tục được phân tích về khả năng khai thác và mức độ ảnh hưởng, làm cơ sở lựa chọn các giả thuyết cần kiểm chứng ở bước khai thác.

e) Khai thác lỗ hổng

Người kiểm thử thực hiện các thao tác khai thác có kiểm soát để kiểm chứng lỗ hổng và đánh giá tác động thực tế. Hoạt động này tuân theo phạm vi đã thỏa thuận, đồng thời thu thập bằng chứng về kết quả đạt được để phục vụ đánh giá rủi ro và lập báo cáo.

f) Hậu khai thác

Sau khi khai thác thành công, người kiểm thử đánh giá mức độ truy cập và kiểm soát có thể đạt được, chẳng hạn khả năng leo thang đặc quyền, tiếp cận dữ liệu quan trọng hoặc di chuyển giữa các hệ thống. Các hoạt động được lựa chọn theo mục tiêu và phạm vi cho phép nhằm làm rõ hậu quả của việc lỗ hổng bị khai thác.

g) Báo cáo kết quả

Kết quả kiểm thử được tổng hợp thành báo cáo gồm mô tả lỗ hổng, bằng chứng, mức độ rủi ro, tác động và khuyến nghị khắc phục. Báo cáo cung cấp căn cứ để tổ chức ưu tiên xử lý các điểm yếu và kiểm tra lại hiệu quả của biện pháp khắc phục.

1.1.3. Các khung tiêu chuẩn và phương pháp kiểm thử xâm nhập

Bên cạnh PTES, nhiều khung tiêu chuẩn và phương pháp luận khác cũng được sử dụng trong lĩnh vực kiểm thử xâm nhập nhằm hướng dẫn và chuẩn hóa quá trình đánh giá an toàn thông tin. Mỗi khung có mục tiêu, phạm vi áp dụng và đặc điểm riêng. Bảng 1.1 trình bày một số khung tiêu chuẩn và phương pháp luận kiểm thử xâm nhập được sử dụng phổ biến hiện nay. [2], [3]

Bảng 1.1. So sánh các phương pháp luận kiểm thử xâm nhập

Khung / Phương pháp luận	**Đơn vị phát triển**	**Mục tiêu chính**	**Ưu điểm**	**Hạn chế**
NIST SP 800-115	NIST	Hướng dẫn hoạt động đánh giá an toàn thông tin.	Có tính hệ thống, được sử dụng rộng rãi làm tài liệu tham chiếu.	Cung cấp tổng quan kỹ thuật và quy trình đánh giá, không thay thế hướng dẫn kiểm thử chi tiết cho từng loại lỗ hổng hoặc ứng dụng.

Khung / Phương pháp luận	Đơn vị phát triển	Mục tiêu chính	Ưu điểm	Hạn chế
PTES	PTES Team	Chuẩn hóa quy trình thực hiện kiểm thử xâm nhập.	Quy trình rõ ràng, bao quát toàn bộ vòng đời của một cuộc kiểm thử xâm nhập.	Các giai đoạn cần được cụ thể hóa theo mục tiêu, phạm vi và môi trường kiểm thử.
OWASP WSTG	OWASP Foundation	Hướng dẫn kiểm thử bảo mật ứng dụng web.	Nội dung chi tiết, được cập nhật thường xuyên và sử dụng rộng rãi trong kiểm thử web.	Chỉ áp dụng cho kiểm thử ứng dụng web.

1.2. Tổng quan về an toàn ứng dụng web và các lỗ hổng phổ biến

Ứng dụng web thường tiếp nhận dữ liệu từ người dùng và kết nối với cơ sở dữ liệu, API hoặc dịch vụ bên ngoài. Các lỗi trong xử lý đầu vào, kiểm soát truy cập, thiết kế hoặc cấu hình có thể tạo ra điểm yếu cho phép truy cập trái phép, làm lộ dữ liệu hay ảnh hưởng đến hoạt động của ứng dụng.

An toàn ứng dụng web hướng tới bảo đảm tính bí mật, toàn vẹn và sẵn sàng của dữ liệu và dịch vụ. Trong kiểm thử xâm nhập, việc nắm được các nhóm rủi ro và lỗ hổng phổ biến giúp định hướng khảo sát bề mặt tấn công, lựa chọn đầu vào cần kiểm tra và đánh giá kết quả. Đồ án sử dụng OWASP Top 10 làm cơ sở tham chiếu, đồng thời tập trung thực nghiệm đối với SQL Injection và Cross-Site Scripting.

1.2.1. Tổng quan về OWASP Top 10

OWASP Foundation là tổ chức phi lợi nhuận hoạt động trong lĩnh vực an toàn phần mềm, cung cấp các tài liệu, công cụ và hướng dẫn nhằm hỗ trợ phát triển ứng dụng an toàn. OWASP Top 10 là tài liệu tổng hợp mười nhóm rủi ro bảo mật quan trọng đối với ứng dụng web, được sử dụng để nâng cao nhận thức và hỗ trợ nhận diện các điểm yếu trong quá trình phát triển, vận hành và kiểm thử ứng dụng.

Các nhóm rủi ro thuộc phiên bản OWASP Top 10:2021 và nội dung mô tả tương ứng được trình bày trong Bảng 1.2. [4]

Bảng 1.2. Các nhóm rủi ro trong OWASP Top 10:2021

Mã	Nhóm rủi ro	Mô tả
A01	Broken Access Control	Kiểm soát truy cập không đầy đủ, cho phép người dùng thực hiện các hành động vượt quá quyền hạn được cấp.
A02	Cryptographic Failures	Các vấn đề liên quan đến việc bảo vệ dữ liệu bằng cơ chế mã hóa không phù hợp hoặc không đầy đủ, dẫn đến nguy cơ lộ thông tin nhạy cảm.
A03	Injection	Lỗ hổng xảy ra khi dữ liệu đầu vào không được kiểm soát cho phép kẻ tấn công chèn các câu lệnh hoặc mã độc vào hệ thống.
A04	Insecure Design	Các điểm yếu phát sinh từ quá trình thiết kế ứng dụng chưa tích hợp đầy đủ các nguyên tắc bảo mật.
A05	Security Misconfiguration	Cấu hình bảo mật không phù hợp trên ứng dụng, máy chủ hoặc các thành phần liên quan, làm gia tăng nguy cơ bị khai thác.
A06	Vulnerable and Outdated Components	Việc sử dụng các thư viện, framework hoặc thành phần phần mềm chứa lỗ hổng đã biết hoặc không còn được hỗ trợ cập nhật.
A07	Identification and Authentication Failures	Các vấn đề liên quan đến xác thực danh tính, quản lý tài khoản và quản lý phiên làm việc của người dùng.
A08	Software and Data Integrity Failures	Không đảm bảo tính toàn vẹn của phần mềm và dữ liệu trong quá trình phát triển, cập nhật hoặc triển khai hệ thống.
A09	Security Logging and Monitoring Failures	Thiếu cơ chế ghi nhận, giám sát và phân tích nhật ký bảo mật, gây khó khăn trong việc phát hiện và xử lý sự cố.
A10	Server-Side Request Forgery (SSRF)	Lợi dụng ứng dụng để gửi yêu cầu trái phép tới các tài nguyên nội bộ hoặc bên ngoài mà ứng dụng không được phép truy cập.

1.2.2. Một số lỗ hổng bảo mật ứng dụng web phổ biến

Trong các nhóm rủi ro được đề cập trong OWASP Top 10, nhiều lỗ hổng bảo mật ứng dụng web xuất hiện thường xuyên trong thực tế và có mức độ ảnh hưởng lớn đến tính bảo mật của hệ thống. Các lỗ hổng này có thể cho phép kẻ tấn công truy cập trái phép vào dữ liệu, chiếm quyền người dùng, thực thi mã độc hoặc khai thác

các tài nguyên nội bộ của hệ thống. Việc tìm hiểu các lỗ hổng phổ biến giúp chuyên gia kiểm thử xây dựng các kịch bản đánh giá phù hợp, đồng thời là cơ sở cho việc phát triển các hệ thống hỗ trợ kiểm thử xâm nhập ứng dụng web.

a) SQL Injection (SQLi)

SQL Injection là một lỗ hổng thuộc nhóm Injection (A03) trong OWASP Top 10:2021, xảy ra khi dữ liệu đầu vào của người dùng không được kiểm tra hoặc xử lý đúng cách trước khi được đưa vào các câu truy vấn cơ sở dữ liệu. Kẻ tấn công có thể lợi dụng lỗ hổng này để chèn các câu lệnh SQL độc hại nhằm truy xuất, sửa đổi hoặc xóa dữ liệu trái phép. Trong những trường hợp nghiêm trọng, SQL Injection có thể dẫn đến việc chiếm quyền kiểm soát hệ quản trị cơ sở dữ liệu hoặc ảnh hưởng đến toàn bộ ứng dụng.

b) Cross-Site Scripting (XSS)

Cross-Site Scripting cũng là một lỗ hổng thuộc nhóm Injection (A03) trong OWASP Top 10:2021, cho phép kẻ tấn công chèn các đoạn mã script độc hại, thường là JavaScript, vào nội dung của trang web. Khi người dùng truy cập vào trang chứa mã độc, trình duyệt sẽ thực thi đoạn mã trong ngữ cảnh của ứng dụng. XSS thường được chia thành ba dạng chính gồm Stored XSS**,** Reflected XSS và DOM-based XSS. Lỗ hổng này có thể bị khai thác để đánh cắp cookie phiên, giả mạo người dùng hoặc thu thập các thông tin nhạy cảm.

c) Insecure Direct Object References (IDOR)

Insecure Direct Object References là một dạng lỗ hổng thuộc nhóm Broken Access Control (A01) trong OWASP Top 10:2021, phát sinh khi ứng dụng cho phép truy cập trực tiếp đến các đối tượng dữ liệu thông qua các tham số do người dùng kiểm soát nhưng không thực hiện kiểm tra quyền truy cập tương ứng. Kẻ tấn công có thể thay đổi các tham số như mã định danh người dùng, số tài liệu hoặc tên tệp để truy cập trái phép vào dữ liệu thuộc về người dùng khác, dẫn đến nguy cơ rò rỉ thông tin hoặc truy cập trái phép vào các tài nguyên quan trọng.

d) Server-Side Request Forgery (SSRF)

Server-Side Request Forgery là một lỗ hổng thuộc nhóm A10 trong OWASP Top 10:2021, cho phép kẻ tấn công lợi dụng ứng dụng để gửi các yêu cầu từ phía máy chủ đến các tài nguyên nội bộ hoặc bên ngoài mà thông thường không thể truy cập trực tiếp. Thông qua SSRF, kẻ tấn công có thể khai thác các dịch vụ nội bộ, thu thập thông tin hệ thống hoặc vượt qua các cơ chế kiểm soát truy cập mạng. Trong một số trường hợp, SSRF có thể là bước đệm để thực hiện các cuộc tấn công phức tạp hơn đối với hạ tầng của hệ thống.

e) Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery là kỹ thuật tấn công lợi dụng phiên đăng nhập hợp lệ của người dùng để khiến họ vô tình thực hiện các yêu cầu ngoài ý muốn đến ứng dụng web. Mặc dù không còn được xếp thành một nhóm rủi ro riêng trong OWASP Top 10:2021, CSRF vẫn là một lỗ hổng phổ biến liên quan đến quản lý phiên và xác thực yêu cầu, thường xảy ra khi ứng dụng không triển khai các cơ chế xác minh nguồn gốc của yêu cầu, chẳng hạn như CSRF Token. Nếu khai thác thành công, kẻ tấn công có thể thực hiện các thao tác thay mặt người dùng như thay đổi thông tin tài khoản, cập nhật mật khẩu hoặc thực hiện các giao dịch trái phép.

Các lỗ hổng trên là những vấn đề bảo mật phổ biến trong ứng dụng web và thường được lựa chọn làm đối tượng kiểm thử trong quá trình đánh giá an toàn thông tin. Đồng thời, đây cũng là các lỗ hổng thường được sử dụng trong các nghiên cứu về ứng dụng AI tạo sinh nhằm đánh giá khả năng của mô hình ngôn ngữ lớn trong việc hỗ trợ phân tích, phát hiện và kiểm thử các lỗ hổng bảo mật ứng dụng web.

1.3. Tổng quan về AI tạo sinh và mô hình ngôn ngữ lớn

1.3.1. Khái niệm AI tạo sinh và đặc trưng nổi bật

AI tạo sinh (Generative Artificial Intelligence - Generative AI) là nhánh của trí tuệ nhân tạo có khả năng học đặc điểm của dữ liệu và tạo ra nội dung như văn bản, hình ảnh, âm thanh hoặc mã nguồn. Đối với đề tài này, khả năng xử lý ngôn ngữ, tổng hợp thông tin và sinh nội dung là cơ sở để AI hỗ trợ phân tích dữ liệu kiểm thử, đề xuất hành động và trình bày kết quả.

Một nền tảng quan trọng của AI tạo sinh là các mô hình nền tảng (Foundation Models), được huấn luyện trước trên dữ liệu quy mô lớn và có thể thích ứng với nhiều nhiệm vụ. Mô hình ngôn ngữ lớn (Large Language Models - LLM) là nhóm tiêu biểu, xử lý và sinh ngôn ngữ dựa trên thông tin ngữ cảnh. Các dòng mô hình như GPT, Llama, Claude và Gemini cho phép người dùng định hướng tác vụ bằng chỉ dẫn ngôn ngữ tự nhiên.

LLM có thể thực hiện nhiều nhiệm vụ như trả lời câu hỏi, tóm tắt, phân tích thông tin và sinh mã nguồn thông qua việc thay đổi yêu cầu đầu vào. Khả năng khai thác ngữ cảnh giúp mô hình kết hợp yêu cầu của người dùng với dữ liệu được cung cấp, nhưng tính liên tục của một nhiệm vụ dài còn phụ thuộc vào cách hệ thống quản lý và bổ sung thông tin. Một số mô hình cũng hỗ trợ dữ liệu đa phương thức, mở rộng cách tiếp nhận thông tin ngoài văn bản.

AI tạo sinh vẫn có thể tạo nội dung sai lệch hoặc thiếu căn cứ (hallucination), chịu ảnh hưởng của chất lượng dữ liệu và gặp khó khăn với nhiệm vụ chuyên sâu. Việc sử dụng mô hình còn đặt ra yêu cầu về tài nguyên và chi phí. Vì vậy, các kỹ thuật Prompt Engineering, truy xuất tri thức, Tool Calling và kiến trúc Agent được kết hợp để cung cấp ngữ cảnh phù hợp, hỗ trợ thực hiện nhiệm vụ và kiểm tra kết quả.

1.3.2. Kiến trúc Transformer và cơ chế hoạt động của LLM

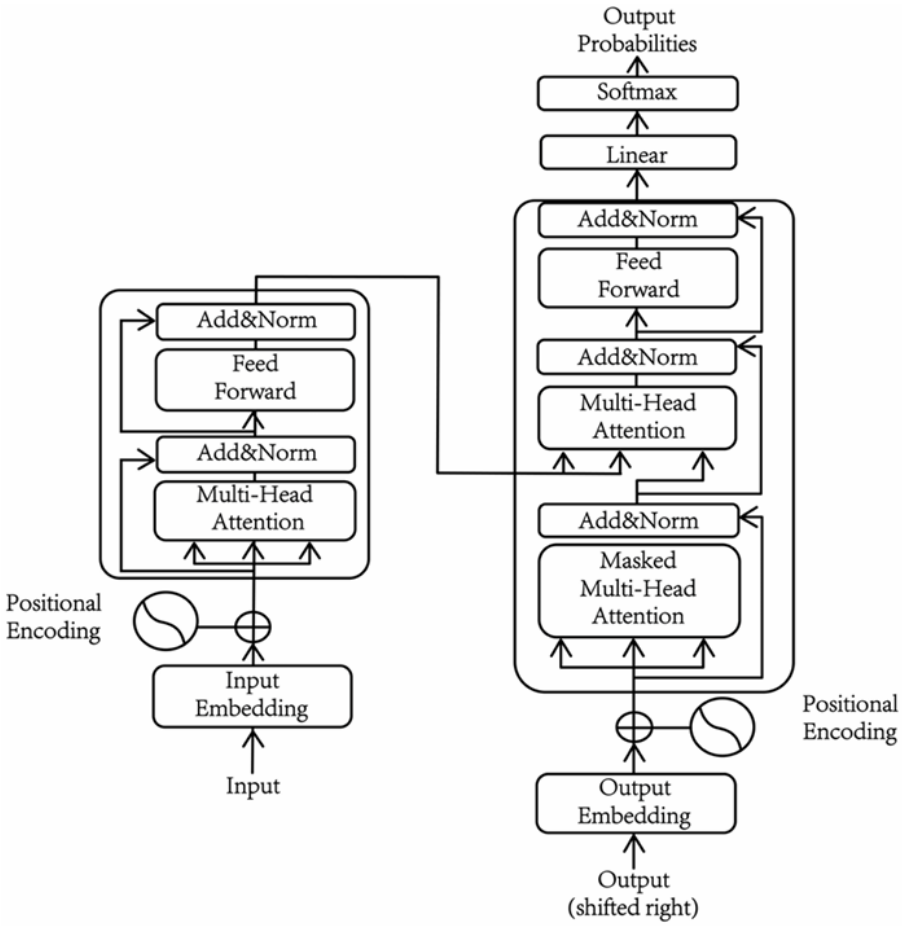
Transformer là kiến trúc mạng nơ-ron nền tảng của nhiều mô hình ngôn ngữ lớn. Trước đó, mạng nơ-ron hồi tiếp (RNN) và mạng nhớ dài - ngắn hạn (LSTM) thường xử lý chuỗi theo trình tự, gặp khó khăn khi học quan hệ giữa các token ở khoảng cách xa và hạn chế khả năng tính toán song song trong huấn luyện.

Transformer sử dụng cơ chế Attention để khai thác quan hệ giữa các phần tử trong chuỗi. Cách tổ chức này hỗ trợ mô hình hóa ngữ cảnh và xử lý song song trong huấn luyện, tạo điều kiện phát triển các mô hình ngôn ngữ trên dữ liệu quy mô lớn. [5]

a) Kiến trúc Transformer

Transformer ban đầu được giới thiệu với kiến trúc Encoder-Decoder. Từ kiến trúc này, nhiều biến thể đã được phát triển, trong đó kiến trúc decoder-only được sử dụng phổ biến trong các mô hình ngôn ngữ sinh văn bản, tiêu biểu như dòng GPT và nhiều mô hình cùng loại.

Transformer bổ sung thông tin về thứ tự token thông qua cơ chế biểu diễn vị trí, chẳng hạn Positional Encoding. Thông tin này giúp mô hình phân biệt vị trí và khai thác quan hệ theo thứ tự trong chuỗi, bên cạnh quan hệ được học bằng Attention.



Hình 1.1. Kiến trúc tổng quát của Transformer

Mỗi tầng Transformer kết hợp Multi-Head Attention và Feed Forward Network để khai thác quan hệ giữa các token và biến đổi biểu diễn dữ liệu. Residual Connection và Layer Normalization hỗ trợ duy trì quá trình huấn luyện ổn định khi mô hình gồm nhiều tầng.

b) Cơ chế Self-Attention

Self-Attention giúp Transformer xác định mức độ liên quan giữa một token với các token khác trong cùng chuỗi dữ liệu. Qua cơ chế này, mô hình khai thác thông tin từ các vị trí khác nhau để biểu diễn ngữ cảnh của token đang xét, kể cả khi các token có liên quan nằm cách xa nhau trong chuỗi.

Để nâng cao khả năng biểu diễn ngữ cảnh, Transformer sử dụng Multi-Head Attention, trong đó nhiều đầu Attention được thực hiện song song. Mỗi đầu có thể học các mối quan hệ khác nhau trong dữ liệu, chẳng hạn như quan hệ ngữ pháp, ngữ nghĩa hoặc sự liên kết giữa các thành phần trong văn bản. Các kết quả sau đó được kết hợp để tạo thành biểu diễn giàu thông tin hơn cho chuỗi đầu vào.

Các cơ chế Attention giúp Transformer khai thác quan hệ trong chuỗi mà không phải truyền thông tin lần lượt qua từng bước như RNN. Khả năng xử lý song

song này chủ yếu được tận dụng trong huấn luyện, còn mô hình sinh văn bản tự hồi quy vẫn tạo các token đầu ra theo trình tự.

c) Cơ chế hoạt động của mô hình ngôn ngữ lớn

LLM được huấn luyện trước trên khối lượng lớn dữ liệu văn bản và mã nguồn để học các quy luật ngôn ngữ. Với mô hình sinh văn bản tự hồi quy, nhiệm vụ phổ biến là dự đoán token tiếp theo dựa trên các token đứng trước, từ đó hình thành khả năng tạo nội dung theo ngữ cảnh.

Khi người dùng cung cấp một Prompt, dữ liệu đầu vào trước tiên được chia thành các đơn vị nhỏ gọi là token thông qua quá trình Tokenization. Các token sau đó được biểu diễn dưới dạng các vector số và đưa vào mô hình Transformer để phân tích ngữ cảnh thông qua các tầng xử lý và cơ chế Self-Attention.

Đối với các mô hình ngôn ngữ tự hồi quy (autoregressive), trong quá trình suy luận, mô hình sử dụng các tham số đã được học cùng với ngữ cảnh đầu vào để dự đoán token tiếp theo. Token mới được tạo ra tiếp tục được đưa vào ngữ cảnh để mô hình dự đoán token tiếp theo, và quá trình này được lặp lại cho đến khi đạt điều kiện kết thúc. Chuỗi token được sinh ra sau đó được chuyển đổi thành văn bản đầu ra để cung cấp cho người dùng.

Cùng một mô hình có thể hỗ trợ trả lời câu hỏi, tóm tắt, sinh mã nguồn hoặc phân tích thông tin thông qua những yêu cầu khác nhau. Chất lượng đầu ra phụ thuộc vào ngữ cảnh, cách xây dựng Prompt và năng lực của mô hình, nên việc tổ chức đầu vào là yêu cầu quan trọng khi ứng dụng LLM vào kiểm thử xâm nhập.

1.3.3. Prompt Engineering và Chain-of-Thought

Chất lượng đầu ra của LLM phụ thuộc không chỉ vào mô hình mà còn vào cách tổ chức yêu cầu và dữ liệu đầu vào. Prompt Engineering giúp xác định nhiệm vụ, phạm vi, ràng buộc và định dạng kết quả, trong khi Chain-of-Thought (CoT) hỗ trợ xử lý những nhiệm vụ cần suy luận qua nhiều bước.

a) Prompt Engineering

Prompt gồm các chỉ dẫn, thông tin ngữ cảnh và dữ liệu được cung cấp cho LLM để định hướng quá trình xử lý. Prompt Engineering là việc thiết kế và điều chỉnh các nội dung này nhằm tạo đầu ra phù hợp với tác vụ mà không thay đổi trọng số mô hình như khi tinh chỉnh.

Một Prompt rõ ràng giúp mô hình nhận biết mục tiêu, giới hạn và yêu cầu đầu ra, đồng thời giảm cách hiểu không phù hợp. Trong hệ thống Agent, Prompt còn cần

phân biệt chỉ dẫn thực hiện nhiệm vụ với dữ liệu quan sát để kết quả công cụ được sử dụng đúng vai trò.

Thông thường, một Prompt có thể bao gồm các thành phần chính:

- Instruction: Mô tả nhiệm vụ mà mô hình cần thực hiện, yêu cầu về nội dung hoặc định dạng kết quả đầu ra.
- Examples: Cung cấp các cặp đầu vào - đầu ra mẫu nhằm minh họa cho mô hình cách thực hiện nhiệm vụ.
- Input: Dữ liệu hoặc câu hỏi thực tế mà mô hình cần xử lý.

Thông tin ngữ cảnh có thể được bổ sung để làm rõ mục tiêu, dữ liệu nền và các ràng buộc. Chẳng hạn, một yêu cầu phân tích phản hồi HTTP cần nêu phương thức, điểm đầu vào đang xem xét và yêu cầu chỉ đưa ra nhận định dựa trên bằng chứng được cung cấp.

Theo số lượng ví dụ được cung cấp, hai cách xây dựng Prompt thường gặp là Zero-shot và Few-shot Prompting.

Zero-shot Prompting cung cấp yêu cầu và dữ liệu trực tiếp, không kèm ví dụ mẫu. Cách này phù hợp khi nhiệm vụ và định dạng đầu ra được mô tả rõ. Ví dụ, người dùng có thể yêu cầu mô hình phân tích một phản hồi HTTP, chỉ ra dấu hiệu nghi vấn SQL Injection và đề xuất bước kiểm tra tiếp theo mà chưa kết luận có lỗ hổng.

Few-shot Prompting bổ sung một số cặp đầu vào - đầu ra mẫu trước yêu cầu chính để minh họa cách xử lý và trình bày kết quả. Với cùng tác vụ phân tích HTTP, các ví dụ có thể thể hiện cách phân biệt dấu hiệu lỗi thông thường với dấu hiệu cần kiểm chứng. Mô hình vận dụng các mẫu này trong ngữ cảnh, nhưng kết quả vẫn cần được đánh giá trên dữ liệu thực tế.

Role Prompting định hướng mô hình theo một vai trò, chẳng hạn trợ lý kiểm thử bảo mật, để phù hợp với phạm vi phân tích và cách diễn đạt. Structured Prompting tổ chức yêu cầu và đầu ra theo cấu trúc rõ ràng như Markdown hoặc JSON, hỗ trợ hệ thống tiếp nhận và sử dụng kết quả trong các bước tiếp theo.

Trong kiểm thử xâm nhập, Prompt Engineering có thể hỗ trợ phân tích kết quả công cụ, giải thích dấu hiệu lỗ hổng, đề xuất bước kiểm tra và tổng hợp báo cáo. Yêu cầu nên nêu rõ phạm vi mục tiêu, dữ liệu hiện có và tiêu chí kết luận. Cách tổ chức này giúp hạn chế phản hồi không liên quan, nhưng không bảo đảm loại bỏ hoàn toàn hallucination hoặc thay thế việc kiểm chứng bằng chứng.

b) Chain-of-Thought (CoT)

Chain-of-Thought (CoT) khuyến khích mô hình xử lý nhiệm vụ thông qua các bước suy luận trung gian trước khi đưa ra kết quả. Kỹ thuật này có thể hỗ trợ phân tích thông tin, lập kế hoạch và giải quyết các bài toán có nhiều quan hệ phụ thuộc, thay vì yêu cầu mô hình trả lời ngay từ dữ liệu ban đầu. [6]

Zero-shot CoT sử dụng chỉ dẫn yêu cầu giải quyết vấn đề theo từng bước mà không kèm mẫu. Few-shot CoT cung cấp các ví dụ có phần suy luận trung gian và kết quả để định hướng cách xử lý những nhiệm vụ tương tự. Hiệu quả của hai cách tiếp cận phụ thuộc vào mô hình, chất lượng chỉ dẫn và đặc điểm tác vụ.

CoT có thể cải thiện cách tổ chức phân tích, nhưng các bước suy luận và kết luận vẫn có thể sai. Việc tạo thêm nội dung còn làm tăng số token, thời gian phản hồi và chi phí xử lý. Do đó, trong kiểm thử xâm nhập, cần đối chiếu nhận định với dữ liệu thực nghiệm thay vì coi lời giải thích của mô hình là bằng chứng.

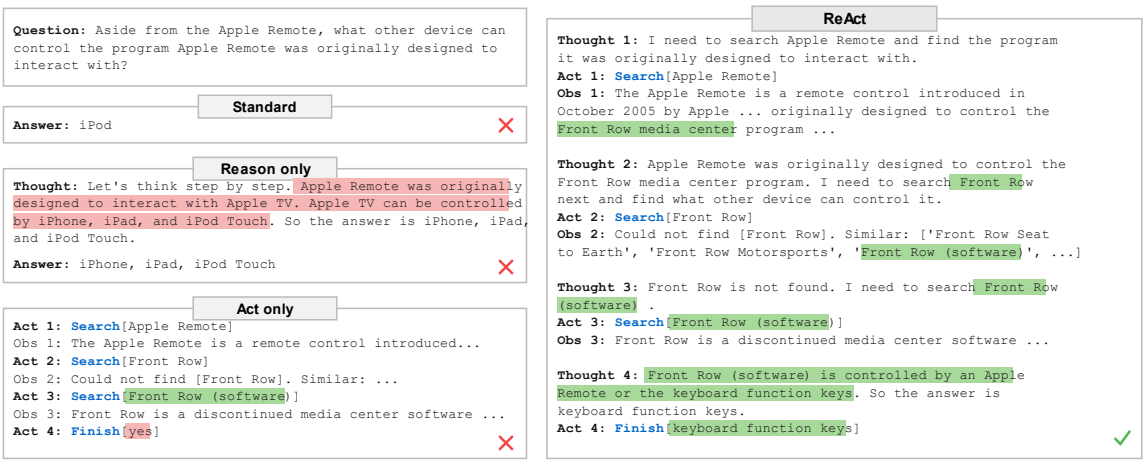
CoT chủ yếu sử dụng thông tin đã có trong ngữ cảnh và không tự cung cấp khả năng thực thi công cụ. Để thu thập thông tin mới và điều chỉnh hướng kiểm tra dựa trên phản hồi từ môi trường, hệ thống cần kết hợp suy luận với cơ chế hành động, như cách tiếp cận ReAct được trình bày tiếp theo.

1.3.4. Cơ chế ReAct trong mô hình ngôn ngữ lớn

ReAct (Reasoning and Acting) kết hợp suy luận của mô hình ngôn ngữ với hành động trong một chu trình xử lý. Khi nhiệm vụ cần thông tin ngoài ngữ cảnh hoặc thao tác với môi trường, mô hình có thể đề xuất sử dụng công cụ, tiếp nhận kết quả và dựa vào đó xác định bước tiếp theo. [7]

Chu trình ReAct gồm Thought, Action và Observation. Thought biểu diễn việc phân tích thông tin hiện có và lựa chọn hướng xử lý. Ở bước Action, mô hình đề xuất hành động như gọi API, tìm kiếm hoặc sử dụng công cụ, còn hệ thống tích hợp đảm nhiệm việc thực thi. Observation là kết quả trả về, được bổ sung vào ngữ cảnh để phục vụ lần suy luận tiếp theo.

Chu trình được lặp lại cho đến khi nhiệm vụ có thể kết thúc hoặc đạt điều kiện dừng. Phản hồi từ môi trường giúp mô hình điều chỉnh hành động thay vì chỉ dựa vào kế hoạch ban đầu. Hình 1.2 minh họa sự khác nhau giữa các phương pháp Prompting và cơ chế ReAct.



Hình 1.2. So sánh các phương pháp Prompting và cơ chế ReAct

So với CoT chủ yếu phân tích ngữ cảnh sẵn có, ReAct bổ sung khả năng thu thập thông tin qua hành động. Trong kiểm thử web, hệ thống có thể gửi một yêu cầu HTTP, phân tích phản hồi, lựa chọn đầu vào cần kiểm tra rồi tiếp tục thu thập bằng chứng. Mỗi kết quả quan sát tạo cơ sở cho bước tiếp theo, nhưng không tự chứng minh kết luận của mô hình là đúng.

Hiệu quả của ReAct phụ thuộc vào năng lực lựa chọn hành động, độ tin cậy của công cụ và cách duy trì ngữ cảnh. Hành động không phù hợp hoặc phản hồi bị diễn giải sai có thể dẫn tới kiểm tra lặp hay kết luận thiếu căn cứ. Nhiều vòng tương tác cũng làm tăng chi phí và thời gian, nên hệ thống cần giới hạn thực thi và kiểm soát phạm vi.

Nguyên lý suy luận - hành động - quan sát của ReAct là cơ sở để xây dựng Agent thực hiện nhiệm vụ kiểm thử nhiều bước. Khi kết hợp với quản lý trạng thái và cơ chế gọi công cụ, hệ thống có thể duy trì tiến trình, điều chỉnh hướng kiểm tra và tổng hợp kết quả dựa trên các dữ liệu đã thu được.

1.3.5. Agent AI và cơ chế Tool Calling trong mô hình ngôn ngữ lớn

a) Khái niệm Agent AI dựa trên mô hình ngôn ngữ lớn

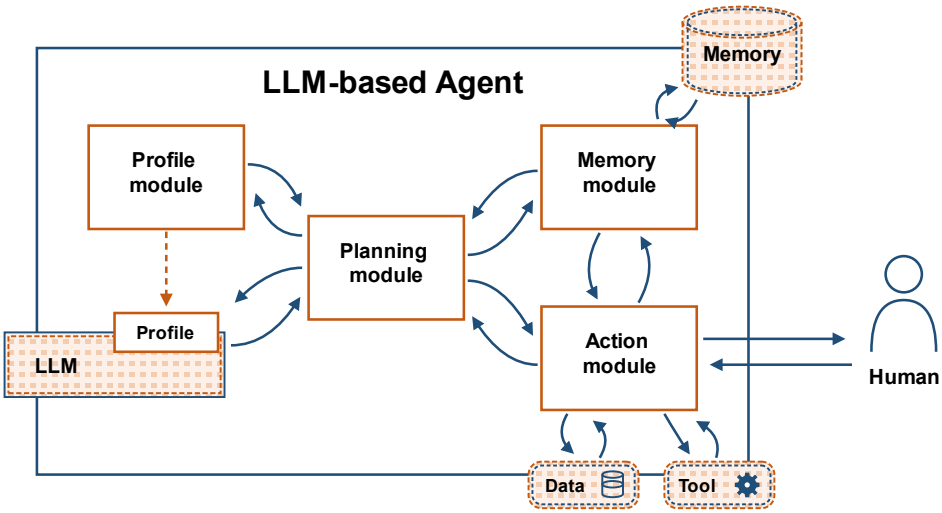
LLM có thể phân tích và sinh nội dung, nhưng việc sử dụng mô hình như một giao diện hội thoại chưa đủ để thực hiện nhiệm vụ kéo dài có nhiều bước và tương tác với môi trường. Hệ thống cần bổ sung cơ chế duy trì trạng thái, lập kế hoạch và thực thi hành động.

Agent AI dựa trên mô hình ngôn ngữ lớn (LLM-based Agent) kết hợp LLM với các thành phần hỗ trợ để tiếp nhận mục tiêu, lựa chọn bước xử lý và sử dụng công cụ. LLM cung cấp khả năng phân tích và đề xuất, còn hệ thống điều phối tổ chức vòng xử lý, quản lý dữ liệu và kiểm soát thực thi.

Khác với mô hình ngôn ngữ chỉ tạo ra phản hồi dựa trên một lần xử lý đầu vào, Agent AI có khả năng thực hiện các nhiệm vụ theo chu trình nhiều bước. Agent có thể tiếp nhận mục tiêu ban đầu, xây dựng kế hoạch thực hiện, sử dụng các công cụ bên ngoài để thu thập thông tin, đánh giá kết quả nhận được và điều chỉnh hành động tiếp theo dựa trên trạng thái hiện tại.

b) Kiến trúc tổng quát của LLM-based Agent

Kiến trúc tổng quát của LLM-based Agent có thể được mô tả thông qua bốn nhóm chức năng gồm Profile, Memory, Planning và Action như Hình 1.3. Các thành phần này phối hợp để xác định vai trò của Agent, duy trì thông tin, xây dựng kế hoạch và thực hiện hành động trong quá trình xử lý nhiệm vụ.



Hình 1.3. Kiến trúc tổng quát của hệ thống LLM-based Agent

Profile xác định vai trò, mục tiêu và phạm vi nhiệm vụ của Agent, qua đó định hướng quá trình phân tích và lựa chọn hành động. Trong kiểm thử xâm nhập, thành phần này có thể mô tả vai trò trợ lý bảo mật, đối tượng cần đánh giá và yêu cầu sử dụng bằng chứng khi đưa ra kết luận. Các giới hạn thực thi được kiểm soát độc lập bởi hệ thống.

Memory lưu trữ và cung cấp thông tin để Agent sử dụng kết quả đã thu thập, đồng thời duy trì tính liên tục giữa các bước xử lý. Bộ nhớ ngắn hạn phục vụ nhiệm vụ đang thực hiện, còn bộ nhớ dài hạn lưu những thông tin có thể tái sử dụng trong các nhiệm vụ sau. Đối với kiểm thử xâm nhập, thông tin được lưu có thể gồm kết quả khảo sát, các đầu vào đã phát hiện và tiến độ kiểm tra. Khi đưa vào ngữ cảnh của mô hình, dữ liệu cần được lựa chọn phù hợp để hạn chế vượt giới hạn ngữ cảnh hoặc nhầm lẫn thông tin giữa các mục tiêu.

Planning hỗ trợ phân tích mục tiêu, phân rã nhiệm vụ và lựa chọn trình tự xử lý. Trong kiểm thử ứng dụng web, kế hoạch có thể bao gồm khảo sát mục tiêu, phân tích đầu vào và xác minh các dấu hiệu lỗ hổng. Kế hoạch được điều chỉnh theo phản hồi từ ứng dụng và tiến độ công việc để phù hợp với thông tin thu được trong quá trình thực hiện.

Action cho phép Agent thực hiện các thao tác như truy vấn dữ liệu, gửi yêu cầu HTTP hoặc gọi công cụ chuyên dụng. Các yêu cầu hành động được chuyển đến thành phần thực thi và kiểm tra theo phạm vi, quyền cùng các giới hạn của hệ thống. Kết quả thực hiện được đưa trở lại Agent để bổ sung thông tin và xác định bước xử lý tiếp theo.

Sự phối hợp giữa bốn thành phần tạo thành quá trình xử lý nhiệm vụ nhiều bước. Mức độ tự chủ của Agent phụ thuộc vào cách tổ chức các thành phần, khả năng sử dụng công cụ và yêu cầu giám sát trong từng hệ thống.

c) Cơ chế Tool Calling trong mô hình ngôn ngữ lớn

Tool Calling, còn được gọi là Function Calling trong một số nền tảng, cho phép mô hình ngôn ngữ lớn yêu cầu sử dụng công cụ hoặc dịch vụ bên ngoài. Thông qua cơ chế này, Agent có thể thu thập dữ liệu và thực hiện các thao tác phục vụ nhiệm vụ, sau đó sử dụng kết quả để tiếp tục phân tích.

Quá trình bắt đầu khi LLM xem xét yêu cầu và thông tin hiện có để xác định có cần sử dụng công cụ hay không. Nếu cần, mô hình lựa chọn công cụ và tạo yêu cầu gọi gồm tên công cụ cùng các tham số đầu vào. LLM chỉ tạo yêu cầu, còn việc thực thi do thành phần trung gian của hệ thống đảm nhiệm.

Thành phần trung gian tiếp nhận yêu cầu, kiểm tra các điều kiện thực thi và gọi công cụ tương ứng. Kết quả được đưa trở lại mô hình như thông tin bổ sung để đánh giá tiến độ và xác định bước tiếp theo. Tùy theo kết quả thu được, mô hình có thể yêu cầu sử dụng thêm công cụ hoặc tổng hợp câu trả lời. Quá trình này có thể lặp lại nhiều lần trong cùng một nhiệm vụ.

Chẳng hạn, trong hệ thống hỗ trợ kiểm thử xâm nhập, Agent có thể yêu cầu sử dụng Nmap để khảo sát các cổng và dịch vụ trên mục tiêu. Thành phần thực thi của hệ thống tiến hành thao tác trong phạm vi được phép rồi chuyển kết quả về cho mô hình. Dựa trên các dịch vụ được ghi nhận, Agent tiếp tục lựa chọn nội dung cần kiểm tra.

d) Mối quan hệ giữa ReAct, Tool Calling và ứng dụng

ReAct mô tả cách tổ chức chu trình suy luận, hành động và quan sát, còn Tool Calling cung cấp cơ chế để mô hình yêu cầu sử dụng công cụ và tiếp nhận kết quả.

Hai cơ chế có thể được kết hợp khi các hành động trong chu trình ReAct được thực hiện thông qua Tool Calling. Kết quả từ công cụ trở thành dữ liệu quan sát để mô hình đánh giá và điều chỉnh hướng xử lý. Tuy nhiên, yêu cầu gọi công cụ do mô hình tạo ra vẫn phải được hệ thống kiểm tra trước khi thực thi.

Trong kiểm thử xâm nhập, sự kết hợp này giúp Agent lựa chọn phép thử, thu thập phản hồi và điều chỉnh các bước kiểm tra theo thông tin thực tế. Các thành phần quản lý trạng thái duy trì tiến độ và kết quả đã thu được, trong khi cơ chế kiểm soát thực thi giới hạn hành động theo phạm vi và quyền được cấp. Nhờ đó, hệ thống có thể hỗ trợ tự động hóa một phần quy trình kiểm thử, đồng thời cho phép người kiểm thử theo dõi và can thiệp khi cần.

1.4. Ứng dụng AI tạo sinh trong kiểm thử xâm nhập

1.4.1. Hạn chế của kiểm thử xâm nhập truyền thống

Mặc dù quy trình kiểm thử xâm nhập đã được chuẩn hóa thông qua nhiều phương pháp luận như PTES, OWASP Web Security Testing Guide (WSTG) hay OSSTMM, việc triển khai trong thực tế vẫn còn phụ thuộc đáng kể vào chuyên gia kiểm thử và các công cụ bảo mật truyền thống. Điều này dẫn đến nhiều hạn chế về hiệu quả, khả năng mở rộng và mức độ tự động hóa của quá trình kiểm thử.

a) Phụ thuộc nhiều vào kinh nghiệm của chuyên gia

Chất lượng của một cuộc kiểm thử phụ thuộc lớn vào trình độ chuyên môn và kinh nghiệm của người thực hiện. Việc nhận diện lỗ hổng, lựa chọn phương pháp khai thác, xây dựng chuỗi tấn công và đánh giá mức độ rủi ro đều đòi hỏi kiến thức chuyên sâu. Do đó, kết quả kiểm thử có thể khác nhau giữa các chuyên gia hoặc giữa các tổ chức thực hiện.

b) Tiêu tốn nhiều thời gian và nguồn lực

Một cuộc kiểm thử hoàn chỉnh bao gồm nhiều giai đoạn như thu thập thông tin, phân tích bề mặt tấn công, xác minh lỗ hổng, khai thác, hậu khai thác và lập báo cáo. Mặc dù các công cụ như Nmap, Burp Suite, Nikto hay Metasploit hỗ trợ tự động hóa một số tác vụ, người kiểm thử vẫn phải xác minh kết quả, loại bỏ các cảnh báo dương tính giả (false positive) và lựa chọn phương pháp khai thác phù hợp. Điều này làm tăng đáng kể thời gian và chi phí thực hiện.

c) Thiếu khả năng suy luận theo ngữ cảnh

Các công cụ kiểm thử truyền thống chủ yếu hoạt động dựa trên luật hoặc chữ ký nhận diện. Vì vậy, chúng gặp khó khăn trong việc phát hiện các lỗ hổng nghiệp vụ, các chuỗi tấn công nhiều bước hoặc các lỗ hổng chưa có mẫu nhận diện.

d) Khó mở rộng trong môi trường phát triển hiện đại

Trong các mô hình phát triển phần mềm liên tục như CI/CD, hệ thống thay đổi thường xuyên với tần suất triển khai cao. Việc thực hiện kiểm thử xâm nhập theo phương pháp truyền thống khó đáp ứng yêu cầu đánh giá bảo mật liên tục do phần lớn các bước phân tích và ra quyết định vẫn phụ thuộc vào con người.

e) Quá trình lập báo cáo còn thủ công

Sau khi hoàn thành kiểm thử, chuyên gia cần tổng hợp kết quả, mô tả lỗ hổng, phân tích mức độ ảnh hưởng, đưa ra bằng chứng khai thác và đề xuất biện pháp khắc phục. Công việc này thường được thực hiện thủ công, mất nhiều thời gian và dễ thiếu tính nhất quán giữa các báo cáo.

Những hạn chế trên là động lực quan trọng thúc đẩy việc nghiên cứu ứng dụng AI tạo sinh nhằm hỗ trợ tự động hóa một phần quy trình kiểm thử xâm nhập, giảm khối lượng công việc thủ công và nâng cao hiệu quả đánh giá an toàn thông tin.

1.4.2. Các nghiên cứu và hệ thống AI trong kiểm thử xâm nhập

Các công trình ứng dụng LLM trong kiểm thử xâm nhập khác nhau ở vai trò điều phối, bên thực hiện hành động và cách duy trì tiến trình. Phần này khảo sát ba nhóm gồm AI hỗ trợ chuyên gia, Agent trực tiếp điều phối thực thi và hệ thống phối hợp nhiều tác tử. Đây là cách tổ chức nội dung để làm rõ cơ chế, không phải một thang tự động hóa cố định. Một hệ thống có thể kết hợp nhiều cơ chế và mức tham gia của con người cần được xem xét theo công trình, phiên bản hoặc chế độ được dẫn.

a) Kiểm thử xâm nhập có AI hỗ trợ

Trong hướng AI hỗ trợ chuyên gia (AI-assisted Pentesting), mô hình ngôn ngữ được sử dụng để phân tích thông tin và đề xuất bước kiểm thử. Chuyên gia bảo mật vẫn lựa chọn và thực hiện các thao tác kỹ thuật. Đây là hướng tiếp cận được sử dụng trong giai đoạn đầu ứng dụng LLM vào kiểm thử xâm nhập.

Về nguyên lý hoạt động, người kiểm thử cung cấp thông tin về mục tiêu, kết quả quét hoặc thông báo lỗi cho mô hình ngôn ngữ. Dựa trên ngữ cảnh hội thoại, AI phân tích thông tin, giải thích kết quả, gợi ý các bước kiểm thử tiếp theo, sinh câu lệnh hoặc đề xuất payload phù hợp. AI không trực tiếp điều khiển quy trình kiểm thử và cũng không duy trì vòng lặp thực thi liên tục, mà phản hồi theo từng yêu cầu của người dùng trong mỗi lượt tương tác. Người kiểm thử giữ quyền lựa chọn hành động, thực thi và kiểm chứng kết quả, trong khi AI hỗ trợ phân tích và đề xuất.

Ưu điểm của hướng tiếp cận này là dễ triển khai, không yêu cầu thay đổi quy trình kiểm thử hiện có và hạn chế rủi ro do AI không trực tiếp tương tác với hệ thống mục tiêu. Tuy nhiên, khả năng tự động hóa còn thấp và hiệu quả phụ thuộc nhiều vào chất lượng câu lệnh đầu vào cũng như kinh nghiệm của người sử dụng.

PentestGPT trong công trình công bố tại USENIX Security 2024 tổ chức ba mô-đun phục vụ lập luận kiểm thử, sinh bước thực hiện và phân tích thông tin. Hệ thống duy trì tiến trình bằng cây tác vụ, trong khi người kiểm thử thực hiện các bước được đề xuất và cung cấp kết quả trở lại. Vì vậy, công trình này thể hiện cơ chế hỗ trợ điều phối và quản lý ngữ cảnh, chưa đồng nghĩa với một Agent tự thực thi toàn bộ thao tác trên mục tiêu. Các mô hình hội thoại như ChatGPT hoặc Claude cũng có thể được sử dụng ở vai trò trợ lý khi chỉ tiếp nhận dữ liệu và đề xuất bước kiểm tra cho người dùng. [8]

Hướng hỗ trợ chuyên gia giúp bổ sung khả năng phân tích vào quy trình hiện có và giữ con người ở vị trí lựa chọn, thực hiện hoặc kiểm chứng hành động. Mức tự động hóa phụ thuộc cách tích hợp công cụ và trạng thái, nên không thể xác định chỉ từ việc hệ thống sử dụng LLM hoặc có nhiều mô-đun xử lý.

b) Kiểm thử xâm nhập sử dụng tác tử AI

Nhằm khắc phục những hạn chế của mô hình AI hỗ trợ, nhiều nghiên cứu đã phát triển các hệ thống AI Agent có khả năng chủ động hơn trong quá trình kiểm thử. Thay vì chỉ phản hồi theo yêu cầu của người dùng, Agent có thể lập kế hoạch, lựa chọn hành động và tương tác với các công cụ kiểm thử để thực hiện chuỗi nhiệm vụ nhiều bước.

Hướng tiếp cận này kết hợp LLM với ReAct, Tool Calling và Memory để tổ chức vòng xử lý suy luận - hành động - quan sát. Agent phân tích trạng thái hiện tại, lựa chọn công cụ, thực hiện thao tác và sử dụng kết quả để điều chỉnh bước tiếp theo. Việc duy trì kế hoạch và ngữ cảnh cho phép hệ thống xử lý các tác vụ có nhiều bước phụ thuộc nhau. Mức độ chủ động thực thi còn phụ thuộc vào thiết kế và cơ chế giám sát của từng hệ thống.

Nhiều hệ thống vẫn áp dụng cơ chế Human-in-the-Loop, trong đó AI đề xuất hoặc thực hiện một số tác vụ dưới sự giám sát và xác nhận của chuyên gia nhằm đảm bảo an toàn và tuân thủ phạm vi kiểm thử. Điều này cho thấy Agent không đồng nghĩa với việc hệ thống phải hoàn toàn tự chủ, mà mức độ tham gia của con người có thể được điều chỉnh tùy theo mục tiêu và yêu cầu của từng hệ thống.

hackingBuddyGPT là nguyên mẫu dùng LLM để lựa chọn và thực hiện các bước leo thang đặc quyền trên Linux, sau đó sử dụng kết quả lệnh làm thông tin cho

bước tiếp theo. Trong công trình được dẫn, mục tiêu là chuyển từ tài khoản quyền thấp lên quyền cao trong môi trường đánh giá được kiểm soát. Phạm vi chuyên biệt này cho phép khảo sát vòng hành động tự động, nhưng kết quả không thể được suy rộng trực tiếp thành khả năng kiểm thử đầy đủ ứng dụng web. [9]

Pentest Copilot được xem xét theo mô tả và bản minh họa XSS do BugBase công bố ngày 15/8/2025. Ở chế độ này, hệ thống có thể duyệt ứng dụng, thu thập yêu cầu, tạo phép kiểm tra và thực hiện hành động trong trình duyệt. Do đó, không phù hợp khi xếp chế độ này vào nhóm chỉ tư vấn và không trực tiếp sử dụng công cụ. Đây là nguồn mô tả chức năng của nhà phát triển, không phải căn cứ độc lập để so sánh hiệu quả với các công trình khác. [10]

Điểm phân biệt của hướng Agent trực tiếp thực thi là hệ thống kết nối quyết định của mô hình với công cụ và phản hồi từ môi trường. Tuy nhiên, khả năng tự chủ còn phụ thuộc phạm vi được giao, giới hạn hành động và cơ chế giám sát. Việc tự thực hiện nhiều bước không tự bảo đảm kết luận kiểm thử chính xác.

c) Kiểm thử xâm nhập sử dụng hệ thống đa tác tử

Sự phát triển của các hệ thống AI Agent tiếp tục dẫn đến xu hướng xây dựng kiến trúc Multi-Agent nhằm giải quyết những bài toán kiểm thử có quy mô lớn và yêu cầu phối hợp nhiều nhiệm vụ.

Khi nhiệm vụ mở rộng, một Agent đơn phải đảm nhiệm nhiều chức năng như lập kế hoạch, truy xuất tri thức, sinh lệnh và đánh giá kết quả. Việc phân chia trách nhiệm cho các tác tử chuyên biệt được nghiên cứu nhằm giảm tải ngữ cảnh và hỗ trợ phối hợp trong những quy trình kiểm thử phức tạp.

Trong kiến trúc Multi-Agent, mỗi tác tử đảm nhiệm một chức năng như thu thập thông tin, phân tích lỗ hổng, lập kế hoạch, thực thi công cụ hoặc đánh giá kết quả. Các tác tử trao đổi thông tin và sử dụng bộ nhớ chia sẻ để duy trì trạng thái chung. Cách tổ chức này tạo điều kiện cho chuyên môn hóa và mở rộng hệ thống, nhưng hiệu quả phụ thuộc vào chất lượng phân công, trao đổi dữ liệu và đồng bộ giữa các tác tử.

PentestAgent trong công trình AsiaCCS 2025 phối hợp các Agent phục vụ thu thập thông tin, phân tích lỗ hổng và khai thác, kết hợp RAG để bổ sung tri thức cho các bước xử lý. Mô hình tham gia lựa chọn hành động, công cụ thực hiện thao tác, còn thông tin thu được được chuyển giữa các vai trò để tiếp tục nhiệm vụ. Cách phân công này hỗ trợ chuyên môn hóa, nhưng cần duy trì nhất quán dữ liệu và tiến độ giữa các tác tử. Kết quả công bố cần được hiểu trong môi trường và tập tác vụ của nghiên cứu. [11]

AutoPentest cũng sử dụng cấu trúc phối hợp nhiều tác tử, gồm Planner xây dựng kế hoạch, Supervisor phân công bước tiếp theo và các Worker chuyên biệt thực hiện tác vụ bằng công cụ. Tri thức truy xuất hỗ trợ các Worker, còn người dùng cung cấp mục tiêu và giám sát những hành động cần kiểm soát. Công trình được đánh giá trên ba máy Hack The Box theo dạng CTF, vì vậy mức tự chủ và hiệu quả cần gắn với điều kiện đánh giá đó, không dùng làm kết luận chung cho mọi ứng dụng web. [12]

Mặc dù có tiềm năng nâng cao mức độ tự động hóa, các hệ thống Multi-Agent cũng đặt ra nhiều thách thức về điều phối tác tử, đồng bộ trạng thái, quản lý bộ nhớ và kiểm soát phạm vi thực thi trong môi trường thực tế.

Ba hướng khảo sát thể hiện những cách phân chia trách nhiệm giữa mô hình, công cụ và chuyên gia kiểm thử. AI hỗ trợ chuyên gia chủ yếu cung cấp thông tin và đề xuất, trong khi các hệ thống Agent bổ sung cơ chế duy trì tiến trình và điều phối tác vụ. Multi-Agent mở rộng khả năng phân công cho nhiều tác tử. Tuy nhiên, mức tự động hóa và độ tin cậy cần được xem xét theo cơ chế thực thi, giám sát và điều kiện đánh giá của từng giải pháp.

1.4.3. So sánh và đánh giá các giải pháp hiện có

Các hướng khảo sát khác nhau ở cách phân chia trách nhiệm giữa con người, mô hình và công cụ. Bảng 1.3 so sánh các đặc điểm thiết kế ở mức khái quát, trong đó kiến trúc một hay nhiều tác tử được tách khỏi mức tự chủ thực thi. Các mô tả không thay thế đánh giá theo điều kiện cụ thể của từng công trình.

Bảng 1.3. So sánh các hướng ứng dụng AI trong kiểm thử xâm nhập

Tiêu chí	AI hỗ trợ chuyên gia	Agent tác nhân đơn	Hệ thống đa tác tử
Vai trò của AI	Phân tích dữ liệu và đề xuất bước kiểm thử cho chuyên gia	Một Agent điều phối quyết định, hành động và phản hồi	Nhiều Agent phân chia trách nhiệm và phối hợp tiến trình
Mức tự chủ thực thi	Trong chế độ chỉ tư vấn, người dùng thực hiện thao tác	Phụ thuộc công cụ, quyền và cơ chế giám sát	Phụ thuộc quyền, cơ chế phối hợp và giám sát, không mặc nhiên cao hơn tác nhân đơn
Khả năng lập kế hoạch	Đề xuất kế hoạch để người dùng xem xét	Có thể duy trì kế hoạch và điều chỉnh theo phản hồi	Có thể phân công lập kế hoạch và thực hiện cho các vai trò

Tiêu chí	AI hỗ trợ chuyên gia	Agent tác nhân đơn	Hệ thống đa tác tử
Khả năng sử dụng công cụ	Người dùng thực hiện hoặc chuyển kết quả cho mô hình	Agent đề xuất lời gọi, hệ thống tích hợp thực thi	Các Agent sử dụng công cụ theo trách nhiệm và cơ chế điều phối
Quản lý ngữ cảnh và trạng thái	Qua hội thoại hoặc trạng thái hỗ trợ của hệ thống	Duy trì ngữ cảnh và trạng thái của Agent	Trao đổi hoặc chia sẻ trạng thái giữa các Agent
Vai trò của con người	Lựa chọn, thực hiện và kiểm chứng bước kiểm thử	Thiết lập mục tiêu, giám sát hoặc phê duyệt theo chế độ	Thiết lập mục tiêu, giám sát và can thiệp theo thiết kế
Khả năng mở rộng	Phụ thuộc quy trình và mức tích hợp	Có thể bổ sung công cụ, kỹ năng và nguồn dữ liệu	Có thể bổ sung vai trò, đồng thời cần quản lý phối hợp
Độ phức tạp triển khai	Thường thấp hơn khi chỉ bổ sung trợ lý hội thoại	Cần tổ chức vòng điều phối, trạng thái và công cụ	Thêm yêu cầu phân công, trao đổi dữ liệu và đồng bộ

Theo Bảng 1.3, AI-assisted Pentesting phù hợp với việc hỗ trợ chuyên gia phân tích thông tin, sinh câu lệnh và tổng hợp báo cáo. Người kiểm thử giữ vai trò lựa chọn, thực hiện và kiểm chứng hành động nên kết quả vẫn phụ thuộc đáng kể vào kinh nghiệm của họ. Cách tiếp cận này thuận lợi khi bổ sung AI vào quy trình hiện có, nhưng chưa cung cấp đầy đủ cơ chế điều phối liên tục cho một nhiệm vụ kiểm thử nhiều bước.

Các hệ thống AI Agent bổ sung khả năng duy trì ngữ cảnh, lập kế hoạch và phối hợp sử dụng công cụ để xử lý chuỗi tác vụ. Kết quả công cụ trở thành thông tin đầu vào cho quyết định tiếp theo, giúp giảm một phần thao tác điều phối thủ công. Dù vậy, việc lựa chọn hành động và diễn giải kết quả vẫn có thể sai lệch. Vì vậy, cơ chế kiểm chứng và Human-in-the-Loop cần được cân nhắc theo phạm vi và mức tác động của từng thao tác.

Multi-Agent phân chia các trách nhiệm cho nhiều tác tử chuyên biệt, thuận lợi khi nhiệm vụ cần nhiều vai trò phối hợp. Đổi lại, hệ thống phải xử lý thêm việc trao đổi dữ liệu, đồng bộ trạng thái và quản lý bộ nhớ chung. Số lượng tác tử không tự quyết định chất lượng kiểm thử. Việc lựa chọn kiến trúc cần dựa trên độ phức tạp

của nhiệm vụ, nguồn lực triển khai và khả năng theo dõi, đánh giá quá trình thực hiện.

Trong phạm vi đồ án, kiến trúc Single-Agent được lựa chọn để tập trung nghiên cứu vòng điều phối, quản lý ngữ cảnh, sử dụng công cụ và ghi nhận kết quả trên ứng dụng web. Cách tổ chức này phù hợp với mục tiêu xây dựng nguyên mẫu và phạm vi thực nghiệm SQLi, XSS, đồng thời thuận lợi cho việc quan sát từng bước xử lý. Đây là lựa chọn theo phạm vi nghiên cứu, không phải kết luận rằng Agent đơn luôn hiệu quả hơn hệ thống đa tác tử.

1.4.4. Thách thức và xu hướng phát triển

a) Thách thức

Mặc dù các nghiên cứu về ứng dụng AI tạo sinh trong kiểm thử xâm nhập đã đạt được nhiều kết quả tích cực, việc xây dựng và triển khai các hệ thống AI Pentest Agent trong thực tế vẫn đối mặt với nhiều thách thức về độ tin cậy, khả năng suy luận, quản lý ngữ cảnh, tích hợp công cụ và chi phí triển khai.

Độ tin cậy của mô hình là một thách thức lớn khi ứng dụng mô hình ngôn ngữ lớn vào kiểm thử xâm nhập. Hiện tượng hallucination có thể khiến mô hình đưa ra nhận định không chính xác về lỗ hổng, đề xuất payload không phù hợp hoặc diễn giải sai kết quả từ công cụ kiểm thử. Các sai lệch này có thể dẫn đến việc bỏ sót lỗ hổng nghiêm trọng, phát sinh cảnh báo sai hoặc lựa chọn phương pháp khai thác không phù hợp. Vì vậy, kết quả do hệ thống sinh ra cần được kiểm chứng và xác thực trước khi áp dụng vào kiểm thử thực tế.

Khả năng suy luận và lập kế hoạch nhiều bước là yêu cầu quan trọng vì kiểm thử xâm nhập gồm nhiều giai đoạn liên kết chặt chẽ, từ thu thập thông tin, phân tích bề mặt tấn công, xác định lỗ hổng đến khai thác và hậu khai thác. Hệ thống cần duy trì suy luận, lập kế hoạch và điều chỉnh chiến lược dựa trên kết quả của từng giai đoạn trước đó. Mặc dù các kiến trúc AI Agent đã cải thiện khả năng xử lý chuỗi nhiệm vụ phức tạp, việc xây dựng quá trình suy luận ổn định và nhất quán trong các tình huống thực tế vẫn là một bài toán khó.

Quản lý ngữ cảnh và bộ nhớ gặp khó khăn do hệ thống phải xử lý lượng lớn thông tin trong suốt quá trình kiểm thử, như kết quả quét, cấu trúc mạng, nhật ký hoạt động và trạng thái của từng nhiệm vụ. Giới hạn cửa sổ ngữ cảnh của mô hình ngôn ngữ lớn có thể khiến hệ thống mất thông tin quan trọng hoặc đưa ra quyết định thiếu nhất quán. Với kiến trúc Multi-Agent, việc đồng bộ bộ nhớ và chia sẻ ngữ cảnh giữa nhiều tác tử cũng làm tăng độ phức tạp trong thiết kế và vận hành.

Tích hợp và điều phối công cụ kiểm thử ảnh hưởng lớn đến hiệu quả của AI Pentest Agent khi tương tác với các công cụ bảo mật như Nmap, Burp Suite, Metasploit, Nikto hay SQLMap. Việc chọn công cụ không phù hợp, truyền tham số sai hoặc phân tích chưa chính xác kết quả đầu ra có thể làm gián đoạn quá trình kiểm thử. Vì vậy, xây dựng cơ chế Tool Calling ổn định, linh hoạt và có khả năng mở rộng vẫn là một thách thức kỹ thuật quan trọng.

Kiểm soát an toàn và phạm vi thực thi là yêu cầu cần được chú trọng khi hệ thống được trao quyền chủ động thực hiện hành động trên mục tiêu. Những thao tác vượt ngoài phạm vi được cấp phép hoặc ảnh hưởng đến tính sẵn sàng của hệ thống cần được kiểm soát chặt chẽ. Nhiều nghiên cứu áp dụng cơ chế Human-in-the-Loop, trong đó chuyên gia xác nhận các hành động quan trọng trước khi thực thi nhằm đảm bảo an toàn và tuân thủ phạm vi kiểm thử.

Chi phí tính toán và yêu cầu tài nguyên tăng lên khi AI Pentest Agent thực hiện nhiều lượt gọi mô hình để phân tích thông tin, lập kế hoạch, sinh lệnh và đánh giá kết quả. Điều này làm tăng chi phí API hoặc yêu cầu tài nguyên khi triển khai mô hình cục bộ. Các thành phần RAG, Memory và Multi-Agent còn bổ sung nhu cầu xử lý, lưu trữ và trao đổi dữ liệu. Vì vậy, thiết kế hệ thống cần cân đối chất lượng xử lý, thời gian thực hiện và nguồn lực vận hành, đặc biệt trong điều kiện tài nguyên hạn chế.

Sự khác biệt về môi trường và tiêu chí đánh giá gây khó khăn cho việc so sánh trực tiếp giữa các công trình. Các nghiên cứu được đánh giá trên những môi trường như DVWA, OWASP Juice Shop, VulHub, Hack The Box hoặc các bài toán CTF, với mục tiêu, tập tác vụ, quyền sử dụng công cụ và tiêu chí đánh giá khác nhau. Để đánh giá mức cải thiện của một giải pháp, cần mô tả rõ điều kiện thực nghiệm và xem xét đồng thời kết quả kiểm thử, độ tin cậy, thời gian và chi phí thực hiện.

b) Xu hướng phát triển

Các hướng phát triển của AI Pentest Agent tập trung vào việc bổ sung tri thức bảo mật, cải thiện quản lý ngữ cảnh, nâng cao khả năng tự chủ và tích hợp vào quy trình phát triển phần mềm.

Tích hợp Retrieval-Augmented Generation (RAG) [13] với các nguồn tri thức bảo mật như CVE, CWE, CAPEC, MITRE ATT&CK, NVD hoặc ExploitDB cho phép hệ thống bổ sung thông tin liên quan trong quá trình suy luận. Hướng này hỗ trợ phân tích lỗ hổng và lựa chọn phương án kiểm thử, đồng thời giảm sự phụ thuộc vào kiến thức sẵn có của mô hình. Tuy nhiên, chất lượng truy hồi và mức phù hợp

của nguồn dữ liệu vẫn cần được kiểm soát, vì thông tin được bổ sung không thay thế bằng chứng kiểm thử trên mục tiêu.

Cải thiện quản lý bộ nhớ và ngữ cảnh hướng tới phát triển các cơ chế bộ nhớ ngắn hạn, dài hạn và chia sẻ nhằm duy trì trạng thái xuyên suốt quá trình kiểm thử. Việc quản lý ngữ cảnh tốt hơn giúp hệ thống xử lý hiệu quả hơn các nhiệm vụ nhiều bước và khắc phục một phần giới hạn về cửa sổ ngữ cảnh của mô hình ngôn ngữ lớn.

Nâng cao khả năng tự chủ trong thực thi tập trung vào việc tăng cường khả năng lập kế hoạch, lựa chọn công cụ và điều chỉnh chiến lược kiểm thử theo từng tình huống cụ thể. Xu hướng này vẫn đi kèm các cơ chế kiểm soát nhằm đảm bảo an toàn và hạn chế rủi ro trong quá trình thực thi.

Tích hợp vào quy trình DevSecOps và CI/CD là một hướng nghiên cứu trong bối cảnh phát triển phần mềm hiện đại. Việc tích hợp cho phép tự động kích hoạt các tác vụ đánh giá bảo mật trong từng giai đoạn phát triển phần mềm, góp phần rút ngắn thời gian phát hiện và xử lý lỗ hổng.

Từ những phân tích trên có thể thấy, mặc dù các nghiên cứu đã đạt được nhiều kết quả tích cực trong việc hỗ trợ và tự động hóa quy trình kiểm thử xâm nhập, việc xây dựng một hệ thống AI Pentest Agent có khả năng lập kế hoạch nhiều bước, quản lý ngữ cảnh xuyên suốt phiên làm việc và tích hợp hiệu quả với các công cụ kiểm thử vẫn còn là những vấn đề cần tiếp tục nghiên cứu.

1.5. Kết luận chương 1

Chương 1 đã trình bày cơ sở lý thuyết và tổng quan nghiên cứu liên quan đến đề tài ứng dụng AI tạo sinh trong kiểm thử xâm nhập. Trước hết, chương đã hệ thống hóa các khái niệm, quy trình và phương pháp kiểm thử xâm nhập, đồng thời phân tích các khung tiêu chuẩn và những lỗ hổng bảo mật phổ biến trên ứng dụng web. Qua đó, các hạn chế của phương pháp kiểm thử truyền thống đã được chỉ ra, làm rõ nhu cầu ứng dụng AI nhằm nâng cao mức độ tự động hóa và hiệu quả kiểm thử.

Bên cạnh đó, chương đã giới thiệu nền tảng của AI tạo sinh và mô hình ngôn ngữ lớn, cùng các kỹ thuật quan trọng như Prompt Engineering, Chain-of-Thought, ReAct, Tool Calling và AI Agent. Đồng thời, các hướng nghiên cứu ứng dụng AI tạo sinh trong kiểm thử xâm nhập đã được khảo sát, so sánh và đánh giá, từ mô hình AI hỗ trợ chuyên gia đến các hệ thống AI Agent và Multi-Agent, qua đó làm rõ những ưu điểm, hạn chế, thách thức và xu hướng phát triển của lĩnh vực này.

Những nội dung đã trình bày trong chương là cơ sở khoa học và thực tiễn để xây dựng hệ thống AI Pentest Agent. Trên nền tảng đó, Chương 2 sẽ tập trung phân tích yêu cầu, đề xuất kiến trúc và thiết kế các thành phần của hệ thống AI Pentest Agent hỗ trợ kiểm thử xâm nhập ứng dụng web.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG AI PENTEST AGENT

2.1. Mô tả bài toán

Đồ án đặt ra bài toán xây dựng KAgent, một AI Pentest Agent hỗ trợ đánh giá bảo mật ứng dụng web. Hệ thống cần tiếp nhận yêu cầu, phân tích mục tiêu và trạng thái hiện tại, lựa chọn công cụ rồi sử dụng kết quả thực thi để xác định bước kiểm tra tiếp theo.

Đầu vào gồm yêu cầu của người kiểm thử, thông tin mục tiêu và dữ liệu thu thập trong quá trình kiểm tra. LLM phối hợp với cơ chế quản lý trạng thái và công cụ để hỗ trợ ra quyết định. Đầu ra gồm thông tin khảo sát, kết quả kiểm tra và các phát hiện được ghi nhận cùng bằng chứng liên quan.

KAgent tổ chức xử lý theo nguyên lý suy luận - hành động - quan sát của ReAct. Các bước không tạo thành chuỗi cố định mà được điều chỉnh theo phản hồi công cụ và tiến độ đã ghi nhận, qua ba hoạt động chính: lập kế hoạch, thực hiện hành động và cập nhật trạng thái từ kết quả.

Hệ thống hoạt động trong phạm vi được cấp quyền, với trọng tâm thực nghiệm SQLi và XSS. Người kiểm thử thiết lập mục tiêu, theo dõi tiến trình và xác nhận thao tác khi cần. Agent đảm nhiệm phân tích và điều phối, đồng thời phải duy trì ngữ cảnh, trạng thái và các điều kiện kiểm soát thực thi.

2.2. Các yêu cầu chức năng và yêu cầu phi chức năng

Từ bài toán đã được xác định tại mục 2.1, hệ thống AI Pentest Agent cần đáp ứng các yêu cầu về chức năng và chất lượng nhằm hỗ trợ người kiểm thử trong quá trình thực hiện kiểm thử bảo mật. Các yêu cầu này là cơ sở để thiết kế kiến trúc và các thành phần của hệ thống.

2.2.1. Yêu cầu chức năng

Các yêu cầu chức năng của hệ thống AI Pentest Agent được trình bày trong Bảng 2.1.

Bảng 2.1. Các yêu cầu chức năng của hệ thống

Mã	Tên yêu cầu	Mô tả
FR01	Tiếp nhận và xử lý yêu cầu kiểm thử	Cho phép người kiểm thử nhập mục tiêu và yêu cầu kiểm thử. Hệ thống phân tích thông tin đầu vào để xác định nhiệm vụ cần thực hiện.

Mã	Tên yêu cầu	Mô tả
FR02	Quản lý phiên, ngữ cảnh và trạng thái	Lưu trữ và duy trì thông tin cần thiết trong phiên kiểm thử, bao gồm mục tiêu, lịch sử tương tác, hành động và kết quả đã thu thập.
FR03	Phân tích và lập kế hoạch	Phân tích trạng thái hiện tại và thông tin đã thu thập để xác định hành động tiếp theo. Kế hoạch có thể được điều chỉnh khi có thông tin mới.
FR04	Lựa chọn và thực thi công cụ	Lựa chọn công cụ phù hợp với nhiệm vụ và thực hiện công cụ với các tham số tương ứng.
FR05	Tiếp nhận và phân tích kết quả	Tiếp nhận kết quả từ công cụ và phân tích những thông tin cần thiết cho các bước xử lý tiếp theo.
FR06	Kiểm soát quyền và xác nhận thao tác	Kiểm tra quyền thực thi đối với các hành động và yêu cầu người kiểm thử xác nhận khi cần thiết. Người kiểm thử có thể can thiệp vào quá trình thực thi.
FR07	Quản lý kết quả và phát hiện bảo mật	Ghi nhận và quản lý các kết quả, thông tin và phát hiện bảo mật trong quá trình kiểm thử.
FR08	Tương tác và giám sát	Cho phép người kiểm thử trao đổi với Agent và theo dõi quá trình thực hiện cũng như kết quả trả về.
FR09	Xử lý lỗi trong quá trình thực thi	Xử lý các lỗi phát sinh khi thực hiện tác vụ hoặc gọi công cụ, đồng thời hỗ trợ quá trình tiếp tục hoặc điều chỉnh khi cần.

2.2.2. Yêu cầu phi chức năng

Các yêu cầu phi chức năng của hệ thống AI Pentest Agent được trình bày trong Bảng 2.2.

Bảng 2.2. Các yêu cầu phi chức năng của hệ thống

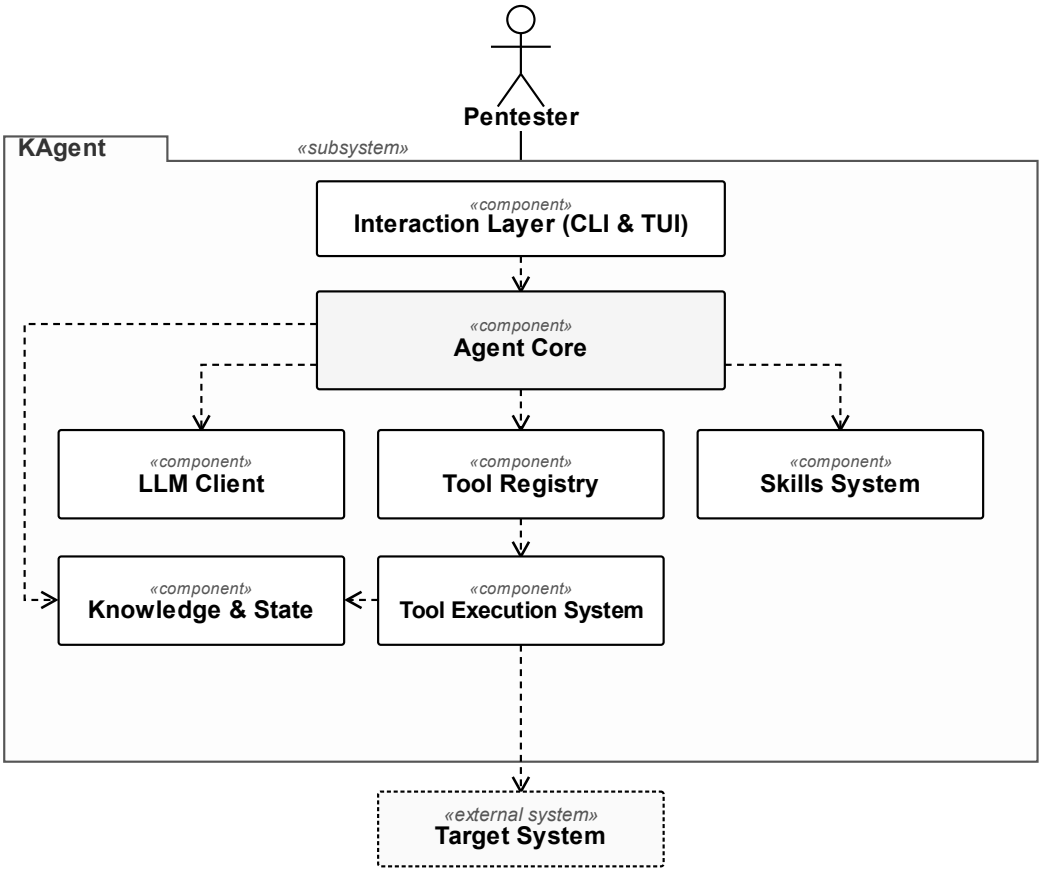
Mã	Tên yêu cầu	Mô tả
NFR01	An toàn và kiểm soát phạm vi	Đảm bảo Agent chỉ thực hiện các hành động trong phạm vi mục tiêu và quyền được cấp phép. Các thao tác cần xác nhận phải được kiểm soát trước khi thực hiện.
NFR02	Độ chính xác và tin cậy của kết quả	Kết quả phân tích cần dựa trên thông tin thực tế thu được từ công cụ và quá trình kiểm thử. Hệ thống cần hạn chế ảnh hưởng của hiện tượng hallucination trong quá trình ra quyết định.
NFR03	Độ tin cậy và khả năng chịu lỗi	Hệ thống cần duy trì trạng thái phiên khi xảy ra lỗi trong quá trình thực thi. Lỗi của một công cụ hoặc tác vụ không nên làm mất toàn bộ thông tin đã lưu khi có thể xử lý và phục hồi.

Mã	Tên yêu cầu	Mô tả
NFR04	Hiệu năng	Đảm bảo thời gian phản hồi phù hợp với quá trình tương tác và xử lý lượng ngữ cảnh phát sinh trong phiên kiểm thử một cách hợp lý.
NFR05	Khả năng mở rộng	Cho phép bổ sung hoặc thay thế công cụ và các thành phần hỗ trợ mà không cần thay đổi lớn trong cơ chế hoạt động của Agent.
NFR06	Khả năng bảo trì	Các thành phần của hệ thống được tổ chức rõ ràng, thuận tiện cho việc sửa đổi, kiểm thử, gỡ lỗi và phát triển.
NFR07	Khả năng quan sát và truy vết	Ghi nhận quá trình xử lý, hoạt động của công cụ và kết quả thực thi để người kiểm thử có thể theo dõi và xem lại khi cần.
NFR08	Khả năng sử dụng	Giao diện hỗ trợ người kiểm thử nhập yêu cầu, theo dõi tiến trình, xem kết quả và thực hiện các thao tác xác nhận hoặc can thiệp.

2.3. Thiết kế kiến trúc hệ thống

2.3.1. Kiến trúc tổng quát hệ thống

Hệ thống KAgent được tổ chức thành các thành phần có chức năng và trách nhiệm khác nhau. Kiến trúc tổng quát của hệ thống được trình bày trong Hình 2.1, thể hiện sự phân tách giữa giao tiếp với người kiểm thử, điều phối của tác nhân AI, các thành phần hỗ trợ và thực thi hoạt động kiểm thử.



Hình 2.1. Kiến trúc tổng quát hệ thống

KAgent được thiết kế theo kiến trúc Single-Agent, trong đó một Agent Core điều phối các thành phần hỗ trợ và công cụ kiểm thử. Ở mức kiến trúc tổng quát, hệ thống được xem xét trong mối quan hệ giữa ba đối tượng chính gồm Pentester, KAgent và Target System. Pentester tương tác với KAgent thông qua Interaction Layer. Bên trong KAgent, Agent Core giữ vai trò điều phối trung tâm, sử dụng các thành phần hỗ trợ để xử lý yêu cầu, quản lý tiến trình kiểm thử, lựa chọn hành động phù hợp và tổ chức quá trình thực thi trên Target System.

Interaction Layer (CLI & TUI) là thành phần giao tiếp trực tiếp với Pentester. Lớp này cung cấp giao diện dòng lệnh (CLI) và giao diện văn bản tương tác (TUI), cho phép người dùng nhập yêu cầu kiểm thử, theo dõi quá trình thực hiện và nhận kết quả. Interaction Layer chủ yếu chịu trách nhiệm tiếp nhận và hiển thị thông tin, không trực tiếp thực hiện các thao tác kiểm thử.

Agent Core là thành phần trung tâm của KAgent. Thành phần này tiếp nhận yêu cầu từ Interaction Layer, xây dựng ngữ cảnh xử lý, xác định trạng thái hiện tại của quá trình kiểm thử và điều phối các bước tiếp theo. Agent Core kết nối khả năng suy luận của mô hình ngôn ngữ với các hoạt động thực thi thông qua các thành phần

hỗ trợ của hệ thống, đồng thời truy xuất thông tin từ Knowledge & State để bổ sung trạng thái phiên làm việc và các kết quả đã tích lũy vào ngữ cảnh.

Các thành phần hỗ trợ chính của Agent Core gồm:

LLM Client chịu trách nhiệm kết nối KAgent với các mô hình ngôn ngữ lớn (LLM). Thành phần này xử lý việc gửi yêu cầu, nhận phản hồi và chuyển đổi kết quả về dạng phù hợp để Agent Core tiếp tục xử lý. Việc tách LLM Client thành một thành phần riêng giúp KAgent có thể hỗ trợ nhiều nhà cung cấp và mô hình khác nhau mà không phải thay đổi phần xử lý chính.

Skills System quản lý các kỹ năng được sử dụng trong quá trình kiểm thử. Mỗi kỹ năng cung cấp hướng dẫn, quy trình xử lý và các yêu cầu cần thiết cho một nhóm nhiệm vụ cụ thể, giúp Agent Core bổ sung kiến thức, lựa chọn cách thức thực hiện phù hợp và định hướng các bước xử lý trong từng tình huống kiểm thử.

Tool Registry quản lý và cung cấp thông tin về các công cụ mà KAgent có thể sử dụng, hỗ trợ Agent Core tra cứu và lựa chọn công cụ phù hợp với nhiệm vụ. Việc đăng ký và tra cứu công cụ được tập trung tại đây, tạo cơ chế thống nhất cho toàn hệ thống, giúp việc bổ sung hoặc thay thế công cụ trong quá trình phát triển không ảnh hưởng đáng kể đến các thành phần khác của hệ thống.

Tool Execution System thực hiện các thao tác kiểm thử sau khi công cụ được lựa chọn. Thành phần này tiếp nhận yêu cầu thực thi, thực hiện thao tác tương ứng và kết nối hoạt động của KAgent với Target System. Sau khi thao tác hoàn tất, kết quả thực thi được chuyển về hệ thống để tiếp tục phân tích, cập nhật trạng thái và phục vụ các bước xử lý tiếp theo.

Knowledge & State quản lý trạng thái và các thông tin được tích lũy trong quá trình hệ thống hoạt động, bao gồm trạng thái phiên làm việc, phạm vi kiểm thử, các đầu vào đã phát hiện, các điểm nghi ngờ cần kiểm tra, kết quả xác thực, bằng chứng, kết quả phát hiện và thông tin về mức độ bao phủ kiểm thử. Thành phần này cung cấp khả năng lưu trữ và truy xuất dữ liệu cần thiết cho quá trình xử lý, hỗ trợ Agent Core xác định trạng thái hiện tại và tiếp tục các bước kiểm thử tiếp theo.

Về mặt quan hệ giữa các thành phần, Pentester tương tác với Interaction Layer, sau đó Interaction Layer chuyển yêu cầu đến Agent Core để xử lý. Agent Core sử dụng LLM Client để tương tác với mô hình ngôn ngữ, sử dụng Skills System khi cần bổ sung kiến thức và quy trình kiểm thử, đồng thời thông qua Tool Registry để lựa chọn công cụ. Sau khi công cụ được lựa chọn, yêu cầu thực thi được chuyển đến Tool Execution System để thực hiện thao tác tương ứng trên Target System. Trong quá trình hoạt động, kết quả thực thi được cập nhật vào Knowledge & State,

còn Agent Core có thể truy xuất các trạng thái, kết quả và thông tin đã được tích lũy tại đây để bổ sung vào ngữ cảnh và xác định các bước xử lý tiếp theo. Do đó, Knowledge & State vừa là nơi lưu trữ trạng thái và kết quả của quá trình kiểm thử, vừa cung cấp dữ liệu cần thiết cho hoạt động điều phối của Agent Core.

2.3.2. Kiến trúc các thành phần của hệ thống

Dựa trên kiến trúc tổng quát ở Mục 2.3.1, các thành phần bên trong KAgent được tổ chức thành tám nhóm với trách nhiệm riêng. Các nhóm này phối hợp để tiếp nhận yêu cầu, tổ chức quá trình suy luận, lựa chọn và thực thi công cụ, duy trì trạng thái kiểm thử và tổng hợp kết quả. Cách phân chia dưới đây làm cơ sở cho thiết kế luồng xử lý và các mô-đun chức năng ở những mục tiếp theo.

Lớp tương tác (Interaction Layer) gồm CLI và TUI, đóng vai trò kết nối người kiểm thử với hệ thống. CLI tiếp nhận tham số và cấu hình, khởi tạo các thành phần cần thiết và xử lý lệnh điều khiển. Trong phiên làm việc, TUI hiển thị diễn biến, tiếp nhận yêu cầu và các quyết định của người kiểm thử. Yêu cầu được chuyển tới Agent Core để xử lý, còn kết quả được đưa trở lại giao diện. Sự phân chia này tách trách nhiệm giao tiếp khỏi việc phân tích và thực thi kiểm thử.

Thành phần điều phối (Agent Core) quản lý quá trình xử lý yêu cầu thông qua Agent Loop, Decision Planner, Prompt & Context, Event System và Sanitization. Agent Loop duy trì vòng xử lý và theo dõi điều kiện dừng. Decision Planner đề xuất hướng kiểm thử dựa trên mục tiêu, phạm vi và trạng thái hiện tại, còn Prompt & Context chuẩn bị dữ liệu gửi tới LLM. Event System chuyển các sự kiện xử lý giữa các thành phần để cập nhật tiến trình. Sanitization xử lý nội dung nhạy cảm trước khi nội dung được sử dụng trong phiên. Các thành phần này giúp Agent Core phối hợp việc suy luận, sử dụng công cụ và theo dõi trạng thái kiểm thử.

Lớp giao tiếp mô hình (LLM Client) cung cấp cơ chế làm việc với các dịch vụ LLM được hỗ trợ. Provider Clients thực hiện giao tiếp với từng dịch vụ, còn Client Factory khởi tạo client theo cấu hình. Model Management quản lý mô hình và tham số sử dụng. Retry & Transport tổ chức truyền nhận và thử lại những lỗi có thể phục hồi. Các kiểu dữ liệu chung cho thông điệp, lời gọi công cụ và phản hồi giúp Agent Core xử lý kết quả thống nhất, đồng thời hạn chế sự phụ thuộc vào đặc điểm giao tiếp riêng của từng dịch vụ.

Hệ thống kỹ năng (Skills System) cung cấp hướng dẫn và quy trình kiểm thử cho Agent. Skill Registry quản lý danh sách kỹ năng cùng thông tin mô tả, Discovery phát hiện kỹ năng trong phạm vi lưu trữ, còn Skill Loading nạp nội dung cần thiết

vào ngữ cảnh. Mỗi kỹ năng xác định các bước xử lý và yêu cầu đối với kết quả. Với một số kỹ năng, hệ thống kiểm tra điều kiện hoàn tất để hỗ trợ Agent theo dõi nhiệm vụ. Nội dung kỹ năng định hướng hành động, còn quyền và giới hạn thực thi được kiểm soát bởi các cơ chế riêng của hệ thống.

Bộ đăng ký công cụ (Tool Registry) cung cấp điểm truy cập thống nhất để Agent tìm kiếm và điều phối công cụ. Tool Lookup quản lý đăng ký và tra cứu, Dispatch xác định loại công cụ rồi chuyển yêu cầu tới cơ chế thực thi tương ứng, còn Alias quản lý tên thay thế và thông tin hiển thị. Thông qua nhóm này, Agent sử dụng công cụ theo giao diện chung mà không phải trực tiếp xử lý cách thực hiện từng thao tác. Việc tra cứu và điều phối công cụ được tách khỏi trách nhiệm thực thi.

Nhóm tri thức và trạng thái (Knowledge & State) duy trì dữ liệu cần thiết cho các bước xử lý tiếp theo. Session & Memory quản lý lịch sử, trạng thái và thông tin phục vụ lưu, tải hoặc tiếp tục phiên. Coverage & Findings quản lý mức bao phủ và các phát hiện, phối hợp với dữ liệu quy trình, xác thực và bằng chứng. Intelligence & Target cung cấp tri thức, thông tin mục tiêu và phạm vi kiểm thử. Các nguồn dữ liệu này hỗ trợ Agent lập kế hoạch, nhận biết công việc đã thực hiện và xác định phần còn lại.

Hệ thống thực thi công cụ (Tool Execution System) thực hiện các thao tác do Tool Registry điều phối. File & Shell hỗ trợ thao tác tệp, tìm kiếm và thực thi lệnh. Network & Web thực hiện tương tác HTTP và web. Security & Findings phục vụ kiểm thử, ghi nhận bằng chứng và phát hiện. Các công cụ thực thi theo điều kiện kiểm soát, sau đó trả kết quả về Agent để phục vụ phân tích và cập nhật tiến trình.

Hạ tầng dùng chung (Shared Infrastructure) cung cấp các dịch vụ cấu hình, kiểm soát quyền, ghi nhật ký và xử lý thông tin nhạy cảm cho toàn hệ thống. Configuration quản lý tham số khởi tạo và vận hành. Permission kiểm tra phạm vi, quyền và giới hạn tài nguyên trước khi công cụ thực hiện thao tác. Logger ghi lại hoạt động để phục vụ theo dõi và truy vết. Redaction che giấu thông tin nhạy cảm trong các dữ liệu được xử lý, với cách áp dụng được phân biệt theo nguồn dữ liệu như trình bày tại Mục 2.4.2.

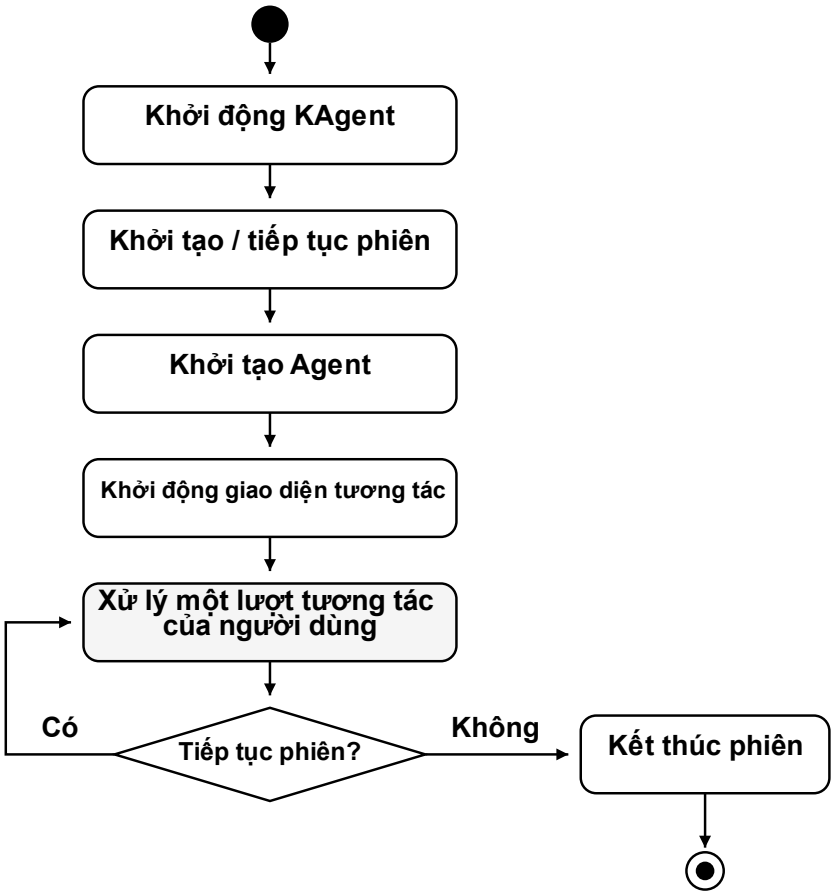
Với cách tổ chức trên, Agent Core giữ vai trò điều phối, các công cụ thực hiện thao tác và Knowledge & State duy trì thông tin phục vụ những bước tiếp theo. Lớp tương tác trình bày tiến trình cho người kiểm thử, còn hạ tầng dùng chung hỗ trợ cấu hình và kiểm soát hoạt động. Quan hệ giữa các nhóm thành phần được cụ thể hóa trong thiết kế luồng xử lý ở Mục 2.4.

2.4. Thiết kế luồng xử lý và cơ chế hoạt động

2.4.1. Quy trình hoạt động của hệ thống

a) Quy trình tổng quan vận hành của hệ thống

Quy trình tổng quan vận hành của KAgent mô tả các bước chính từ khi người dùng khởi động hệ thống, thiết lập phiên làm việc, khởi tạo Agent cho đến khi phiên kết thúc. Quy trình này cho thấy cách KAgent tổ chức một phiên làm việc và xử lý nhiều lượt tương tác của người dùng trong cùng một phiên. Sơ đồ tổng quan quy trình vận hành của hệ thống KAgent được trình bày trong Hình 2.2.



Hình 2.2. Sơ đồ tổng quan quy trình vận hành của hệ thống KAgent

Khởi động KAgent: Người dùng khởi động KAgent thông qua giao diện dòng lệnh, từ đó hệ thống bắt đầu quá trình khởi tạo và chuẩn bị môi trường cho phiên làm việc.

Khởi tạo / tiếp tục phiên: Hệ thống tạo một phiên làm việc mới hoặc tiếp tục một phiên đã được lưu trước đó. Phiên làm việc lưu giữ trạng thái và các thông tin cần thiết trong quá trình người dùng tương tác với hệ thống, qua đó duy trì tính liên tục giữa các lượt xử lý.

Khởi tạo Agent: Hệ thống kết nối Agent với LLM Client, hệ thống kỹ năng, các công cụ đã đăng ký và các thành phần quản lý trạng thái. Sau bước này, Agent có thể tiếp nhận yêu cầu và sử dụng các thành phần đã được chuẩn bị để xử lý nhiệm vụ trong phiên.

Khởi động giao diện tương tác: Hệ thống mở giao diện trên terminal để người dùng nhập yêu cầu, theo dõi tiến trình và xem kết quả. Giao diện cũng tiếp nhận quyết định cho phép hoặc từ chối khi một hành động cần người dùng xác nhận.

Xử lý một lượt tương tác: Khi người dùng gửi yêu cầu, KAgent tiếp nhận và chuyển yêu cầu cho Agent xử lý. Agent phân tích yêu cầu dựa trên ngữ cảnh, trạng thái phiên và các kết quả kiểm thử đã được duy trì, đồng thời có thể sử dụng các kỹ năng và công cụ cần thiết trước khi tạo kết quả trả về cho người dùng.

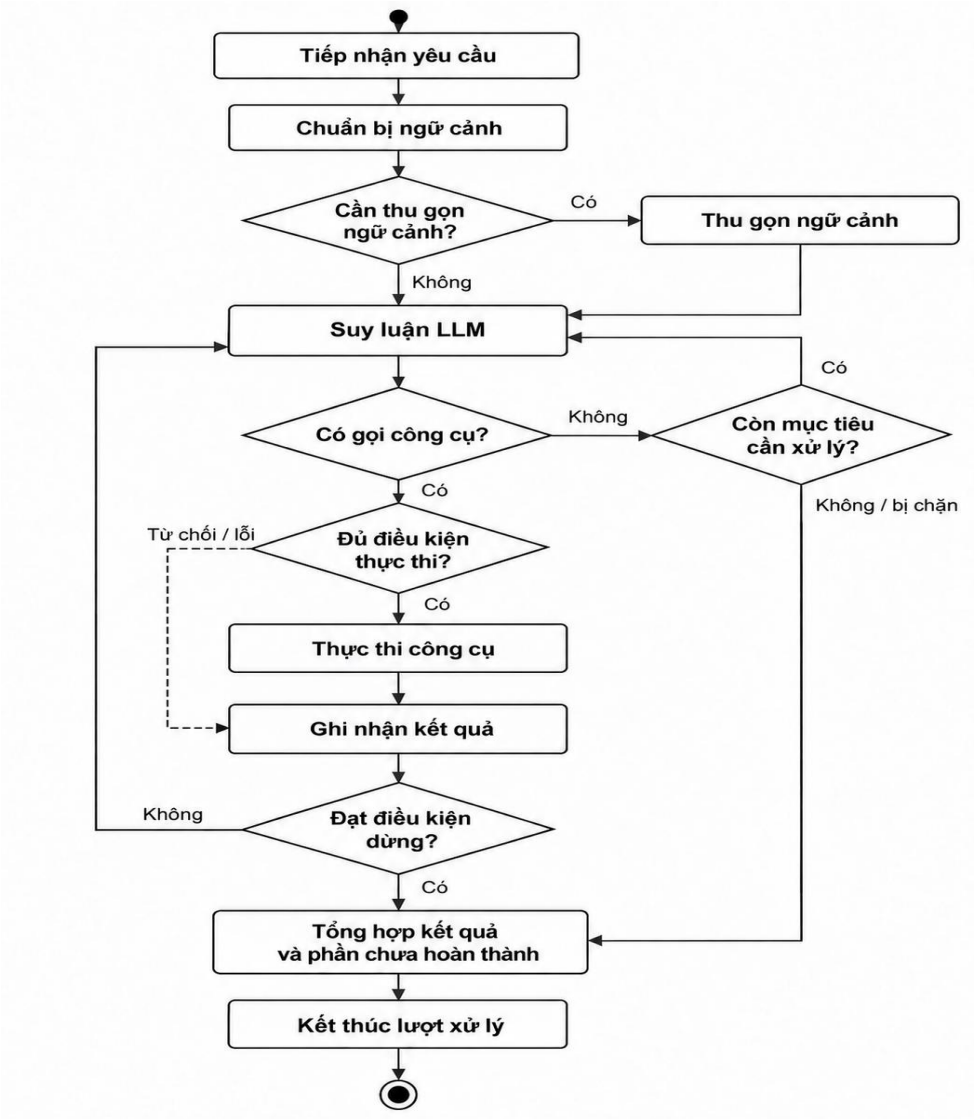
Tiếp tục phiên: Sau khi hoàn thành một lượt tương tác, hệ thống kiểm tra xem phiên làm việc có tiếp tục hay không. Nếu tiếp tục, KAgent quay lại bước Xử lý một lượt tương tác để tiếp nhận yêu cầu tiếp theo. Trạng thái và thông tin của phiên được giữ lại để phục vụ các lượt xử lý sau. Nếu kết thúc, hệ thống chuyển sang bước Kết thúc phiên làm việc.

Kết thúc phiên làm việc: Khi người dùng thoát khỏi KAgent, hệ thống dừng quá trình tương tác, kết thúc các hoạt động theo dõi đang chạy và đóng các kết nối tích hợp đã mở.

Qua quy trình trên, phiên làm việc giúp duy trì trạng thái tương tác giữa người dùng và KAgent, cho phép hệ thống xử lý nhiều lượt yêu cầu trong cùng một phiên mà không cần khởi tạo lại Agent cho từng yêu cầu. Mỗi lượt tương tác được xử lý như một chu trình riêng nhưng vẫn kế thừa trạng thái và các thông tin đã được duy trì trong phiên.

b) Quy trình xử lý một lượt tương tác

Trong một phiên làm việc, mỗi yêu cầu của người dùng được Agent xử lý như một lượt tương tác. Quá trình này bắt đầu khi yêu cầu được tiếp nhận và kết thúc khi Agent tạo phản hồi cuối cùng hoặc khi quá trình xử lý đạt một trong các điều kiện dừng của hệ thống. Quy trình xử lý một lượt tương tác của KAgent được trình bày trong Hình 2.3.



Hình 2.3. Quy trình xử lý một lượt tương tác của KAgent

Lượt tương tác bắt đầu bằng việc tiếp nhận yêu cầu và chuẩn bị ngữ cảnh dựa trên trạng thái hiện tại của phiên, bao gồm lịch sử tương tác, trạng thái kiểm thử và các khả năng hành động. Các mục tiêu kiểm thử được nhận diện từ yêu cầu được duy trì trong workflow. Quá trình khảo sát và phân tích đầu vào hình thành Candidate. Các lượt kiểm tra sau đó thu thập bằng chứng để Agent đánh giá và ghi nhận kết quả trước khi xem xét tạo Finding.

Trước khi gửi yêu cầu tới LLM, KAgent kiểm tra kích thước ngữ cảnh để xác định có cần thu gọn lịch sử hay không. Nếu đạt ngưỡng thu gọn, hệ thống tạo bản tóm lược và kiểm tra mức giảm kích thước trước khi thay thế lịch sử cũ. Nếu chưa cần thu gọn, KAgent tiếp tục suy luận với lịch sử và trạng thái hiện có. Cơ chế này được trình bày tại Mục 2.4.2.

Tại bước suy luận bằng LLM, Agent sử dụng ngữ cảnh hiện tại để tạo phản hồi hoặc yêu cầu thực hiện hành động thông qua các công cụ được cung cấp. Khi

phản hồi không yêu cầu công cụ, hệ thống còn đối chiếu trạng thái workflow và các mục tiêu kiểm thử đã ghi nhận trước khi chấp nhận kết thúc. Nếu vẫn còn công việc có thể xử lý, vòng lặp được tiếp tục. Nếu bị chặn hoặc đạt giới hạn, hệ thống tổng hợp phần đã thực hiện và những nội dung còn chưa hoàn thành.

Đối với hành động yêu cầu sử dụng công cụ, KAgent kiểm tra tính hợp lệ và quyền thực hiện trước khi tiến hành thao tác. Khi đáp ứng các điều kiện cần thiết, công cụ được thực thi và kết quả được ghi nhận. Trường hợp hành động không hợp lệ, không được phép hoặc bị từ chối, hệ thống ghi nhận lý do tương ứng. Tùy theo kết quả và trạng thái hiện tại, quá trình xử lý có thể tiếp tục hoặc kết thúc lượt tương tác theo các điều kiện dừng đã thiết lập.

Sau mỗi yêu cầu gọi công cụ, KAgent ghi nhận kết quả thực thi hoặc lý do không thực hiện và bổ sung thông tin này vào ngữ cảnh. Các thay đổi về điểm đầu vào, Candidate hoặc kết quả kiểm tra được cập nhật vào trạng thái kiểm thử qua các công cụ quản lý tương ứng. Agent sử dụng dữ liệu vừa ghi nhận để lựa chọn bước tiếp theo. Vì vậy, một lượt tương tác có thể gồm nhiều lần suy luận và sử dụng công cụ trước khi tạo phản hồi cuối cùng.

Để hạn chế vòng xử lý kéo dài, KAgent kiểm soát số bước thực hiện. Khi đạt giới hạn số bước, hệ thống chuyển sang tổng hợp kết quả cùng những công việc còn chưa hoàn thành. Nếu gặp trở ngại không thể khắc phục trong phạm vi được phép, lượt xử lý được dừng và trạng thái tương ứng được ghi nhận. Trạng thái phiên đã lưu vẫn được giữ để phục vụ việc xem lại hoặc tiếp tục ở lượt sau.

Nhìn chung, quy trình xử lý một lượt tương tác của KAgent được tổ chức theo vòng lặp suy luận - thực hiện hành động - tiếp nhận kết quả. Kết quả của mỗi bước được ghi nhận và đưa trở lại ngữ cảnh làm việc để Agent tiếp tục suy luận và lựa chọn hành động tiếp theo. Vòng lặp kết thúc khi Agent tạo phản hồi cuối cùng hoặc khi một điều kiện dừng của hệ thống được kích hoạt, qua đó cho phép Agent thực hiện các nhiệm vụ nhiều bước trong cùng một lượt tương tác.

2.4.2. Cơ chế quản lý ngữ cảnh và trạng thái phiên

a) Cấu trúc trạng thái và các nguồn dữ liệu

KAgent duy trì trạng thái phiên để liên kết các lượt tương tác và hỗ trợ kiểm thử nhiều bước. Trạng thái này gồm lịch sử hội thoại, mục tiêu, thông tin đích kiểm thử (Target), bộ nhớ phiên (Session Memory), phạm vi được thiết lập và trạng thái quy trình (Workflow State). Workflow State tổ chức thông tin về tiến độ, đầu vào,

điểm nghi vấn, kết quả kiểm tra và tham chiếu bằng chứng, đồng thời được lưu cùng phiên để sử dụng ở các lượt tiếp theo.

Bảng 2.3. Cấu trúc trạng thái và các nguồn dữ liệu hỗ trợ của KAgent

Nhóm	Thành phần	Vai trò
Trạng thái phiên	Session History	Duy trì nội dung tương tác trong phiên
	Session Memory	Lưu các thông tin cần thiết để tiếp tục xử lý
	Target / Scope	Xác định mục tiêu và phạm vi kiểm thử
	Workflow State	Theo dõi mục tiêu, tiến độ, kết quả kiểm tra và tham chiếu bằng chứng.
Nguồn dữ liệu hỗ trợ	Durable Memory	Cung cấp thông tin được truy hồi khi phù hợp
	Intelligence	Cung cấp thông tin hỗ trợ quá trình phân tích
	Engagement	Cung cấp phạm vi và các điều kiện của quá trình kiểm thử
	Coverage	Theo dõi mức độ bao phủ kiểm thử
	Findings	Lưu các phát hiện đã được xác nhận

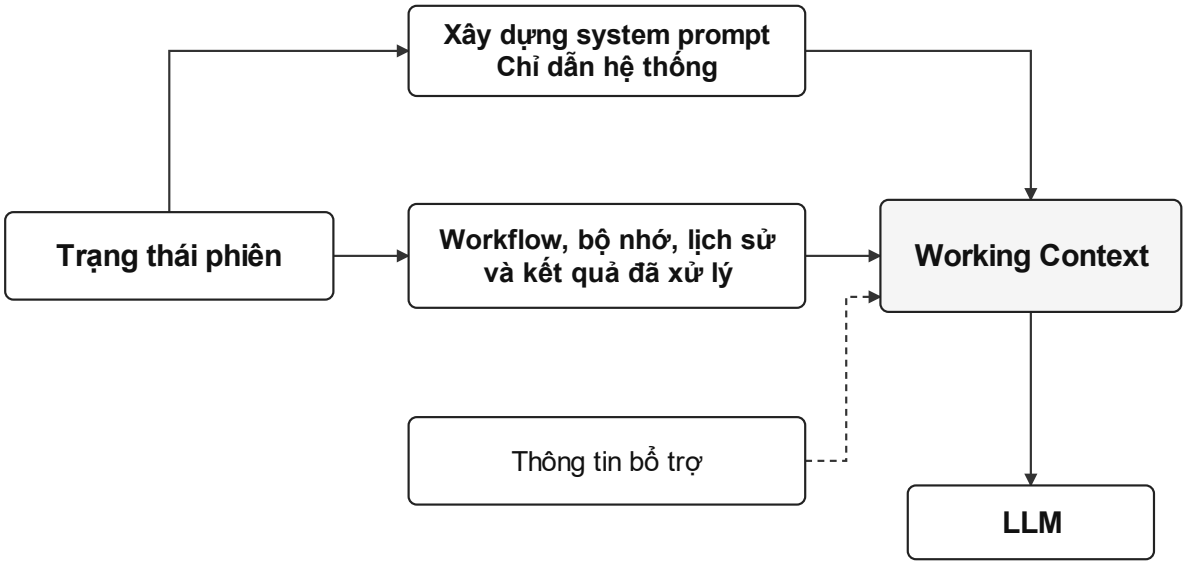
Bên cạnh trạng thái phiên, hệ thống quản lý các nguồn dữ liệu hỗ trợ theo vai trò riêng. Durable Memory lưu thông tin dài hạn có thể truy hồi, Intelligence cung cấp tri thức hỗ trợ phân tích, còn Engagement chứa thông tin phục vụ xây dựng chỉ dẫn hiện tại. Coverage theo dõi mức bao phủ và Findings quản lý các phát hiện đã ghi nhận. Những nguồn này bổ sung dữ liệu cho Agent nhưng không thay thế trạng thái quy trình trong phiên.

b) Xây dựng ngữ cảnh cho một lượt tương tác

Trước mỗi lượt tương tác, KAgent xây dựng ngữ cảnh từ trạng thái và các dữ liệu phù hợp với nhiệm vụ. Chỉ dẫn hệ thống (system prompt) xác định vai trò cùng ràng buộc vận hành, trong khi trạng thái quy trình, bộ nhớ phiên và thông tin truy hồi được cung cấp dưới dạng dữ liệu tham khảo. Sự phân biệt này giúp dữ liệu đã lưu không bị sử dụng như chỉ dẫn của người kiểm thử, quyền thực thi hoặc kết luận đã được chấp nhận.

Ngữ cảnh làm việc (Working Context) kết hợp chỉ dẫn hệ thống, lịch sử hội thoại, yêu cầu hiện tại và thông tin hỗ trợ. Tùy nhiệm vụ, hệ thống bổ sung định hướng từ Decision Planner, tri thức Intelligence và dữ liệu Memory. Nội dung chi

tiết của kỹ năng chỉ được đưa vào khi kỹ năng tương ứng được nạp hoặc kích hoạt, giúp ngữ cảnh tập trung vào công việc đang xử lý.



Hình 2.4. Quy trình xây dựng ngữ cảnh cho một lượt tương tác KAgent

Lịch sử hội thoại giúp LLM theo dõi nội dung trao đổi, còn dữ liệu workflow và bộ nhớ phiên cung cấp thông tin về mục tiêu, tiến độ và kết quả đã ghi nhận. KAgent kết hợp các nguồn này với thông tin truy hồi liên quan đến nhiệm vụ để chuẩn bị đầu vào, đồng thời giới hạn lượng dữ liệu bổ sung vào ngữ cảnh.

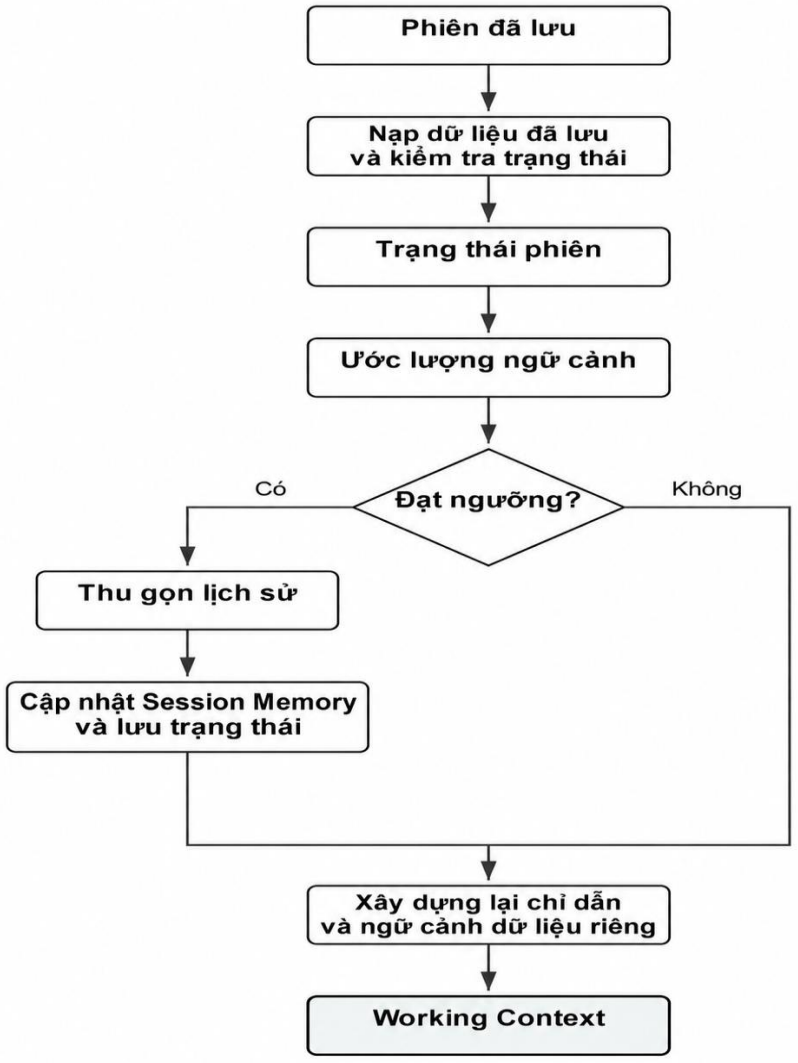
c) Thu gọn ngữ cảnh và khôi phục phiên

Trong quá trình kiểm thử, lịch sử hội thoại tích lũy thêm yêu cầu, phản hồi và kết quả công cụ, làm tăng lượng dữ liệu gửi tới LLM. Khi ngữ cảnh đạt ngưỡng thu gọn đã thiết lập, KAgent tạo bản tóm lược về mục tiêu, tiến độ và công việc còn phải xử lý. Hệ thống ước lượng kích thước ngữ cảnh trước và sau khi thu gọn để kiểm tra mức giảm. Chỉ khi đạt mức giảm tối thiểu, bản tóm lược mới được đưa vào bộ nhớ phiên và sử dụng cùng phần hội thoại gần nhất để thay thế lịch sử cũ. Nếu chưa đạt yêu cầu này, lịch sử cũ được giữ lại.

Để hạn chế lượng dữ liệu đưa vào ngữ cảnh, KAgent lưu các kết quả công cụ dài từ những nguồn được hỗ trợ vào tệp riêng, đồng thời giữ bản xem trước và tham chiếu trong lịch sử hội thoại. Khi kích thước ngữ cảnh tăng cao, hệ thống có thể tiếp tục rút gọn các kết quả công cụ cũ theo chính sách lưu giữ tương ứng, ưu tiên bảo toàn những thông tin quan trọng phục vụ quá trình kiểm thử. Khi cần phân tích thêm, Agent có thể truy xuất phần dữ liệu đã lưu sau khi kiểm tra quyền truy cập. Thông tin nhạy cảm trong kết quả thông thường được che giấu trước khi đưa vào ngữ cảnh. Riêng phản hồi HTTP từ mục tiêu kiểm thử có thể giữ một phần nội dung gốc sau

khi kiểm tra nguồn dữ liệu, phạm vi và quyền hiện hành. Khác với cơ chế tạo bản tóm lược lịch sử hội thoại, việc rút gọn kết quả công cụ chỉ tác động đến ngữ cảnh làm việc, không làm thay đổi các bản ghi trạng thái và bằng chứng kiểm thử được quản lý riêng trong workflow.

Để tiếp tục kiểm thử sau khi phiên làm việc bị gián đoạn, KAgent lưu lịch sử hội thoại, thông tin mục tiêu, bộ nhớ phiên, phạm vi kiểm thử và trạng thái workflow. Khi mở lại phiên, hệ thống nạp các dữ liệu này và xây dựng lại chỉ dẫn cho LLM. Các lượt xác thực chưa kết thúc được ghi nhận là gián đoạn để phân biệt với những lượt đã hoàn tất. Quyền thực thi tạm thời không được khôi phục, vì vậy các thao tác tiếp theo phải được kiểm tra quyền lại trước khi thực hiện.



Hình 2.5. Cơ chế thu gọn ngữ cảnh và khôi phục phiên của KAgent

Bản chụp ngữ cảnh (Context Snapshot) ghi lại thông tin mục tiêu, mô hình đang sử dụng, bộ nhớ phiên và lịch sử hội thoại tại thời điểm tạo bản chụp. Người

vận hành sử dụng bản chụp để xem lại thông tin của phiên và theo dõi quá trình xử lý. Việc khôi phục phiên dựa trên dữ liệu phiên đã lưu, không sử dụng bản chụp này.

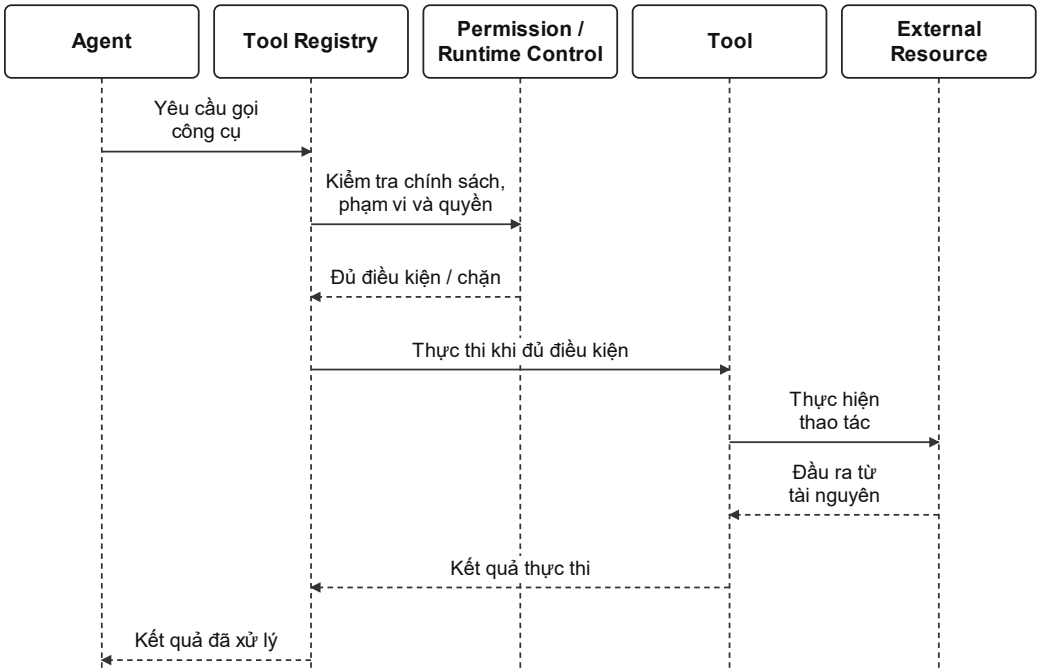
2.4.3. Cơ chế xử lý và thực thi công cụ

Để thực hiện hành động do LLM đề xuất, KAgent tiếp nhận yêu cầu gọi công cụ, kiểm tra tham số và quyền sử dụng, sau đó chuyển yêu cầu tới công cụ đã đăng ký. Kết quả được đưa trở lại Agent làm dữ liệu cho bước suy luận tiếp theo. Phần này trình bày cơ chế xử lý một yêu cầu gọi công cụ và cách điều phối khi phản hồi của LLM chứa nhiều yêu cầu.

a) Xử lý và thực thi một yêu cầu gọi công cụ

Đối với một yêu cầu gọi công cụ, Agent tiếp nhận yêu cầu từ phản hồi của LLM và kiểm tra các thông tin cần thiết trước khi thực hiện. Trong trường hợp lượt tương tác hoặc kỹ năng đang thực hiện áp dụng giới hạn đối với công cụ hay hành động được phép sử dụng, Agent kiểm tra yêu cầu có phù hợp với phạm vi hiện tại hay không trước khi tiếp tục.

Yêu cầu phù hợp được chuyển đến Tool Registry để tra cứu và điều phối tới công cụ tương ứng. Thành phần này tách việc quản lý công cụ khỏi cách thực hiện từng thao tác cụ thể. Nếu không tìm thấy công cụ, hệ thống trả thông tin lỗi về Agent. Cơ chế xử lý một yêu cầu được trình bày trong Hình 2.6.



Hình 2.6. Cơ chế xử lý và thực thi một yêu cầu gọi công cụ của KAgent

Để ngăn công cụ thực hiện thao tác ngoài quyền được cấp, hệ thống kiểm tra phạm vi tài nguyên hoặc mục tiêu, giới hạn thao tác và hiệu lực của quyền hiện hành tùy theo loại công cụ. Các trạng thái từ chối, thu hồi hoặc hết hạn được kiểm tra khi áp dụng. Với hành động cần phê duyệt, người kiểm thử xem thông tin thao tác và quyết định cho phép hoặc từ chối. Chế độ YOLO có thể tự xử lý bước phê duyệt theo chính sách, nhưng các kiểm tra về phạm vi và quyền vẫn được thực hiện. Đối với công cụ thực thi lệnh, hệ thống kiểm tra yêu cầu trước khi khởi chạy tiến trình và kiểm soát thời gian chạy, việc hủy tác vụ cùng lượng đầu ra thu nhận. Các lệnh được thực hiện dưới quyền của tài khoản vận hành. Các tương tác qua công cụ HTTP tích hợp được kiểm tra phạm vi mục tiêu và quyền hiện hành theo cơ chế riêng.

Sau khi các điều kiện cần thiết được thỏa mãn, Tool Registry chuyển yêu cầu đến công cụ tương ứng để thực thi. Một số công cụ có thể áp dụng thêm các bước kiểm tra an toàn hoặc xác thực đầu vào riêng trước khi thực hiện thao tác. Công cụ sau đó tương tác với tài nguyên hoặc mục tiêu phù hợp với chức năng của mình. Kết quả thực thi, bao gồm kết quả thành công hoặc thông tin lỗi, được trả về thông qua cơ chế điều phối và chuyển lại cho Agent.

Agent sử dụng kết quả công cụ để đánh giá thao tác vừa thực hiện và xác định cần kiểm tra thêm hay có thể kết thúc. Những điểm đầu vào mới, Candidate hoặc kết quả xác thực được ghi nhận qua các công cụ quản lý trạng thái. Việc ghi nhận này giúp các bước tiếp theo sử dụng tiến độ đã lưu thay vì chỉ dựa vào phần mô tả trong hội thoại.

b) Xử lý nhiều yêu cầu gọi công cụ

Phản hồi của LLM có thể chứa nhiều yêu cầu gọi công cụ. KAgent xử lý tuần tự khi cần giữ thứ tự hoặc quan hệ phụ thuộc, và có thể thực hiện đồng thời các yêu cầu độc lập khi điều kiện cho phép. Mỗi yêu cầu đều tuân theo cơ chế kiểm tra và thực thi đã trình bày ở phần a).

Với xử lý tuần tự, một yêu cầu được hoàn tất trước khi chuyển sang yêu cầu tiếp theo. Cách tổ chức này phù hợp khi thao tác sau phụ thuộc vào trạng thái hoặc kết quả của thao tác trước, giúp duy trì trình tự cần thiết của quá trình kiểm tra.

Với xử lý đồng thời, những yêu cầu phù hợp được thực hiện song song trong giới hạn đồng thời do hệ thống quy định nhằm giảm thời gian chờ và kiểm soát tài nguyên. Kết quả của từng yêu cầu được thu nhận sau khi hoàn tất và trả về Agent để tiếp tục phân tích. Việc thực hiện đồng thời không bỏ qua kiểm tra phạm vi, quyền hoặc điều kiện riêng của từng công cụ.

Sau khi các yêu cầu gọi công cụ được xử lý, KAgent tổng hợp kết quả hoặc thông tin lỗi tương ứng với từng yêu cầu và chuyển lại cho Agent. Agent sử dụng những thông tin này để đánh giá kết quả thực hiện, cập nhật trạng thái liên quan khi cần thiết và xác định các bước xử lý tiếp theo.

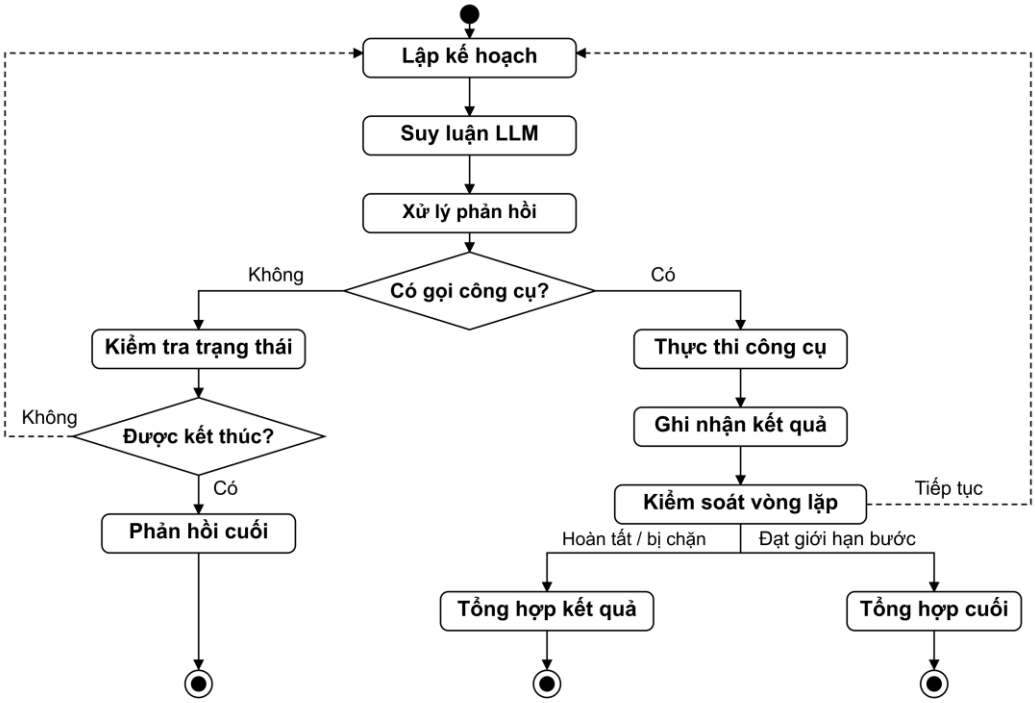
2.5. Thiết kế chi tiết các mô-đun chức năng hệ thống

2.5.1. Mô-đun điều phối và lập kế hoạch tác vụ

a) Cơ chế điều phối vòng lặp Agent

Agent Core điều phối phiên làm việc thông qua vòng xử lý giữa mô hình ngôn ngữ và các công cụ kiểm thử. Từ yêu cầu người dùng, trạng thái mục tiêu và ngữ cảnh hiện có, Agent lựa chọn thao tác, tiếp nhận kết quả rồi cập nhật thông tin phục vụ bước tiếp theo. Vòng lặp này kết nối kế hoạch điều phối với việc thực hiện và theo dõi tiến trình của nhiệm vụ.

Trong quá trình xử lý, Decision Planner cung cấp định hướng dựa trên yêu cầu và trạng thái kiểm thử hiện tại. Định hướng này được cập nhật theo tiến trình để Agent lựa chọn bước tiếp theo, trong khi việc thực thi vẫn chịu các điều kiện về quyền, phạm vi và trạng thái quy trình. Hình 2.7 mô tả vòng xử lý chính của Agent.



Hình 2.7. Cơ chế điều phối vòng lặp Agent

Trong mỗi bước xử lý, Agent gửi ngữ cảnh hiện tại tới LLM và xử lý phản hồi nhận được. Nếu phản hồi chứa yêu cầu gọi công cụ, Agent chuyển các yêu cầu

này tới hệ thống thực thi. Kết quả được ghi nhận và bổ sung vào ngữ cảnh làm việc. Sau đó, Agent kiểm soát vòng lặp để xác định có tiếp tục xử lý hay chuyển sang nhánh kết thúc. Các bước này tương ứng với các nút xử lý phản hồi, thực thi công cụ, ghi nhận kết quả và kiểm soát vòng lặp trong Hình 2.7.

Mỗi bước trong vòng lặp tương ứng với một chu kỳ tương tác với mô hình. Trong cùng một phản hồi, LLM có thể đề xuất nhiều lời gọi công cụ và các lời gọi này vẫn thuộc cùng một bước xử lý. Tuy nhiên, không phải phản hồi nào của mô hình cũng chứa yêu cầu gọi công cụ.

Khi mô hình không phát sinh yêu cầu gọi công cụ, Agent không nhất thiết kết thúc lượt xử lý ngay. Đối với chế độ Whole-target và những nhiệm vụ Direct có các mục tiêu kiểm thử được ghi nhận, Agent đối chiếu trạng thái workflow trước khi chấp nhận phản hồi cuối. Việc hoàn tất một Candidate không đồng nghĩa với hoàn thành nhiệm vụ nếu vẫn còn mục tiêu có thể xử lý. Những mục tiêu bị chặn, tạm hoãn hoặc chưa có đủ ngữ cảnh được thể hiện là nội dung chưa hoàn thành trong phần tổng hợp. Các điều kiện để lựa chọn Candidate cho bước kiểm chứng được trình bày tại Mục 2.5.5.a.

Đối với nhánh có thực thi công cụ, sau khi tiếp nhận kết quả, Agent đối chiếu trạng thái quy trình để xác định bước xử lý tiếp theo. Agent tiếp tục xử lý khi còn công việc khả thi và chưa đạt điều kiện dừng. Khi quy trình hoàn tất, bị chặn do trở ngại thực tế hoặc đạt giới hạn số bước, Agent chuyển sang xử lý kết thúc phù hợp với trạng thái hiện tại. Vòng lặp không tự dừng chỉ vì nhiều bước liên tiếp chưa tạo thêm kết quả. Các trạng thái chi tiết của quy trình được quản lý bởi mô-đun trạng thái và kết quả kiểm thử, trong khi Agent Loop sử dụng các thông tin này để điều phối quá trình xử lý.

Bên cạnh việc theo dõi tiến độ, Agent duy trì giới hạn số bước cho mỗi lượt tương tác. Khi đạt giới hạn, Agent chuyển sang bước tổng hợp cuối. Ở bước này, mô hình tổng hợp các kết quả đã thu được cùng những nội dung còn chưa hoàn thành và không được cung cấp khả năng gọi thêm công cụ.

Ngoài trường hợp đạt giới hạn số bước, các nhánh kết thúc khác phụ thuộc vào trạng thái xử lý. Nếu được phép kết thúc sau phản hồi không gọi công cụ, Agent đưa ra phản hồi cuối. Khi kiểm soát vòng lặp xác định công việc đã hoàn tất, Agent tổng hợp kết quả. Nếu quy trình bị chặn, Agent nêu lý do chưa thể tiếp tục và những công việc chưa hoàn thành.

Như vậy, cơ chế điều phối giúp Agent duy trì tiến trình qua nhiều bước xử lý, cập nhật định hướng theo trạng thái hiện tại và phân biệt việc kết thúc một lượt tương tác với việc hoàn tất toàn bộ mục tiêu kiểm thử.

b) Cơ chế lập kế hoạch của Decision Planner

Decision Planner đề xuất công việc tiếp theo cho Agent dựa trên yêu cầu của người dùng và trạng thái kiểm thử hiện tại. Trước các bước suy luận trong vòng lặp, Planner cập nhật định hướng để phù hợp với tiến độ thực hiện. Kết quả được thể hiện dưới dạng kế hoạch có cấu trúc gọi là DecisionPlan. Agent sử dụng kế hoạch này để lựa chọn bước xử lý, còn các thao tác kiểm thử được thực hiện thông qua hệ thống công cụ.

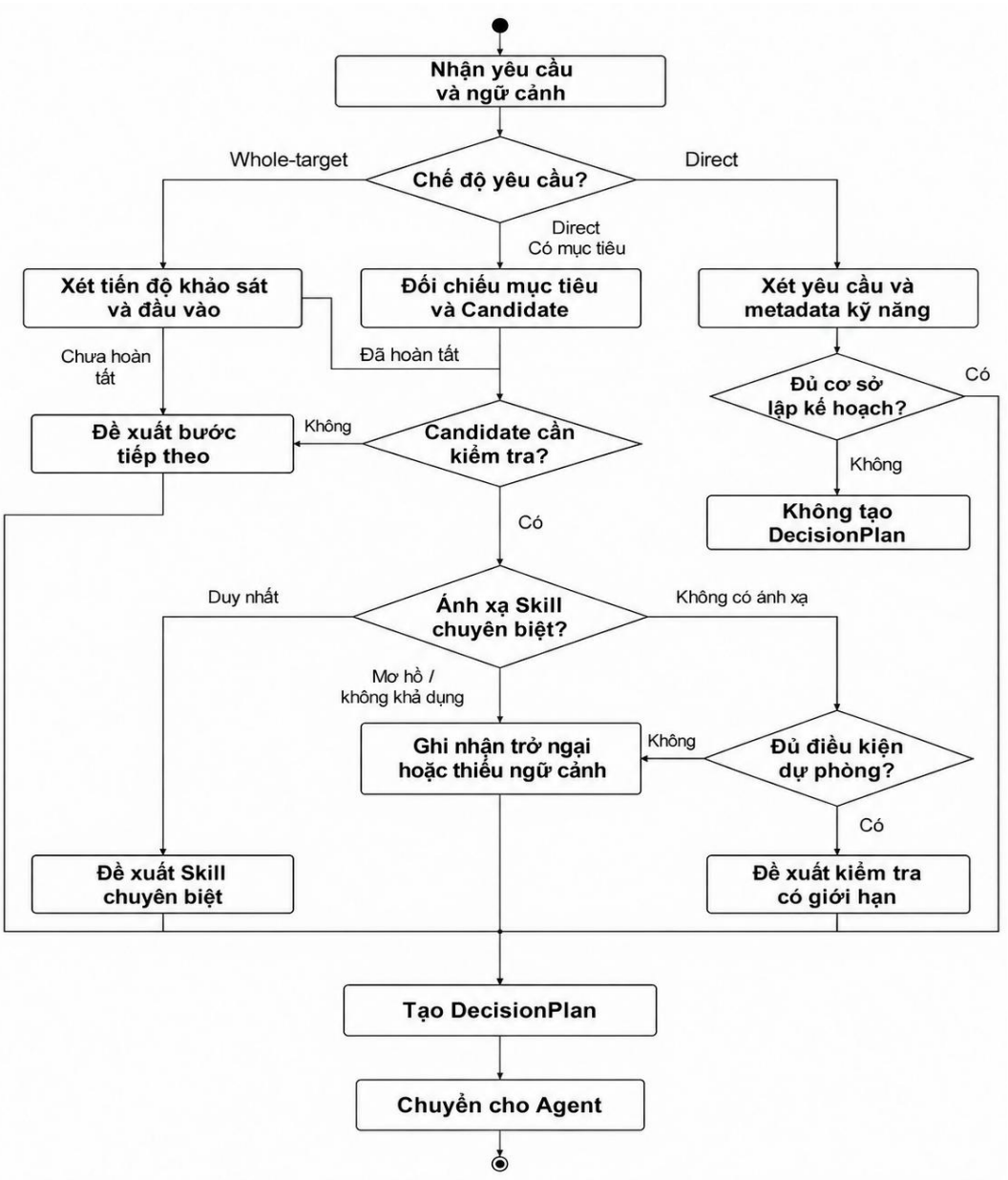
Để lập kế hoạch, Planner sử dụng yêu cầu của người dùng, thông tin đích kiểm thử (Target), các kỹ năng khả dụng và ngữ cảnh lập kế hoạch (PlannerContext). PlannerContext tổng hợp thông tin về kỹ năng đang sử dụng hoặc đã hoàn tất, các giai đoạn đã thực hiện, đầu vào chờ phân tích, các điểm nghi vấn đã ghi nhận (Candidate) và kết quả liên quan. Ngữ cảnh này cũng cho biết chế độ kiểm thử, mục tiêu cần đạt, mức độ bao phủ và những trở ngại hiện có.

Những thông tin trên giúp Planner nhận biết công việc đã thực hiện và phần còn lại để đề xuất bước tiếp theo. Nếu kế hoạch liên quan đến một Candidate cụ thể, trường candidate_id xác định Candidate tương ứng.

Bảng 2.4. Các thành phần của DecisionPlan

Thành phần	Ý nghĩa
recommended_skill	Kỹ năng được đề xuất cho lượt xử lý hiện tại, nếu có
reason	Cơ sở dẫn đến đề xuất của Planner
risk	Mức rủi ro do Planner xác định để bổ sung hướng dẫn kiểm soát
checklist	Các nội dung cần lưu ý trong quá trình xử lý
guidance	Chỉ dẫn định hướng Agent
candidate_id	Định danh Candidate liên quan đến kế hoạch, nếu có

DecisionPlan chỉ mang tính định hướng và không trực tiếp thực hiện các thao tác kiểm thử. Các bước kiểm thử chi tiết vẫn được cung cấp bởi kỹ năng tương ứng khi được nạp, trong khi Agent Core chịu trách nhiệm duy trì vòng lặp và điều phối quá trình thực thi. Quy trình hình thành định hướng xử lý của Decision Planner được khái quát trong Hình 2.8.



Hình 2.8. Quy trình lập kế hoạch của Decision Planner

Decision Planner sử dụng trạng thái có cấu trúc của quy trình để xác định công việc cần thực hiện tiếp theo. Trong chế độ Whole-target, Planner xem xét các pha đã hoàn thành, đầu vào còn chờ phân tích, thông tin bao phủ và Candidate. Ở giai đoạn đầu, Planner ưu tiên các hoạt động thu thập thông tin, liệt kê bề mặt tấn công và phân tích đầu vào. Khi các giai đoạn này hoàn tất, Planner lựa chọn công việc tiếp theo từ các mục tiêu và Candidate đã ghi nhận.

Trong chế độ Direct, Planner đối chiếu các mục tiêu kiểm thử đã ghi nhận cùng Candidate thuộc mục tiêu hiện tại. Nếu chưa có mục tiêu cụ thể, Planner phân tích yêu cầu và siêu dữ liệu của kỹ năng để xác định có đủ cơ sở lập kế hoạch hay không. Việc thiếu ngữ cảnh cụ thể không tự động mở rộng nhiệm vụ sang khảo sát

toàn bộ ứng dụng. Như vậy, kế hoạch được xây dựng theo chế độ kiểm thử và trạng thái hiện tại, thay vì sử dụng cùng một chuỗi kỹ năng cho mọi yêu cầu.

Để lựa chọn kỹ năng phù hợp, Decision Planner phân tích nội dung yêu cầu và đối chiếu với siêu dữ liệu của các kỹ năng được đăng ký. Trước khi đối chiếu, nội dung văn bản được chuẩn hóa nhằm giảm ảnh hưởng của khác biệt về cách viết, ký tự phân cách và khoảng trắng. Việc đề xuất kỹ năng không chỉ dựa trên số lượng từ khóa xuất hiện trong yêu cầu mà còn kết hợp nhiều loại tín hiệu từ siêu dữ liệu (metadata), bao gồm các tín hiệu mạnh và yếu trong triggers, các lớp đối tượng mà kỹ năng có thể xử lý thông qua candidate-classes, pha kiểm thử được mô tả bởi stage và quan hệ phụ thuộc thông qua requires. Các thông tin này được đánh giá cùng trạng thái hiện tại của quy trình để xác định kỹ năng phù hợp.

Khi nhiều kỹ năng cùng khớp với yêu cầu, Planner so sánh mức độ liên quan của các tín hiệu và ngữ cảnh hiện tại. Nếu chưa đủ cơ sở để phân biệt các lựa chọn, Planner không thiết lập kỹ năng đề xuất nhằm tránh đưa ra lựa chọn thiếu căn cứ. Bên cạnh đó, Decision Planner phân biệt các yêu cầu kiểm thử với những truy vấn chỉ mang tính giải thích hoặc cung cấp thông tin. Khi yêu cầu không thể hiện ý định thực hiện hoạt động kiểm thử, Planner bỏ qua việc tạo kế hoạch để Agent xử lý như một tương tác thông thường với mô hình ngôn ngữ.

Đối với Candidate cần kiểm tra, Planner ưu tiên Skill chuyên biệt có ánh xạ duy nhất và khả dụng. Nhánh kiểm tra dự phòng có giới hạn chỉ được xem xét khi thực sự không có ánh xạ Skill chuyên biệt và đề xuất thao tác cùng ngữ cảnh cần thiết đã được chấp nhận theo điều kiện của hệ thống. Nhánh này không được dùng để thay thế Skill đã có hoặc bỏ qua lỗi nạp Skill. Nếu ánh xạ mơ hồ, Skill không khả dụng hoặc điều kiện dự phòng chưa đáp ứng, Planner ghi nhận trở ngại hoặc phần ngữ cảnh còn thiếu. Candidate chưa đủ điều kiện được giữ ở trạng thái chưa thể tiếp tục kiểm tra.

Sau khi xác định hướng xử lý, Decision Planner đánh giá mức độ rủi ro của yêu cầu và xây dựng các thông tin định hướng cho Agent. Mức rủi ro được biểu diễn trong trường risk và được sử dụng để bổ sung các chỉ dẫn kiểm soát phù hợp cho kế hoạch. Việc thực thi và kiểm soát quyền đối với các công cụ được xử lý độc lập bởi các thành phần chuyên trách, không thuộc trách nhiệm trực tiếp của Decision Planner.

Từ kết quả phân tích kỹ năng, trạng thái mục tiêu và ngữ cảnh quy trình, Planner xây dựng checklist chứa các nội dung cần lưu ý trong lượt xử lý. Các nội

dung này được tổng hợp vào guidance để cung cấp định hướng thống nhất cho Agent, thay vì mô tả trực tiếp từng thao tác kiểm thử.

Khi có đủ cơ sở lập kế hoạch, Planner tổng hợp các thông tin định hướng thành DecisionPlan và cung cấp cho Agent để tiếp tục xử lý.

2.5.2. Mô-đun quản lý tri thức và kỹ năng kiểm thử

a) Mô-đun quản lý tri thức

Mô-đun quản lý tri thức (Intelligence) cung cấp cho Agent những thông tin và kinh nghiệm đã được lưu trữ để hỗ trợ quá trình phân tích và kiểm thử. Tri thức được tổ chức dưới dạng các scenario có cấu trúc, mô tả những tình huống, dấu hiệu và kinh nghiệm có thể tái sử dụng khi gặp các nhiệm vụ tương tự. Intelligence chỉ cung cấp thông tin tham khảo, không trực tiếp thực hiện thao tác kiểm thử hoặc quyết định hành động thay cho Agent. Thành phần IntelligenceStore chịu trách nhiệm lưu trữ, tìm kiếm, xếp hạng và quản lý các scenario. Thành phần này cũng hỗ trợ chuẩn hóa, hạn chế trùng lặp và tích lũy tri thức mới. Những scenario phù hợp được lựa chọn để bổ sung vào ngữ cảnh làm việc của Agent, trong khi việc lựa chọn và thực hiện thao tác kiểm thử vẫn do Agent điều phối. Dữ liệu tri thức được tổ chức theo ba phạm vi Built-in, Project và Personal. Trong đó, Built-in chứa các scenario được khai báo sẵn trong hệ thống, còn Project và Personal lưu trữ những scenario được tích lũy trong quá trình sử dụng. Cách phân chia này giúp hệ thống quản lý riêng tri thức có sẵn và tri thức được hình thành trong từng phạm vi sử dụng.

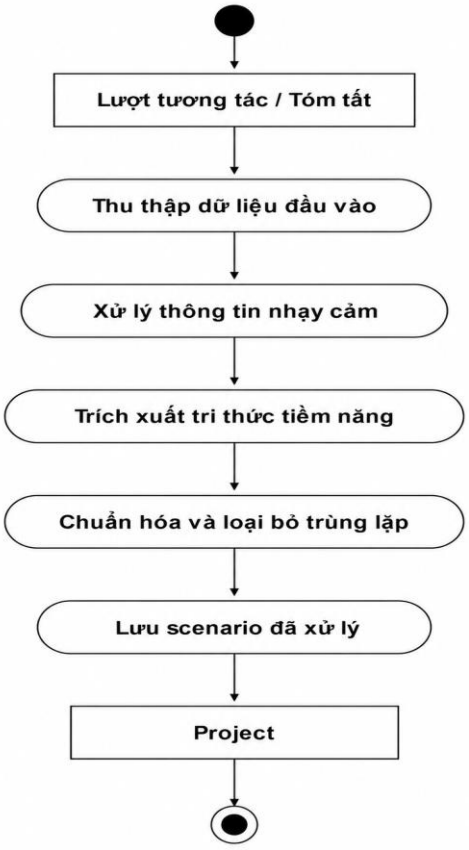
Đơn vị dữ liệu cơ bản của mô-đun là IntelligenceScenario, biểu diễn một tình huống hoặc kinh nghiệm có thể tái sử dụng. Thay vì chỉ lưu trữ dưới dạng văn bản tự do, mỗi scenario được tổ chức thành các nhóm thông tin để phục vụ việc quản lý và truy hồi. Cụ thể, nhóm nội dung tri thức mô tả bản chất của scenario thông qua tiêu đề, phân loại và bài học. Nhóm định hướng kiểm thử chứa các nội dung kiểm tra được khuyến nghị và những điểm cần lưu ý. Nhóm truy hồi và xếp hạng cung cấp các dấu hiệu, công nghệ liên quan và mức độ tin cậy để xác định scenario phù hợp. Nhóm quản lý chứa thông tin định danh, phạm vi và nguồn gốc của scenario. Các nhóm thông tin chính được trình bày trong Bảng 2.5.

Bảng 2.5. Các nhóm thông tin chính của IntelligenceScenario

Nhóm thông tin	Thành phần chính	Vai trò
Nội dung tri thức	title, category, lesson	Mô tả nội dung và bản chất của scenario
Định hướng kiểm thử	recommended_checks, avoid_missing	Cung cấp các nội dung kiểm tra và điểm cần lưu ý
Truy hồi và xếp hạng	triggers, technologies, confidence	Hỗ trợ xác định và xếp hạng scenario phù hợp
Quản lý	id, scope và thông tin nguồn gốc	Định danh, xác định phạm vi và hỗ trợ quản lý scenario

Khi Agent xử lý một nhiệm vụ, KAgent sử dụng thông tin từ lượt tương tác hiện tại để tìm kiếm những scenario có liên quan trong IntelligenceStore. Việc truy hồi dựa trên mức độ phù hợp giữa nội dung truy vấn và các trường thông tin của scenario. Trong đó, những trường thể hiện dấu hiệu trực tiếp như triggers, title và technologies được ưu tiên hơn các trường mô tả hoặc định hướng kiểm thử tổng quát. Sau khi tìm kiếm, IntelligenceStore xếp hạng các scenario dựa trên mức độ phù hợp, độ tin cậy và độ mới của thông tin. Hệ thống chỉ lựa chọn một số scenario phù hợp để bổ sung vào Working Context, thay vì đưa toàn bộ kho tri thức vào ngữ cảnh của mô hình. Trước khi bổ sung vào ngữ cảnh, hệ thống còn kiểm tra nguồn gốc và phạm vi áp dụng của scenario. Vì vậy, không phải mọi thông tin đã lưu đều được tự động sử dụng trong các phiên tiếp theo. Cách làm này giúp Agent tiếp cận thông tin liên quan mà không làm tăng kích thước ngữ cảnh quá mức. Tri thức được truy hồi chỉ hỗ trợ quá trình phân tích, không thay thế kết quả quan sát hoặc bằng chứng thu được từ công cụ kiểm thử.

Bên cạnh việc sử dụng tri thức đã lưu trữ, KAgent còn có thể tích lũy các scenario mới từ dữ liệu hình thành trong quá trình hoạt động. Cơ chế này được kích hoạt khi lượt xử lý có thực thi công cụ hoặc khi hệ thống thực hiện thu gọn lịch sử phiên, thay vì sau mọi lượt tương tác. Quá trình tích lũy tri thức được minh họa trong Hình 2.9.



Hình 2.9. Cơ chế học và tích lũy tri thức

Trước khi trích xuất tri thức, dữ liệu phiên được xử lý nhằm hạn chế lưu trữ thông tin nhạy cảm. Hệ thống sau đó sử dụng các quy tắc và mẫu nhận diện đã xác định để tìm ra những thông tin có khả năng tái sử dụng. Những nội dung liên quan đến quyền hạn hoặc phạm vi tạm thời của phiên được loại trừ để tránh lưu giữ các điều kiện chỉ có hiệu lực trong phiên hiện tại.

Các scenario được trích xuất tiếp tục được chuẩn hóa và kiểm tra trùng lặp trước khi lưu trữ. Việc kiểm tra dựa trên định danh và thông tin mô tả đã được chuẩn hóa nhằm hạn chế lưu nhiều scenario có cùng nội dung. Các scenario được tự động tích lũy được lưu trong phạm vi Project dưới dạng thông tin suy ra từ dữ liệu phiên, không tự động trở thành tri thức Personal hoặc quyền sử dụng cho những phiên làm việc sau.

b) Mô-đun kỹ năng kiểm thử

Bên cạnh mô-đun quản lý tri thức, KAgent sử dụng mô-đun kỹ năng kiểm thử (Skills System) để quản lý các quy trình kiểm thử chuyên biệt. Mỗi kỹ năng được tổ chức dưới dạng tài liệu hướng dẫn (playbook), mô tả phương pháp và các bước khuyến nghị cho một loại nhiệm vụ hoặc tình huống kiểm thử cụ thể. Kỹ năng giúp

Agent lựa chọn cách tiếp cận và công cụ phù hợp, nhưng không trực tiếp thực hiện thao tác kiểm thử hoặc quyết định hành động thay cho Agent.

Skills System chịu trách nhiệm phát hiện, đăng ký, tra cứu và nạp kỹ năng. Thành phần này cung cấp danh mục kỹ năng cho Decision Planner để phục vụ việc lập kế hoạch, đồng thời đưa nội dung kỹ năng đã nạp vào Working Context để Agent tham khảo.

Decision Planner có thể đề xuất kỹ năng phù hợp thông qua DecisionPlan, nhưng việc đề xuất không có nghĩa kỹ năng đó đã được nạp. Skills System vẫn phải kiểm tra các điều kiện sử dụng trước khi nạp kỹ năng. Việc tách biệt hai hoạt động này giúp hệ thống có thể bổ sung hoặc cập nhật kỹ năng mà không cần thay đổi cơ chế điều phối của Agent Core.

Mỗi kỹ năng được tổ chức thành một thư mục riêng, trong đó SKILL.md là tệp mô tả chính. Tệp này gồm phần YAML frontmatter dùng để khai báo thông tin cấu hình và phần thân Markdown chứa nội dung hướng dẫn kiểm thử. Thư mục kỹ năng cũng có thể chứa các tài nguyên bổ trợ như danh sách payload hoặc script hỗ trợ.

Phần YAML frontmatter cung cấp các thuộc tính giúp hệ thống nhận diện, lựa chọn và quản lý kỹ năng. Những thuộc tính chính được chia thành bốn nhóm như trình bày trong Bảng 2.6.

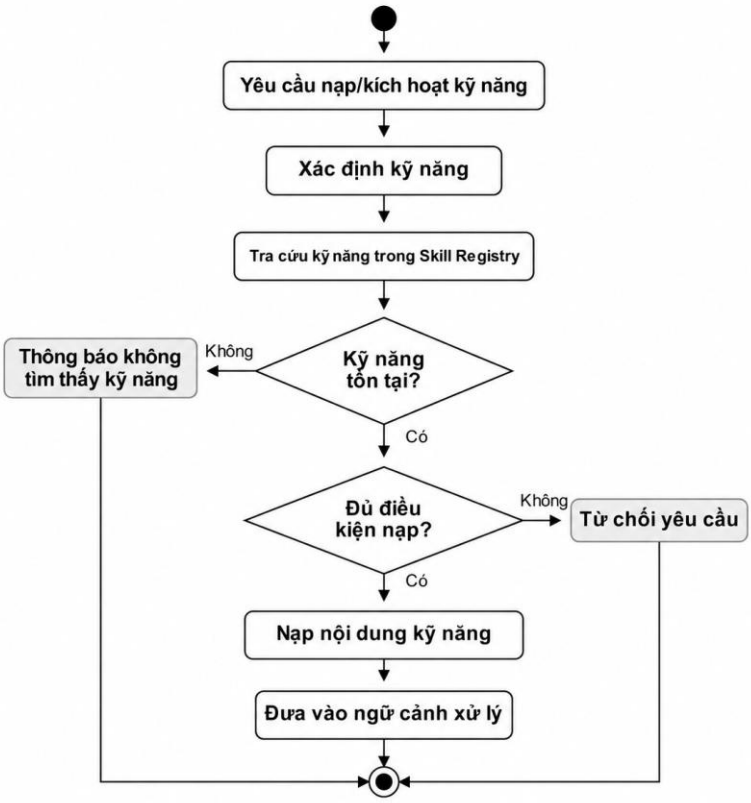
Bảng 2.6. Các nhóm metadata chính của kỹ năng kiểm thử

Nhóm metadata	**Các trường thuộc tính**	**Ý nghĩa thiết kế**
Định danh và giai đoạn	name, description, stage	Định danh kỹ năng, mô tả mục đích sử dụng và xác định giai đoạn kiểm thử tương ứng
Điều phối và tín hiệu	triggers, candidate-classes, requires	Khai báo các tín hiệu nhận diện, lớp đối tượng kiểm thử và quan hệ phụ thuộc phục vụ quá trình đánh giá, đề xuất và điều phối kỹ năng
Kiểm soát thực thi	allowed-tools, disable-model-invocation	Xác định phạm vi công cụ được phép sử dụng và cơ chế nạp kỹ năng từ mô hình
Hoàn tất kỹ năng	completion-artifact	Mẫu đường dẫn artifact bắt buộc phải có để xác nhận kỹ năng hoàn tất, áp dụng đối với các kỹ năng có yêu cầu artifact

Trong đó, requires mô tả quan hệ phụ thuộc để hỗ trợ lập kế hoạch và điều phối, nhưng không phải điều kiện bắt buộc để cho phép nạp kỹ năng. Thuộc tính completion-artifact xác định artifact cần có trước khi hệ thống ghi nhận một kỹ năng đã hoàn tất. Phần thân Markdown của SKILL.md trình bày mục tiêu, phạm vi, các bước thực hiện và cách xác nhận kết quả kiểm thử. Khi kỹ năng được nạp, nội dung hướng dẫn này được bổ sung vào ngữ cảnh để Agent sử dụng trong quá trình xử lý nhiệm vụ.

KAgent phát hiện kỹ năng từ các phạm vi Built-in, Project, Personal và những thư mục được cấu hình bổ sung. Khi khởi động, hệ thống quét các vị trí này, kiểm tra cấu trúc thư mục kỹ năng và đăng ký những kỹ năng hợp lệ vào Skill Registry. Nếu có nhiều kỹ năng trùng tên, mục đăng ký sau sẽ thay thế mục được đăng ký trước. Để tránh đưa quá nhiều nội dung vào ngữ cảnh, KAgent sử dụng cơ chế nạp động (Dynamic loading). Theo mặc định, System Prompt chỉ cung cấp thông tin tóm tắt gồm tên và mô tả của các kỹ năng khả dụng. Nội dung hướng dẫn đầy đủ chỉ được bổ sung vào Working Context khi kỹ năng tương ứng được nạp hoặc kích hoạt.

Kỹ năng có thể được kích hoạt theo hai cách. Khi người dùng chủ động kích hoạt thông qua lệnh giao diện, kỹ năng được ghi nhận để áp dụng từ lượt tương tác tiếp theo. Khi mô hình gọi công cụ load_skill, kỹ năng được nạp và có thể sử dụng ngay trong lượt xử lý hiện tại. Quy trình lựa chọn và nạp kỹ năng được thể hiện trong Hình 2.10.



Hình 2.10. Quy trình lựa chọn và nạp kỹ năng

Trước khi nạp kỹ năng, hệ thống kiểm tra kỹ năng có tồn tại trong Skill Registry và có được phép sử dụng hay không. Kỹ năng bị vô hiệu hóa sẽ không được nạp. Nếu yêu cầu nạp xuất phát từ mô hình thông qua công cụ load_skill, hệ thống còn kiểm tra thuộc tính disable-model-invocation. Khi thuộc tính này được thiết lập, mô hình không được tự nạp kỹ năng mà chỉ người dùng mới có thể chủ động kích hoạt. Việc kiểm tra tại thời điểm nạp được thực hiện độc lập với đề xuất của Decision Planner.

Ngoài nội dung hướng dẫn, mỗi kỹ năng có thể khai báo giới hạn công cụ thông qua thuộc tính allowed-tools. Giới hạn này được áp dụng đối với những thao tác cần cấp quyền. Việc xác định một thao tác có cần quyền hay không phụ thuộc vào loại công cụ, tham số và ngữ cảnh thực hiện. Khi điều kiện kiểm soát được áp dụng, hệ thống đối chiếu công cụ với các giới hạn do kỹ năng đang hoạt động khai báo. Những thao tác không yêu cầu cấp quyền không chịu ràng buộc allowed-tools này.

Khi nhiều kỹ năng cùng hoạt động, một công cụ đáp ứng giới hạn theo kỹ năng nếu có ít nhất một kỹ năng cho phép sử dụng hoặc có kỹ năng không khai báo giới hạn công cụ. Tuy nhiên, việc đáp ứng điều kiện allowed-tools không đồng nghĩa

với việc công cụ được phép thực thi ngay. Các cơ chế kiểm tra quyền, phạm vi và an toàn khác vẫn được thực hiện độc lập trước khi thao tác tác động lên mục tiêu.

Việc hoàn tất kỹ năng cũng phải đáp ứng các điều kiện tương ứng. Với kỹ năng yêu cầu artifact, hệ thống kiểm tra tệp tại đường dẫn được khai báo trong completion-artifact trước khi ghi nhận hoàn tất. Đối với các pha khảo sát trong chế độ Whole-target, hệ thống còn đối chiếu phạm vi, thông tin bao phủ và trạng thái đầu vào liên quan. Như vậy, việc hoàn tất pha không chỉ phụ thuộc vào sự tồn tại của tệp kết quả mà còn phải phù hợp với trạng thái và các điều kiện của quy trình kiểm thử. Đối với kỹ năng xác thực lỗ hổng, việc xác định hoàn tất còn dựa trên kết quả kiểm thử đã ghi nhận và các điều kiện về bằng chứng, thay vì chỉ căn cứ vào tệp artifact.

2.5.3. Mô-đun quản lý và thực thi công cụ

Mô-đun quản lý và thực thi công cụ kết nối các yêu cầu do mô hình ngôn ngữ đề xuất với những thao tác cụ thể trong môi trường kiểm thử. Agent Core điều phối việc sử dụng công cụ dựa trên nhiệm vụ và trạng thái hiện tại, Tool Registry quản lý các công cụ đã đăng ký, còn từng công cụ đảm nhiệm chức năng tương ứng. Cách tổ chức này tách biệt việc lựa chọn hành động, quản lý giao diện công cụ và thực hiện thao tác, đồng thời tạo điều kiện áp dụng các cơ chế kiểm tra tham số, kiểm soát quyền và giới hạn thực thi phù hợp.

a) Quản lý và mô tả công cụ

KAgent sử dụng giao diện chung Tool để thống nhất cách mô tả và tiếp nhận yêu cầu thực thi đối với các công cụ có chức năng khác nhau. Thông qua giao diện này, mỗi công cụ cung cấp tên định danh, mô tả chức năng, lược đồ tham số, yêu cầu phê duyệt mặc định và thành phần thực thi. Nhờ đó, hệ thống có thể quản lý công cụ theo một cấu trúc thống nhất mà không cần đưa logic xử lý riêng của từng thao tác vào Agent Core. Các thành phần chính của giao diện công cụ được trình bày trong Bảng 2.7.

Bảng 2.7. Các thành phần chính của công cụ

Thành phần	Ý nghĩa
Tên định danh (name)	Xác định tên dùng để gọi và tra cứu công cụ trong hệ thống.
Mô tả chức năng (description)	Cung cấp thông tin về chức năng và mục đích sử dụng của công cụ.

Thành phần	Ý nghĩa
Lược đồ tham số (schema)	Mô tả cấu trúc và các ràng buộc của tham số đầu vào.
Yêu cầu phê duyệt mặc định (requires_permission)	Khai báo yêu cầu phê duyệt mặc định trước khi công cụ thực hiện thao tác.
Thành phần thực thi (run)	Thực hiện chức năng của công cụ trên cơ sở dữ liệu đầu vào và các điều kiện thực thi tương ứng.

Bên cạnh giao diện chung, một số công cụ cung cấp thêm cơ chế kiểm tra tham số, xác định yêu cầu phê duyệt theo hành động và mô tả thao tác để người kiểm thử xem xét. Chẳng hạn, các hành động truy vấn và thay đổi dữ liệu trong cùng một công cụ có thể cần những điều kiện kiểm soát khác nhau. Vì vậy, yêu cầu phê duyệt mặc định chỉ là một phần của cơ chế kiểm soát. Việc cho phép thực hiện một thao tác cụ thể còn phụ thuộc vào tham số, phạm vi tài nguyên và chính sách đang có hiệu lực.

Tool Registry lưu các công cụ đã đăng ký và sử dụng tên định danh để tra cứu công cụ tương ứng. Registry cung cấp chức năng đăng ký, tra cứu và điểm truy cập thống nhất cho việc thực thi, đồng thời từ chối đăng ký trùng tên để tránh thay thế công cụ ngoài ý muốn. Việc duy trì danh mục tập trung giúp Agent Core sử dụng công cụ thông qua giao diện đã thống nhất, trong khi cách thực hiện từng chức năng vẫn được quản lý tại công cụ đó. Các giới hạn sử dụng theo kỹ năng và việc điều phối nhiều yêu cầu thuộc trách nhiệm của Agent Core.

Từ những công cụ đã đăng ký, Tool Registry tạo các cấu trúc mô tả ToolSpec để cung cấp cho mô hình ngôn ngữ. Mỗi ToolSpec chứa tên, mô tả chức năng và lược đồ tham số đầu vào, giúp mô hình nhận biết công cụ và hình thành yêu cầu gọi phù hợp. Phần mô tả này được tách khỏi logic thực thi và không mang ý nghĩa cấp quyền cho hành động do mô hình đề xuất. Khi kiểm tra công cụ, hệ thống quy đổi một số tên gọi tương đương về tên định danh thống nhất, giúp các điều kiện kiểm soát được áp dụng nhất quán.

Theo mục đích sử dụng, các công cụ được tổ chức thành những nhóm chức năng như trình bày trong Bảng 2.8. Cách phân nhóm này thể hiện phạm vi trách nhiệm của các công cụ trong kiến trúc, thay vì mô tả một trình tự kiểm thử cố định.

Bảng 2.8. Phân nhóm công cụ theo mục đích sử dụng

Nhóm công cụ	Chức năng chính
Thao tác hệ thống và tệp tin	Thực hiện lệnh cục bộ, đọc, ghi, chỉnh sửa và tìm kiếm dữ liệu phục vụ kiểm thử.
Tương tác mạng và web	Gửi yêu cầu tới ứng dụng mục tiêu, khảo sát dịch vụ, liệt kê tài nguyên và thu thập dữ liệu web hỗ trợ phân tích.
Trình duyệt và thu thập lưu lượng	Tiếp nhận, tra cứu và truy xuất lưu lượng từ trình duyệt hoặc công cụ trung gian trong phạm vi được kiểm soát. Nhóm này không mặc nhiên cung cấp khả năng điều khiển trình duyệt.
Quản lý kỹ năng và tài nguyên	Nạp kỹ năng kiểm thử và truy xuất các tài nguyên hỗ trợ thực hiện nhiệm vụ.
Quản lý trạng thái quy trình	Ghi nhận, cập nhật và truy vấn đầu vào, đối tượng nghi vấn, kết quả xác thực và thông tin bao phủ kiểm thử.
Xác nhận và ghi nhận phát hiện	Kiểm tra điều kiện của kết quả xác thực và bằng chứng liên quan trước khi ghi nhận phát hiện.
Tương tác người dùng và trạng thái quyền	Trao đổi với người kiểm thử khi cần bổ sung thông tin và cung cấp thông tin về trạng thái quyền thực thi.

Các nhóm trên phối hợp để hỗ trợ cả hoạt động tương tác với môi trường kiểm thử và quản lý thông tin trong hệ thống. Kiến trúc giao diện chung cho phép bổ sung công cụ mới thông qua cơ chế đăng ký và điều phối thống nhất, đồng thời duy trì sự phân tách giữa chức năng của từng công cụ và trách nhiệm quản lý của hệ thống.

b) Thực thi và kiểm soát công cụ

Cơ chế tiếp nhận yêu cầu gọi công cụ và đưa kết quả trở lại Agent đã được trình bày tại Mục 2.4.3. Ở cấp mô-đun, việc kiểm soát được phân chia giữa Agent Core, Tool Registry, cơ chế quản lý quyền và từng công cụ. Đối với những thao tác chịu ràng buộc allowed-tools của kỹ năng, Agent Core đối chiếu yêu cầu với giới hạn đang áp dụng. Tool Registry tiếp nhận yêu cầu phù hợp, tra cứu công cụ và áp dụng bước kiểm tra tham số do công cụ cung cấp trước khi chuyển tới phần thực thi.

Lược đồ tham số và bước kiểm tra tại thời điểm thực thi có vai trò bổ sung cho nhau. Lược đồ giúp mô hình tạo yêu cầu theo cấu trúc đã công bố, còn Registry và công cụ kiểm tra dữ liệu thực tế nhận được để xác định yêu cầu có đáp ứng các điều kiện thực thi hay không. Tùy chức năng, công cụ có thể kiểm tra thêm đường dẫn tài nguyên, ngữ cảnh yêu cầu hoặc các điều kiện riêng của thao tác. Cách tổ chức

này giúp duy trì giao diện chung nhưng vẫn cho phép áp dụng những kiểm tra phù hợp với từng loại công cụ.

Đối với hành động cần phê duyệt, cơ chế quản lý quyền cung cấp thông tin mô tả thao tác để người kiểm thử xem xét thông qua lớp giao diện. Quyết định cho phép được áp dụng theo phạm vi và điều kiện của yêu cầu đang được xét. Trước khi thực thi, hệ thống kiểm tra quyết định cho phép có còn hiệu lực và phù hợp với yêu cầu hay không. Khi bị từ chối hoặc không đáp ứng điều kiện, yêu cầu không được tiếp tục và thông tin tương ứng được trả về cho Agent. Chế độ YOLO có thể tự xử lý bước phê duyệt theo chính sách, nhưng không loại bỏ các kiểm tra về quyền hiện hành, phạm vi và giới hạn thực thi.

Các biện pháp kiểm soát cụ thể phụ thuộc vào đặc điểm của công cụ. Công cụ HTTP tích hợp kiểm tra yêu cầu theo phạm vi mục tiêu và quyền tương ứng trước khi tương tác với ứng dụng. Công cụ thao tác tệp áp dụng các điều kiện liên quan đến tài nguyên cục bộ và dữ liệu nhạy cảm. Đối với công cụ shell, lệnh được chạy trực tiếp dưới quyền của tài khoản vận hành, kèm theo kiểm soát thời gian chạy, hủy tác vụ và lượng đầu ra thu nhận. Những giới hạn này giúp quản lý quá trình thực thi nhưng không tạo thành cơ chế cô lập ở cấp hệ điều hành.

Kết quả công cụ cung cấp dữ liệu quan sát hoặc thông tin lỗi để Agent đánh giá và lựa chọn bước xử lý tiếp theo. Việc giới hạn dung lượng, xử lý thông tin nhạy cảm và lưu kết quả dài để đọc lại tuân theo cơ chế quản lý ngữ cảnh tại Mục 2.4.2. Dữ liệu được lưu để phục vụ hội thoại không tự trở thành bằng chứng đã được chấp nhận. Khi sử dụng kết quả kiểm thử làm cơ sở ghi nhận phát hiện, hệ thống còn áp dụng cơ chế đăng ký bằng chứng và kiểm tra điều kiện của workflow. Mô-đun thực thi công cụ cung cấp dữ liệu quan sát, còn các thành phần quản lý trạng thái và kết quả kiểm thử đảm nhiệm việc đăng ký bằng chứng và kiểm tra điều kiện ghi nhận phát hiện.

2.5.4. Mô-đun quản lý phiên và bộ nhớ

Mô-đun quản lý phiên và bộ nhớ duy trì tính liên tục của quá trình kiểm thử qua các lượt tương tác và khi người kiểm thử tiếp tục một phiên đã lưu. Các dữ liệu cần thiết để theo dõi nhiệm vụ được quản lý trong trạng thái phiên, còn thông tin có giá trị tái sử dụng giữa nhiều phiên được lưu trong bộ nhớ bền vững. Sự phân chia này giúp hệ thống vừa bảo toàn tiến độ công việc, vừa lựa chọn được thông tin liên quan mà không phải đưa toàn bộ dữ liệu đã tích lũy vào ngữ cảnh của mô hình.

a) Mô-đun quản lý phiên

Session Store cung cấp cơ chế lưu và nạp dữ liệu phiên cho Agent Core. Thành phần này duy trì lịch sử tương tác, thông tin mục tiêu, khai báo phạm vi, bộ nhớ phiên và trạng thái quy trình, nhưng không trực tiếp lập kế hoạch hoặc lựa chọn hành động kiểm thử. Agent Core sử dụng dữ liệu được nạp để xây dựng lại ngữ cảnh làm việc và đánh giá công việc cần tiếp tục theo các điều kiện hiện hành.

Dữ liệu được lưu của một phiên được biểu diễn bằng SessionFile. Trong cấu trúc này, lịch sử hội thoại, thông tin tóm lược và trạng thái workflow được tổ chức thành các thành phần riêng, vì mỗi loại dữ liệu có vai trò khác nhau đối với việc tiếp tục nhiệm vụ. Các thành phần chính của SessionFile được trình bày trong Bảng 2.9.

Bảng 2.9. Các thành phần chính của SessionFile

Thành phần	Vai trò
id	Định danh phiên làm việc.
target	Lưu thông tin về mục tiêu kiểm thử.
engagement_state	Lưu khai báo phạm vi kiểm thử của phiên, gồm các origin HTTP và phạm vi TCP liên quan. Không khôi phục quyền thực thi tạm thời.
workflow	Lưu trạng thái có cấu trúc của quy trình kiểm thử.
memory	Lưu SessionMemory để duy trì thông tin tóm lược giữa các lượt xử lý.
messages	Duy trì lịch sử tương tác. Những kết quả được lưu riêng có bản xem trước và tham chiếu, không tự nạp lại toàn bộ nội dung khi khôi phục.
updated_at	Ghi nhận thời điểm dữ liệu phiên được cập nhật.

Lịch sử hội thoại giúp duy trì mạch tương tác, trong khi workflow bảo toàn các bản ghi có cấu trúc về đầu vào, Candidate, lượt xác thực và kết quả liên quan. EngagementState lưu khai báo phạm vi của phiên, bao gồm các origin HTTP và phạm vi TCP khi có. Dữ liệu này không biểu diễn toàn bộ chính sách tác nghiệp hoặc các quyền thực thi tạm thời. Các bản ghi workflow được lưu riêng nên việc thu gọn hội thoại không thay thế chúng bằng bản tóm lược.

Khi phiên kéo dài, SessionMemory lưu phần thông tin tóm lược có ích cho việc tiếp tục xử lý sau thu gọn. Cấu trúc này ghi nhận định hướng nhiệm vụ, tiến độ, công việc còn lại và những ghi chú kỹ thuật cần thiết, thay vì sao chép toàn bộ chuỗi

hội thoại. Các nhóm thông tin chính của SessionMemory được trình bày trong Bảng 2.10.

Bảng 2.10. Các nhóm thông tin chính của SessionMemory

Nhóm thông tin	Các trường dữ liệu	Vai trò đối với ngữ cảnh tiếp diễn
Định hướng và công việc còn lại	objectives, plan, todos	Duy trì mục tiêu, kế hoạch và những công việc còn cần xử lý sau khi lịch sử được thu gọn.
Tiến độ và bề mặt đã kiểm tra	completed, tested	Tóm lược công việc đã hoàn thành và các đối tượng đã kiểm tra để hỗ trợ tiếp tục nhiệm vụ.
Ghi nhận kỹ thuật	findings, files, commands	Lưu ghi chú về kết quả đáng chú ý, tệp và lệnh trong phiên. Các ghi chú findings không thay thế Finding đã được chấp nhận.
Thông tin xác thực	credentials	Lưu ghi chú đã xử lý dữ liệu nhạy cảm để tham khảo, không thay thế ngữ cảnh xác thực hiện hành của công cụ.
Quản lý thu gọn	compactions, last_compacted_at, last_summary	Theo dõi số lần thu gọn, thời điểm và nội dung tóm lược gần nhất.

Trong đó, todos bổ sung danh sách công việc còn phải xử lý để Agent tiếp tục nhiệm vụ sau khi lịch sử cũ được thay bằng bản tóm lược. Các trường completed, tested và findings phục vụ ghi nhớ nội dung đã diễn ra, nhưng không thay thế trạng thái kiểm thử có cấu trúc. Đặc biệt, ghi chú findings trong SessionMemory không đồng nghĩa với Finding đã đáp ứng điều kiện xác nhận và lưu báo cáo. Thông tin xác thực trong bộ nhớ cũng chỉ có giá trị tham khảo sau khi xử lý dữ liệu nhạy cảm, không tự khôi phục trạng thái đăng nhập hoặc quyền của công cụ.

Quá trình thu gọn do Agent Core điều phối theo cơ chế tại Mục 2.4.2. Chỉ khi kết quả đạt yêu cầu giảm kích thước ngữ cảnh, phần tóm lược mới được chấp nhận để cập nhật SessionMemory và thay thế lịch sử cũ bằng nội dung cần thiết cùng phần tương tác gần nhất. Những thông tin về mục tiêu, phạm vi và workflow tiếp tục được duy trì riêng. Session Store lưu các thành phần này cùng lịch sử hiện có để việc tiếp tục phiên dựa trên dữ liệu đã được bảo toàn.

Khi khôi phục phiên, hệ thống nạp dữ liệu đã lưu và xây dựng lại chỉ dẫn cho mô hình. Việc khôi phục không tiếp tục một tiến trình công cụ đang chạy hoặc tự tái lập đầy đủ môi trường thực thi trước đó. Các lượt xác thực chưa kết thúc được ghi nhận là gián đoạn. Hệ thống không tự khôi phục trạng thái kích hoạt kỹ năng hoặc các quyền thực thi tạm thời, nên các thao tác tiếp theo vẫn phải được kiểm tra quyền. Khi bắt đầu một lượt xác thực mới, hệ thống sử dụng định danh riêng thay vì dùng lại định danh của lượt đã gián đoạn.

Agent có thể dùng các tham chiếu trong lịch sử và workflow để xem lại dữ liệu đã thu được. Việc đọc lại chỉ thực hiện được khi dữ liệu vẫn còn, tính toàn vẹn được bảo đảm và các điều kiện truy cập tại thời điểm đọc được đáp ứng. Hệ thống không tự nạp toàn bộ nội dung của các kết quả công cụ đã lưu riêng vào ngữ cảnh khi mở lại phiên. Vì vậy, khôi phục phiên cần được hiểu là tiếp tục từ trạng thái thông tin đã lưu, trong khi các điều kiện cho hành động tiếp theo vẫn được đánh giá tại thời điểm thực hiện.

b) Mô-đun quản lý bộ nhớ

Memory Store quản lý những thông tin bền vững có thể tái sử dụng giữa nhiều phiên làm việc. Vai trò này khác với SessionMemory, vốn duy trì bản tóm lược để tiếp tục một phiên cụ thể. Memory Store cũng được quản lý riêng với Intelligence Store đã trình bày tại Mục 2.5.2. Intelligence tổ chức tri thức thành các scenario phục vụ phân tích và kiểm thử, còn bộ nhớ bền vững lưu từng thông tin kỹ thuật, tham chiếu hoặc ghi chú để có thể sử dụng lại khi cần.

Đơn vị dữ liệu cơ bản của kho bộ nhớ là MemoryFact. Mỗi MemoryFact biểu diễn một thông tin có nội dung, mô tả và phạm vi lưu trữ riêng để có thể quản lý và truy hồi khi cần. Các thuộc tính chính được trình bày trong Bảng 2.11.

Bảng 2.11. Các thuộc tính chính của cấu trúc MemoryFact

Thuộc tính	Ý nghĩa
name	Tên định danh của thông tin.
description	Mô tả ngắn giúp nhận biết và hỗ trợ truy hồi.
type	Phân loại nội dung của thông tin.
scope	Xác định phạm vi lưu trữ Project hoặc Personal.
text	Nội dung chi tiết được lưu trữ.
created_at	Thời điểm thông tin được tạo.

MemoryFact được tổ chức theo hai phạm vi Project và Personal. Phạm vi Project gắn với thư mục dữ liệu của dự án hiện tại và hỗ trợ sử dụng lại thông tin trong các phiên thuộc dự án đó. Phạm vi Personal thuộc dữ liệu của người sử dụng và có thể cung cấp thông tin cho nhiều dự án. Cách phân chia này xác định nơi lưu trữ và tập dữ liệu có thể truy hồi, không tự tạo một kho riêng cho mỗi Target. Việc thay đổi mục tiêu kiểm thử cũng không làm cho mọi thông tin đã lưu trong dự án trở nên phù hợp với mục tiêu mới.

Khi bổ sung thông tin vào bộ nhớ, hệ thống xử lý dữ liệu nhạy cảm và tổ chức nội dung thành các bản ghi có thể truy xuất độc lập. Những phát biểu về phạm vi được cho phép, quyết định phê duyệt một lần hoặc trạng thái YOLO của phiên được nhận diện để loại khỏi việc lưu thành bộ nhớ bền vững. Cơ chế này giúp hạn chế việc mang các điều kiện chỉ có hiệu lực trong một phiên sang những nhiệm vụ hoặc mục tiêu khác.

Đối với truy hồi, Agent sử dụng yêu cầu hiện tại kết hợp với thông tin mục tiêu để hình thành truy vấn. Memory Store đối chiếu các từ khóa với tên, mô tả, loại và nội dung của MemoryFact, sau đó xếp hạng theo mức độ phù hợp có xét độ mới của thông tin. Chỉ một số bản ghi phù hợp được lựa chọn và lượng nội dung bổ sung vào ngữ cảnh được giới hạn. Cách tổ chức này giúp cung cấp thông tin cần thiết cho nhiệm vụ mà không phải chuyển toàn bộ kho bộ nhớ tới mô hình.

Dữ liệu được truy hồi được cung cấp như thông tin tham khảo và có thể cần kiểm tra lại khi đã cũ hoặc không còn phù hợp với môi trường hiện tại. Bộ nhớ cung cấp thông tin đã lưu để Agent tham khảo khi xử lý nhiệm vụ, nhưng không thay thế bằng chứng kiểm thử, kết quả workflow hoặc quyết định của người kiểm thử. Các quyền thực thi và phạm vi được cho phép vẫn phải được xác định trong ngữ cảnh phiên hiện hành.

2.5.5. Mô-đun quản lý trạng thái và kết quả kiểm thử

Mô-đun quản lý trạng thái và kết quả kiểm thử tổ chức các thông tin hình thành trong quá trình đánh giá thành những bản ghi có cấu trúc. WorkflowState duy trì tiến trình và quan hệ giữa các đối tượng kiểm thử, Coverage theo dõi các tổ hợp đã được ghi nhận, còn Finding lưu những phát hiện đáp ứng điều kiện xác nhận. Sự phân chia này giúp Agent và Planner sử dụng trạng thái nhất quán, đồng thời phân biệt dấu hiệu nghi vấn, dữ liệu quan sát, bằng chứng đã đăng ký và kết quả đã được chấp nhận.

a) Quản lý trạng thái quy trình kiểm thử

WorkflowState duy trì trạng thái có cấu trúc của quy trình kiểm thử và được lưu như một phần của phiên làm việc. Lịch sử hội thoại phản ánh nội dung tương tác, còn WorkflowState cho phép hệ thống theo dõi những đối tượng đã ghi nhận, công việc đang chờ xử lý và kết quả của từng lượt xác thực. Trạng thái này được cập nhật qua Workflow Tool và cung cấp cho các thành phần điều phối sử dụng, thay vì suy ra tiến độ chỉ từ lời mô tả của mô hình. Các thành phần dữ liệu chính được trình bày trong Bảng 2.12.

Bảng 2.12. Các thành phần dữ liệu chính của WorkflowState

Thành phần	Vai trò
attack_surface_inputs	Quản lý điểm đầu vào của bề mặt ứng dụng và trạng thái xử lý tương ứng.
candidates	Quản lý các đối tượng nghi vấn cần được kiểm tra và xác thực.
attempts	Theo dõi các lượt xác thực và danh tính gắn với Candidate, mục tiêu và ngữ cảnh phiên.
evidence	Quản lý các EvidenceArtifact đã đăng ký và tham chiếu tới bản lưu bằng chứng.
validation_results	Lưu đánh giá có cấu trúc cùng liên kết tới Candidate, lượt xác thực và nguồn bằng chứng.
phase_coverage, phase_completions	Theo dõi nội dung đã xử lý trong các pha khảo sát và lưu các bản ghi hoàn thành pha.
objective	Duy trì mục tiêu workflow và các mục tiêu kiểm thử cần xử lý trong yêu cầu.
persisted_findings	Ghi nhận liên kết giữa Finding đã lưu và kết quả xác thực tương ứng.

Trong chế độ Whole-target, các điểm đầu vào được biểu diễn bằng AttackSurfaceInput, gắn với mục tiêu workflow, origin đích và ngữ cảnh yêu cầu tương ứng. Mỗi điểm đầu vào có trạng thái xử lý để phân biệt những nội dung còn chờ phân tích, đã phân tích, bị chặn hoặc được đưa ra khỏi phạm vi xử lý có ghi nhận lý do. Nhờ các trạng thái này, hệ thống theo dõi được tiến độ xử lý của từng đầu vào đã thu thập. Việc đã liệt kê được một endpoint vì vậy không đồng nghĩa với đã hoàn tất phân tích đối tượng đó.

Candidate biểu diễn một dấu hiệu hoặc giả thuyết về lỗ hổng cần kiểm tra. Khi được liên kết với AttackSurfaceInput trong chế độ Whole-target, Candidate phải

có ngữ cảnh phù hợp về mục tiêu, endpoint, phương thức và các thuộc tính đầu vào liên quan. Với nhiệm vụ kiểm tra trực tiếp hoặc xác thực một Candidate được chỉ định, hệ thống có thể quản lý đối tượng nghi vấn dựa trên yêu cầu đã xác định mà không bắt buộc phải ghi nhận AttackSurfaceInput trước. Vì vậy, việc liên kết Candidate với đầu vào phụ thuộc vào chế độ kiểm thử.

Các đối tượng được nhận diện bằng định danh ổn định dựa trên thuộc tính và ngữ cảnh kiểm thử. Cơ chế này giúp hạn chế tạo nhiều bản ghi cho cùng một đối tượng, đồng thời phân biệt các trường hợp khác phương thức, vị trí đầu vào, kiểu nội dung hoặc ngữ cảnh xác thực. Candidate được theo dõi bằng trạng thái xử lý riêng, như trình bày trong Bảng 2.13.

Bảng 2.13. Các trạng thái xử lý của đối tượng nghi vấn (Candidate)

Trạng thái	Ý nghĩa
new	Đối tượng nghi vấn mới được ghi nhận và chưa bắt đầu xác thực.
queued	Đối tượng đang chờ được xử lý khi đáp ứng các điều kiện tương ứng.
validating	Đối tượng đang được xử lý trong một lượt xác thực do runtime quản lý.
validated	Đối tượng đã có kết quả confirmed hoặc not-confirmed. Trạng thái này không đồng nghĩa đã xác nhận có lỗ hổng.
deferred	Việc xác thực còn bị tạm hoãn hoặc chưa giải quyết được do thiếu điều kiện, trở ngại hay bằng chứng.
dismissed	Đối tượng được đưa ra khỏi tập cần tiếp tục xác thực. Trạng thái này không tự tạo kết luận không có lỗ hổng.

Trạng thái Candidate phản ánh tiến độ xử lý, còn ValidationResult lưu kết luận của một lượt xác thực. Trong luồng xử lý kết quả, confirmed và not-confirmed đưa Candidate về trạng thái validated. Các kết quả chưa giải quyết được đưa đối tượng về deferred để bảo toàn thông tin về trở ngại hoặc điều kiện còn thiếu. Do đó, validated không có nghĩa đã xác nhận có lỗ hổng, còn việc chưa bắt đầu một phép kiểm tra không tự đồng nghĩa với có trở ngại cần tạm hoãn.

Một lượt xác thực (Validation Attempt) ghi nhận một lần kiểm chứng Candidate trong ngữ cảnh do runtime quản lý. Một Candidate có thể được kiểm chứng nhiều lần, nhưng mỗi lượt có định danh riêng và gắn với mục tiêu workflow (Objective), mục tiêu kiểm thử và phiên tương ứng. ValidationResult lưu kết luận

của lượt đó. Cách tổ chức này cho phép theo dõi các lần kiểm chứng mà không gộp bằng chứng của những lượt khác nhau.

Quá trình thực thi công cụ hoặc tiếp nhận lưu lượng HTTP tạo ra dữ liệu quan sát (Observation). Ngoài nội dung được thu nhận, bản ghi giữ thông tin về nguồn, yêu cầu và trạng thái thực thi. Các thông tin như phương thức HTTP, địa chỉ yêu cầu, trạng thái phản hồi và dấu thời gian giúp diễn giải dữ liệu trong đúng ngữ cảnh. Mức đầy đủ, trạng thái cắt bớt và thông tin kiểm tra toàn vẹn cho biết giới hạn của phần đã lưu. Vì vậy, một Observation không mặc nhiên chứa toàn bộ phản hồi của ứng dụng.

Để sử dụng các quan sát này, hệ thống cần truy vết được nguồn gốc dữ liệu (Provenance). Nguồn gốc liên kết quan sát với thành phần thu thập và ngữ cảnh yêu cầu thực tế. Khi Agent khai báo quan sát để hỗ trợ ghi nhận Candidate, runtime đối chiếu các thông tin yêu cầu được hỗ trợ. Lời mô tả của Agent không thay thế metadata do nguồn ghi nhận, còn thông tin chưa thu thập vẫn được xem là chưa xác định.

Bên cạnh nguồn gốc, ràng buộc sở hữu (Ownership) xác định quan sát có thuộc đúng Candidate, lượt xác thực, Objective và phiên làm việc hay không. Quan sát khảo sát có thể được dùng để phân tích và hình thành Candidate mà chưa thuộc một lượt xác thực. Việc dẫn lại quan sát đó không tự làm thay đổi ràng buộc sở hữu. Sự phân biệt này ngăn dữ liệu phục vụ phân tích bị mặc nhiên dùng như bằng chứng kết luận của một lượt kiểm chứng khác.

Trên cơ sở các nguồn quan sát, Agent có thể lập tệp bằng chứng và đăng ký với workflow. EvidenceArtifact biểu diễn bản lưu đã đăng ký, kèm định danh, liên kết Candidate, vị trí lưu và thông tin kiểm tra toàn vẹn. Trước bước đăng ký, tệp phân tích do Agent tạo vẫn chỉ là artifact cục bộ. Bằng chứng văn bản được xử lý thông tin nhạy cảm theo chính sách lưu giữ. Việc đăng ký duy trì bản lưu và liên hệ với nguồn, nhưng chưa xác nhận lỗ hổng hoặc bảo đảm bằng chứng đủ điều kiện cho mọi kết luận.

Việc gắn nhận định với dữ liệu thực tế được gọi là Evidence Grounding. Khi gửi kết quả confirmed hoặc not-confirmed, Agent phải sử dụng bằng chứng phù hợp với Candidate và lượt kiểm chứng tương ứng. Runtime kiểm tra nguồn gốc, ràng buộc sở hữu, tính toàn vẹn và điều kiện thực thi theo hợp đồng đánh giá. Những kiểm tra này xác định tính hợp lệ của hồ sơ. Chúng không thay thế việc đối chiếu ý nghĩa của nhận định với nội dung đã quan sát, cũng không bảo đảm kết luận của

Agent là đúng. Các dạng kết luận của ValidationResult được trình bày trong Bảng 2.14.

Bảng 2.14. Các kết luận xác thực của ValidationResult

Kết luận	Ý nghĩa
confirmed	Agent đánh giá đã xác nhận được giả thuyết lỗ hổng dựa trên bằng chứng đáp ứng điều kiện chấp nhận. Runtime không bảo đảm tính đúng của kết luận.
not-confirmed	Một lượt kiểm tra có giới hạn đã hoàn tất nhưng chưa xác nhận được giả thuyết. Không chứng minh tuyệt đối rằng không có lỗ hổng và không dùng để biểu diễn lỗi thực thi.
blocked	Quá trình xác thực gặp trở ngại và chưa thể hoàn tất.
insufficient-evidence	Chưa có đủ bằng chứng để đưa ra kết luận xác nhận.
deferred	Việc xử lý và đưa ra kết luận được tạm hoãn.
browser-required	Cần khả năng tương tác qua trình duyệt để tiếp tục xác thực.
authorization-required	Cần bổ sung quyền hoặc sự cho phép để tiếp tục xác thực.

Trong vận hành, lỗi thực thi, thiếu quyền hoặc dữ liệu chưa đầy đủ phải được phản ánh bằng trạng thái phù hợp, thay vì diễn giải thành kết luận không có lỗ hổng. Người kiểm thử có thể đối chiếu các kết quả với bằng chứng đã lưu để xem xét lại nhận định của Agent. Việc xem xét này không thay thế các điều kiện về nguồn gốc bằng chứng và quyền thực thi.

WorkflowState còn theo dõi bao phủ pha thông qua PhaseCoverage và các bản ghi hoàn thành pha. Trong Whole-target, một lượt xác thực mới chỉ được bắt đầu sau khi Reconnaissance và Web Enumeration đã hoàn tất với artifact hợp lệ. Candidate cũng cần liên kết với đầu vào đã phân tích phù hợp và có tài liệu bàn giao phân tích theo đường dẫn quy định. Khi những điều kiện này được đáp ứng, đầu vào độc lập còn chờ hoặc bị chặn không tự ngăn Candidate tiếp tục. Các đầu vào đó vẫn được giữ là công việc chưa hoàn thành.

Do đó, trạng thái analyzed của một đầu vào khác với việc Candidate đủ điều kiện xác thực. Cả hai trạng thái này cũng chưa chứng minh Input Analysis toàn cục hoặc mục tiêu Whole-target đã hoàn tất. Những đối tượng bị chặn, tạm hoãn, thiếu bằng chứng hoặc còn nghĩa vụ dọn dữ liệu tiếp tục được theo dõi cho đến khi được xử lý phù hợp. Cơ chế này phân biệt tiến độ từng đối tượng với tiến độ của cả pha

và mục tiêu. Coverage theo tổ hợp kiểm thử được trình bày ở phần tiếp theo có vai trò khác với bao phủ pha.

b) Theo dõi mức độ bao phủ kiểm thử

Coverage ghi nhận tiến độ theo tổ hợp endpoint, tham số và lớp lỗ hổng trong ngữ cảnh tương ứng. Ngữ cảnh còn có thể phân biệt mục tiêu workflow, origin đích, phương thức, vị trí đầu vào, kiểu nội dung, thông tin xác thực và trường hợp kiểm tra. Nhờ đó, những phép kiểm tra có cùng đường dẫn nhưng khác ngữ cảnh không bị mặc nhiên xem là một đối tượng đã được xử lý. Coverage Store lưu các bản ghi này, còn Coverage Tool cung cấp chức năng truy vấn phục vụ theo dõi tiến độ. Các thành phần chính của CoverageEntry được trình bày trong Bảng 2.15.

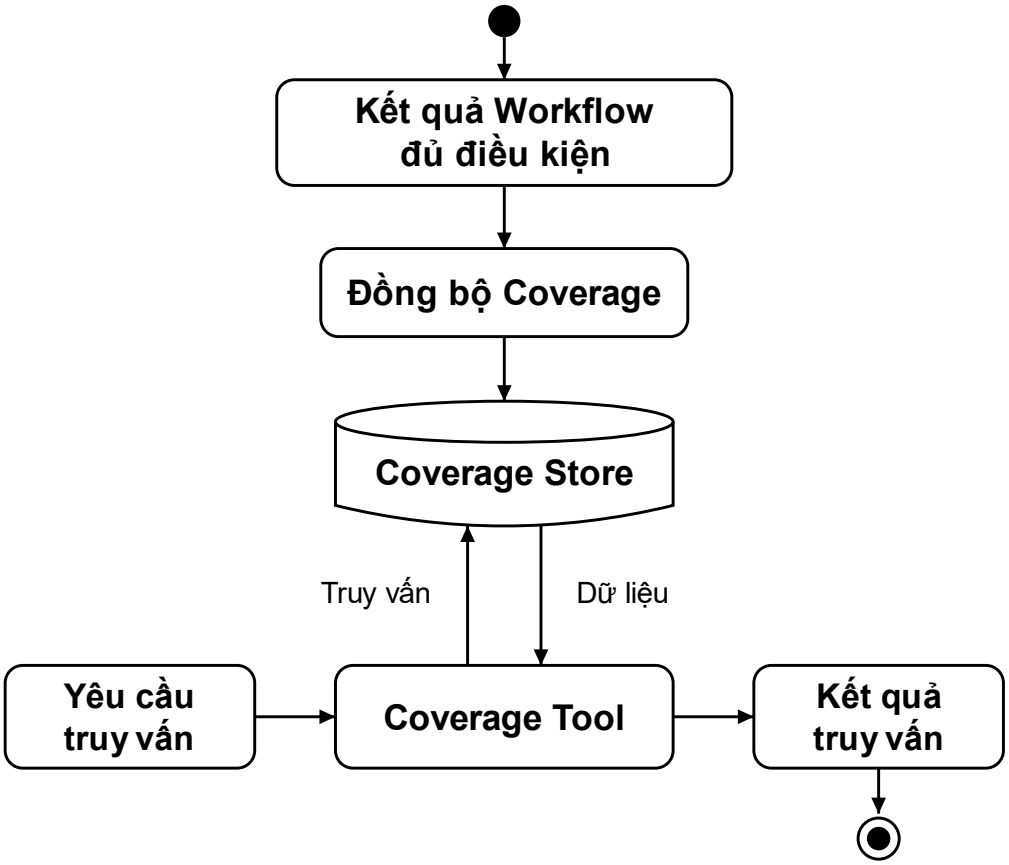
Bảng 2.15. Các thành phần dữ liệu chính của CoverageEntry

Thành phần	Ý nghĩa
endpoint	Endpoint của đối tượng được kiểm thử.
param	Tham số được xem xét tại endpoint tương ứng.
vulnClass	Lớp lỗ hổng được kiểm thử.
status	Trạng thái của tổ hợp, gồm tried, passed, failed, waf-blocked và skipped.
count	Số lượt ghi nhận sau khi áp dụng cơ chế nhận diện và loại trừ trùng lặp. Không tương đương số yêu cầu HTTP hoặc số lần đồng bộ.
firstSeen, lastSeen	Thời điểm ghi nhận lần đầu và cập nhật gần nhất.
notes	Thông tin bổ sung liên quan đến quá trình kiểm tra.
context	Ngữ cảnh phân biệt mục tiêu workflow, origin đích, phương thức, vị trí đầu vào, kiểu nội dung, xác thực và trường hợp kiểm tra khi có.

Các trạng thái tried, waf-blocked và skipped mô tả những dạng thông tin tiến độ mà kho Coverage có thể lưu. Khi đồng bộ từ workflow, kết quả confirmed được ghi vào Coverage với trạng thái failed, còn not-confirmed được ghi với trạng thái passed. Failed ở đây phản ánh phép kiểm tra đã ghi nhận một vấn đề bảo mật, trong khi passed chỉ biểu thị lượt kiểm tra tương ứng chưa xác nhận được giả thuyết. Các nhãn này không thay thế ValidationResult hoặc bằng chứng, và passed không chứng minh tuyệt đối rằng đối tượng an toàn.

Trong runtime hiện tại, Workflow Tool đảm nhiệm việc đồng bộ Coverage từ các kết quả workflow đủ điều kiện, thay cho việc Agent trực tiếp gọi thao tác mark

của Coverage Tool. Trước và sau khi lưu Coverage, hệ thống kiểm tra kết quả xác thực mới nhất, bằng chứng và ngữ cảnh của tổ hợp đang cập nhật. Việc đồng bộ chỉ được ghi nhận là hoàn tất khi dữ liệu đã được lưu thành công và các liên kết vẫn hợp lệ. Nếu lưu thất bại hoặc trạng thái thay đổi, việc đồng bộ vẫn chưa hoàn tất. Hình 2.11 minh họa cách dữ liệu được đồng bộ vào Coverage Store từ kết quả workflow và được truy vấn thông qua Coverage Tool.



Hình 2.11. Quy trình xử lý Coverage Tool

Khi truy vấn, Coverage Tool sử dụng phạm vi và các điều kiện được cung cấp để trả về dữ liệu đã lưu hoặc những tổ hợp chưa có bản ghi phù hợp. Đối với untested, hệ thống chỉ so sánh tập endpoint, tham số và lớp lỗ hổng được cung cấp trong ngữ cảnh tương ứng. Kết quả này hỗ trợ nhận biết phần chưa được ghi nhận trong tập yêu cầu, không tự phát hiện toàn bộ bề mặt ứng dụng hoặc chứng minh rằng các đối tượng đã có bản ghi đều hoàn tất xác thực.

Hệ thống nhận diện các lần đồng bộ lại hoặc cập nhật kết quả của cùng một Candidate trong cùng ngữ cảnh kiểm thử để hạn chế đếm trùng. Vì vậy, count không biểu thị tổng số yêu cầu HTTP hay số lần gọi đồng bộ. Khi lựa chọn công việc tiếp theo, Agent và Planner kết hợp Coverage với trạng thái Candidate, kết quả hiện hành

và các trở ngại trong WorkflowState. Cách sử dụng này giúp theo dõi bao phủ mà vẫn giữ các điều kiện hoàn thành dựa trên hồ sơ kiểm thử có cấu trúc.

c) Xác nhận và quản lý kết quả phát hiện

Finding biểu diễn phát hiện đã đáp ứng các điều kiện ghi nhận của hệ thống và được lưu thành báo cáo. Confirm Finding Tool tiếp nhận yêu cầu, đối chiếu hồ sơ xác thực trong workflow và chuẩn bị dữ liệu phát hiện để chuyển tới Finding Store. Thành phần này không thực hiện một phép kiểm chứng lỗ hổng độc lập, vì vậy dấu hiệu nghi vấn hoặc tệp bằng chứng vừa được tạo không thể chuyển trực tiếp thành Finding.

Điều kiện ghi nhận bao gồm Candidate đã xác định, kết quả mới nhất ở trạng thái confirmed còn được chấp nhận và bằng chứng đã đăng ký phù hợp với hồ sơ hiện hành. Hệ thống kiểm tra tính toàn vẹn, ngữ cảnh mục tiêu và các liên kết tới lượt xác thực, đồng thời không chấp nhận khi việc đồng bộ Coverage tương ứng còn đang chờ hoàn tất. Báo cáo đã lưu chỉ được công nhận cho kết quả xác thực mới nhất khi vẫn khớp với kết quả đó và các bằng chứng liên quan. Các thành phần dữ liệu chính của Finding được trình bày trong Bảng 2.16.

Bảng 2.16. Các thành phần dữ liệu chính của Finding

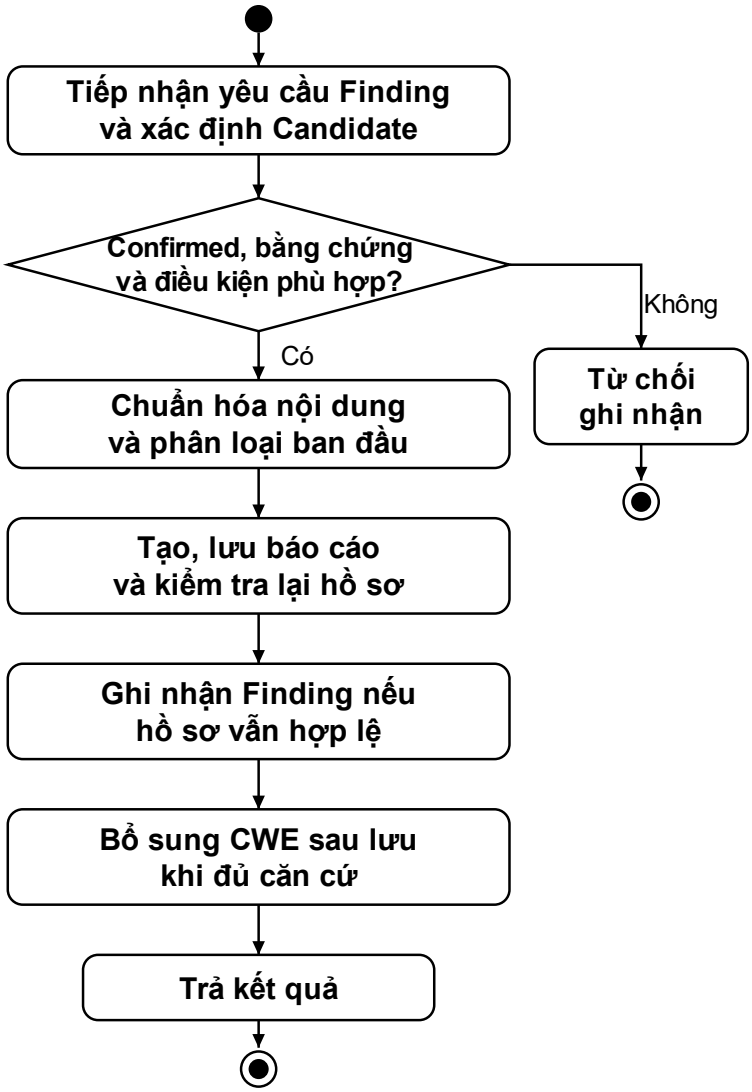
Thành phần	Vai trò
title	Tên của phát hiện được ghi nhận.
severity	Mức độ nghiêm trọng lấy từ đánh giá được chấp nhận.
url, parameter, method	Xác định vị trí và phương thức xuất hiện của phát hiện trên mục tiêu.
candidate_id	Liên kết Finding với Candidate tương ứng trong workflow.
evidence_refs	Tham chiếu tới bằng chứng đã đăng ký được sử dụng trong đánh giá.
observed_impact	Tác động quan sát lấy từ đánh giá được chấp nhận và gắn với bằng chứng.
potential_impact	Tác động có thể xảy ra trong những điều kiện phù hợp, được diễn đạt có điều kiện.
Thông tin tái hiện và quan sát	Payload, trích đoạn phản hồi và thông tin tái hiện hỗ trợ xem xét phát hiện, có xử lý dữ liệu nhạy cảm.
remediation	Khuyến nghị khắc phục phù hợp với thông tin đã được kiểm chứng.

Thành phần	Vai trò
cwe, owasp	Thông tin phân loại được bổ sung khi có cơ sở phù hợp.
Liên kết đánh giá	Gắn Finding với phiên bản kết quả, lượt xác thực và nguồn đánh giá để phân biệt với báo cáo lịch sử.

Mức độ nghiêm trọng và tác động đã quan sát của Finding được lấy từ đánh giá đã được chấp nhận. Các giá trị được cung cấp trong yêu cầu tạo báo cáo không thay thế những thông tin này. Tác động tiềm năng được cung cấp riêng và phải được diễn đạt có điều kiện, phù hợp với giới hạn của bằng chứng. Thông tin tái hiện và trích đoạn phản hồi hỗ trợ việc xem xét phát hiện, đồng thời được xử lý để hạn chế lưu dữ liệu nhạy cảm. Các khuyến nghị khắc phục cần phù hợp với thông tin thực tế đã được kiểm chứng.

Khi có ánh xạ phù hợp từ lớp lỗ hổng, hệ thống bổ sung thông tin phân loại như CWE và OWASP. Đối với trường hợp chưa có ánh xạ trực tiếp, Finding có thể được lưu với phân loại chưa xác định và được xem xét bổ sung CWE sau đó khi có đủ căn cứ từ danh mục được kiểm chứng. Việc bổ sung phân loại không làm thay đổi vai trò của bằng chứng hoặc tự nâng mức độ chắc chắn của kết luận lỗ hổng. [14]

Finding Store biểu diễn dữ liệu phát hiện trong báo cáo Markdown và lưu dưới thư mục artifacts/findings của dự án. Cơ chế lưu tránh ghi đè một báo cáo khác chỉ vì trùng tên và hỗ trợ nhận diện báo cáo đã có của cùng Candidate. Sau khi lưu báo cáo, hệ thống kiểm tra lại kết quả đánh giá và bằng chứng. Chỉ khi hồ sơ vẫn hợp lệ, workflow mới ghi nhận Finding đã được lưu cho kết quả xác thực tương ứng. Quy trình này được khái quát trong Hình 2.12.



Hình 2.12. Quy trình xử lý của Confirm Finding

Hình 2.12 minh họa quy trình ghi nhận Finding, trong đó hệ thống kiểm tra các điều kiện xác thực và tính hợp lệ của hồ sơ trước khi tạo báo cáo. Nếu kết quả đánh giá hoặc bằng chứng liên quan thay đổi trong quá trình lưu, báo cáo đã tạo không tự động được công nhận là Finding còn phù hợp với trạng thái kiểm thử hiện tại. Khi các điều kiện ghi nhận được đáp ứng, kết quả được trả về cho Agent và có thể hiển thị trên giao diện để người kiểm thử theo dõi. Cơ chế này giúp phân biệt giữa kết quả xác thực confirmed, báo cáo đã được lưu và Finding được hệ thống ghi nhận hợp lệ. Tuy nhiên, độ chính xác của kết luận vẫn phụ thuộc vào chất lượng quá trình kiểm thử và bằng chứng thu thập được.

2.6. Kết luận chương 2

Chương 2 đã trình bày quá trình phân tích và thiết kế hệ thống AI Pentest Agent dựa trên cơ sở lý thuyết đã xây dựng ở Chương 1. Trước hết, chương đã mô

tả bài toán đặt ra cho hệ thống, xác định rõ đầu vào, đầu ra và cơ chế xử lý theo hướng tiếp cận tương tự mô hình ReAct, gồm ba pha lập kế hoạch, thực hiện hành động và tiếp nhận kết quả. Trên cơ sở đó, các yêu cầu chức năng và phi chức năng của hệ thống đã được xác định, làm căn cứ cho việc thiết kế kiến trúc và các thành phần ở phần sau.

Bên cạnh đó, chương đã đề xuất kiến trúc tổng quát của KAgent với các thành phần chính gồm Interaction Layer, Agent Core, LLM Client, Skills System, Tool Registry, Tool Execution System và Knowledge & State, đồng thời phân rã chi tiết kiến trúc từng thành phần và mối quan hệ phụ thuộc giữa chúng. Luồng xử lý và cơ chế hoạt động của hệ thống, bao gồm quy trình vận hành tổng quan, cơ chế quản lý ngữ cảnh, trạng thái phiên và cơ chế xử lý, thực thi công cụ, cũng đã được làm rõ. Trên nền tảng kiến trúc đó, năm mô-đun chức năng cốt lõi đã được thiết kế chi tiết, gồm mô-đun điều phối và lập kế hoạch tác vụ, mô-đun quản lý tri thức và kỹ năng kiểm thử, mô-đun quản lý và thực thi công cụ, mô-đun quản lý phiên và bộ nhớ, cùng mô-đun quản lý trạng thái và kết quả kiểm thử.

Những nội dung đã trình bày trong chương là cơ sở thiết kế quan trọng để tiến hành xây dựng và hiện thực hệ thống. Trên nền tảng đó, Chương 3 sẽ tập trung trình bày quá trình xây dựng, thực nghiệm và đánh giá hệ thống AI Pentest Agent.

CHƯƠNG 3. XÂY DỰNG VÀ ĐÁNH GIÁ HỆ THỐNG KAGENT

Trên cơ sở kiến trúc và các mô-đun chức năng đã được phân tích, thiết kế ở Chương 2, Chương 3 tập trung vào quá trình hiện thực, triển khai và đánh giá hệ thống AI Pentest Agent. Nội dung chương bắt đầu với việc trình bày môi trường phát triển và các công nghệ được sử dụng để xây dựng KAgent. Từ đó, các thành phần chính của hệ thống được hiện thực và tích hợp theo kiến trúc đã đề xuất, hình thành nguyên mẫu có khả năng tiếp nhận yêu cầu kiểm thử, phối hợp với mô hình ngôn ngữ lớn, lựa chọn và thực thi công cụ, quản lý trạng thái và tổng hợp kết quả trong quá trình kiểm thử. Sau khi hệ thống được xây dựng, các kịch bản thực nghiệm được thiết lập trong môi trường kiểm soát nhằm khảo sát và đánh giá khả năng hoạt động của KAgent đối với các tác vụ kiểm thử xâm nhập ứng dụng web. Những kết quả thu được là cơ sở phân tích khả năng vận hành và các giới hạn của hệ thống trong phạm vi đã khảo sát.

3.1. Môi trường và công nghệ sử dụng

KAgent được xây dựng trên nền tảng Python và hướng tới môi trường Linux, với hình thức vận hành chủ yếu thông qua giao diện dòng lệnh và giao diện văn bản trên terminal. Trong quá trình hiện thực, hệ thống sử dụng các framework và thư viện phục vụ xây dựng giao diện, giao tiếp HTTP, kết nối với mô hình ngôn ngữ lớn, quản lý dữ liệu và tích hợp với các công cụ bên ngoài. Các công nghệ được lựa chọn theo hướng gọn nhẹ, phù hợp với kiến trúc tác nhân đơn và khả năng mở rộng của hệ thống.

3.1.1. Môi trường triển khai

KAgent được thiết kế để hoạt động trong môi trường Linux, phù hợp với đặc thù của các tác vụ kiểm thử xâm nhập và khả năng tích hợp với các công cụ bảo mật dạng dòng lệnh. Hệ thống được phát triển bằng Python và vận hành thông qua giao diện CLI hoặc giao diện TUI trên terminal. Các thành phần của hệ thống có sử dụng cơ chế xử lý bất đồng bộ để hỗ trợ những tác vụ liên quan đến giao tiếp mạng, tương tác với mô hình ngôn ngữ và thực thi công cụ.

Các nền tảng và yêu cầu triển khai chính được tổng hợp trong Bảng 3.1.

Bảng 3.1. Môi trường triển khai hệ thống KAgent

Thành phần	Môi trường và yêu cầu triển khai
Hệ điều hành	Linux
Ngôn ngữ phát triển	Python 3.11 trở lên
Giao diện vận hành	CLI và TUI trên terminal
Cơ chế xử lý	Hỗ trợ xử lý bất đồng bộ với asyncio
Giao tiếp mạng	HTTP/HTTPS
Công cụ bên ngoài	Có thể tích hợp tùy theo cấu hình và tác vụ kiểm thử

3.1.2. Công nghệ và thư viện sử dụng

Các công nghệ và thư viện sử dụng trong KAgent tập trung vào những nhóm chức năng chính gồm xây dựng giao diện tương tác, giao tiếp mạng, kết nối với mô hình ngôn ngữ, tích hợp công cụ và quản lý dữ liệu. Bảng 3.2 tổng hợp các thành phần kỹ thuật chính và vai trò của chúng trong hệ thống.

Bảng 3.2. Các công nghệ và thư viện chính sử dụng trong KAgent

Công nghệ, thư viện	Vai trò trong KAgent
Python	Ngôn ngữ phát triển và môi trường thực thi chính của hệ thống
Textual và Rich	Xây dựng giao diện TUI và trình bày nội dung trên terminal
httpx	Thực hiện giao tiếp HTTP bất đồng bộ với ứng dụng web và các dịch vụ bên ngoài
REST API của LLM	Kết nối với các nhà cung cấp mô hình ngôn ngữ thông qua các adapter của hệ thống
JSON, JSONL, Markdown và YAML	Tổ chức và lưu trữ cấu hình, trạng thái, tri thức, metadata và kết quả của hệ thống

Python được sử dụng làm ngôn ngữ phát triển chính của KAgent, cung cấp nền tảng cho vòng lặp Agent, hệ thống công cụ, quản lý trạng thái và giao diện người dùng. Cơ chế asyncio được sử dụng tại các thành phần cần xử lý bất đồng bộ, đặc biệt trong những hoạt động liên quan đến giao tiếp mạng và thực thi công cụ.

Giao diện người dùng của KAgent được xây dựng bằng Textual kết hợp với Rich. Textual cung cấp các thành phần phục vụ xây dựng giao diện tương tác trên terminal như widget, cửa sổ tương tác và các thành phần hiển thị trạng thái, trong khi Rich hỗ trợ trình bày nội dung có định dạng và Markdown. Sự kết hợp này cho phép KAgent cung cấp giao diện TUI trực tiếp trên terminal mà không yêu cầu giao diện đồ họa riêng.

Đối với giao tiếp mạng, KAgent sử dụng httpx làm HTTP client bất đồng bộ. Thư viện này được sử dụng trong các tác vụ tương tác với ứng dụng web mục tiêu và trong lớp giao tiếp với các dịch vụ mô hình ngôn ngữ. Thay vì phụ thuộc trực tiếp vào SDK riêng của từng nhà cung cấp, KAgent xây dựng các adapter để giao tiếp với các API tương ứng, từ đó tạo ra một lớp tích hợp thống nhất cho nhiều backend LLM.

Để lưu cấu hình, trạng thái và kết quả mà không cần một hệ quản trị cơ sở dữ liệu riêng, KAgent chủ yếu sử dụng các định dạng tệp. JSON và JSONL lưu một số dữ liệu cấu hình, trạng thái và tri thức. YAML khai báo metadata, còn Markdown chứa hướng dẫn kỹ năng và báo cáo phát hiện.

Trong quá trình phát triển, pytest, pytest-asyncio và Pyright được sử dụng để hỗ trợ kiểm thử các thành phần, các luồng xử lý bất đồng bộ và kiểm tra kiểu tĩnh của mã nguồn. Đây là các công cụ hỗ trợ quá trình phát triển, không phải thành phần bắt buộc trong quá trình vận hành KAgent.

3.2. Hiện thực và triển khai hệ thống

3.2.1. Tổ chức mã nguồn và khởi tạo hệ thống

Để hiện thực thiết kế đã trình bày ở Chương 2, mã nguồn KAgent được tổ chức theo trách nhiệm của từng thành phần. Phần chương trình chính nằm trong thư mục src/, các hướng dẫn phục vụ quá trình kiểm thử được đặt trong skills/, còn tests/ chứa mã dùng để kiểm tra hành vi của hệ thống. Bên trong src/, các thành phần được phân thành các nhóm chức năng như Bảng 3.3. [15]

Phần này trình bày cách các thành phần của KAgent phối hợp xử lý yêu cầu và quản lý dữ liệu, kèm theo các hình minh họa hoạt động của hệ thống.

Bảng 3.3. Các nhóm mã nguồn chính của KAgent

Nhóm chức năng	Thư mục chính	Vai trò
Khởi chạy và giao diện	src/cli/, src/ui/	Tiếp nhận tham số khởi chạy, khởi tạo ứng dụng và cung cấp giao diện tương tác trên terminal
Điều phối và kết nối LLM	src/agent/, src/llm/	Điều phối quá trình xử lý yêu cầu và giao tiếp với dịch vụ LLM được cấu hình
Kỹ năng và công cụ	src/skills/, src/tools/	Nạp các kỹ năng kiểm thử, đăng ký và thực thi các công cụ mà Agent có thể sử dụng
Trạng thái và kết quả	src/session/, src/workflow/, src/coverage/, src/findings/, src/memory/	Quản lý phiên làm việc, tiến trình xử lý, mức độ bao phủ và các dữ liệu được lưu lại
Cấu hình và kiểm soát	src/config/, src/permission/, src/target/	Quản lý cấu hình vận hành, phạm vi mục tiêu kiểm thử và cơ chế cấp quyền khi thực thi công cụ

Khi người dùng khởi chạy chương trình, hệ thống phân tích các tham số dòng lệnh và nạp cấu hình đã lưu. Từ các thông tin này, chương trình khởi tạo client giao tiếp với LLM, nạp các kỹ năng có sẵn, tạo các đối tượng quản lý trạng thái, đồng thời đăng ký các công cụ tích hợp sẵn và công cụ từ những tích hợp được bật. Những thành phần đã được chuẩn bị sau đó được kết nối với Agent trước khi giao diện tương tác trên terminal được mở. Cầu nối Burp được khởi tạo khi có tùy chọn khởi chạy tương ứng.

Mỗi lần khởi chạy, KAgent tạo một phiên làm việc mới nếu người dùng không yêu cầu khôi phục phiên trước đó. Tham số --target cho phép chỉ định mục tiêu kiểm thử ban đầu và không phải là điều kiện để tạo phiên. Khi sử dụng --resume, hệ thống nạp lại dữ liệu của phiên đã lưu để tiếp tục quá trình làm việc. Khi ứng dụng kết thúc, chương trình dừng các thành phần theo dõi tệp và đóng các kết nối tích hợp đã được mở. Cách tổ chức và khởi tạo này tạo nền tảng cho quá trình xử lý yêu cầu của Agent, được trình bày trong Mục 3.2.2.

3.2.2. Hiện thực các chức năng chính

Trong KAgent, các chức năng chính được triển khai thành những thành phần phối hợp qua yêu cầu, kết quả công cụ và trạng thái phiên. Agent điều phối quá trình xử lý, trong khi các thành phần chuyên trách thực hiện việc kết nối LLM, nạp kỹ năng, kiểm soát công cụ và lưu dữ liệu. Phần này trình bày cách phối hợp đó trong chương trình. Các nguyên tắc quản lý trạng thái và bằng chứng đã được giải thích ở Chương 2.

a) Hiện thực vòng lặp điều phối của Agent

Khi tiếp nhận yêu cầu, Agent chuẩn bị ngữ cảnh từ lịch sử phiên và thông tin mục tiêu hiện có. Planner sử dụng trạng thái kiểm thử để đề xuất công việc tiếp theo. Agent bổ sung các chỉ dẫn cần thiết rồi gửi yêu cầu tới LLM. Bước lập kế hoạch cung cấp định hướng, còn thao tác kiểm thử được thực hiện qua hệ thống công cụ.

Phản hồi của LLM có thể chứa nội dung trả lời hoặc yêu cầu gọi công cụ. Với lời gọi công cụ, Agent kiểm tra các ràng buộc hiện hành trước khi chuyển sang thực thi. Kết quả trả về được ghi nhận vào ngữ cảnh để phục vụ bước suy luận sau. Nếu công cụ gặp lỗi hoặc bị từ chối, thông tin đó cũng được đưa trở lại Agent để quá trình xử lý dựa trên trạng thái thực tế.

Agent sử dụng trạng thái workflow để quyết định tiếp tục hay kết thúc lượt xử lý. Phản hồi không gọi công cụ chưa đủ để kết thúc khi vẫn còn công việc khả thi. Khi mục tiêu đã hoàn tất, hệ thống chuyển sang tổng hợp kết quả. Những trường hợp bị chặn, hết giới hạn số bước hoặc mọi lời gọi công cụ đều bị từ chối được xử lý theo nhánh tương ứng. Tín hiệu hủy cho phép dừng quá trình khi cần. Phần tổng hợp phải phân biệt kết quả đã có với công việc chưa hoàn thành, vì kết thúc lượt tương tác không đồng nghĩa hoàn tất mục tiêu kiểm thử.

b) Hiện thực kết nối và tương tác với LLM

Thành phần kết nối LLM chuyển yêu cầu của Agent sang giao thức của dịch vụ được cấu hình. Người dùng lựa chọn nhà cung cấp và mô hình thông qua cấu hình của KAgent. Lớp kết nối sử dụng thông tin về mô hình, địa chỉ dịch vụ và xác thực để chuyển thông điệp hội thoại cùng khai báo công cụ trong Chat Request sang định dạng mà dịch vụ hỗ trợ. Nhờ đó, Agent làm việc với một dạng yêu cầu chung thay vì tự xử lý giao thức của từng dịch vụ.



Hình 3.1. Giao diện chọn nhà cung cấp và mô hình LLM

Hình 3.1 minh họa giao diện cấu hình LLM của KAgent. Bên trái là menu /provider, cho phép lựa chọn nhà cung cấp và thiết lập cấu hình kết nối. Bên phải là hộp thoại lựa chọn mô hình của DeepSeek.

Khi nhận Chat Response, lớp kết nối chuyển nội dung trả lời và các lời gọi công cụ (Tool Calls) về cấu trúc thống nhất để Agent xử lý. Khi Streaming được bật và client có hỗ trợ, lớp kết nối tiếp nhận các phần phản hồi, tập hợp nội dung và lời gọi công cụ, đồng thời cung cấp các đoạn văn bản trả lời để cập nhật giao diện. Khi luồng kết thúc, client trả về Chat Response hoàn chỉnh. Với các giao thức đã được hỗ trợ, việc thay đổi nhà cung cấp LLM không yêu cầu thay đổi logic điều phối chính của Agent.

Lớp kết nối xử lý các lỗi xảy ra khi gửi yêu cầu và nhận phản hồi từ dịch vụ LLM. Với lỗi tạm thời, hệ thống mặc định gửi lại yêu cầu tối đa hai lần và tăng thời gian chờ giữa các lần thử. Trong chế độ nhận phản hồi theo luồng (Streaming), việc thử lại chỉ áp dụng ở bước mở luồng, trước khi xử lý nội dung trả về. Nếu vẫn không thể tiếp tục, KAgent thông báo lỗi và ghi nhận công việc chưa hoàn tất. Các thông báo được lọc để hạn chế làm lộ thông tin xác thực.

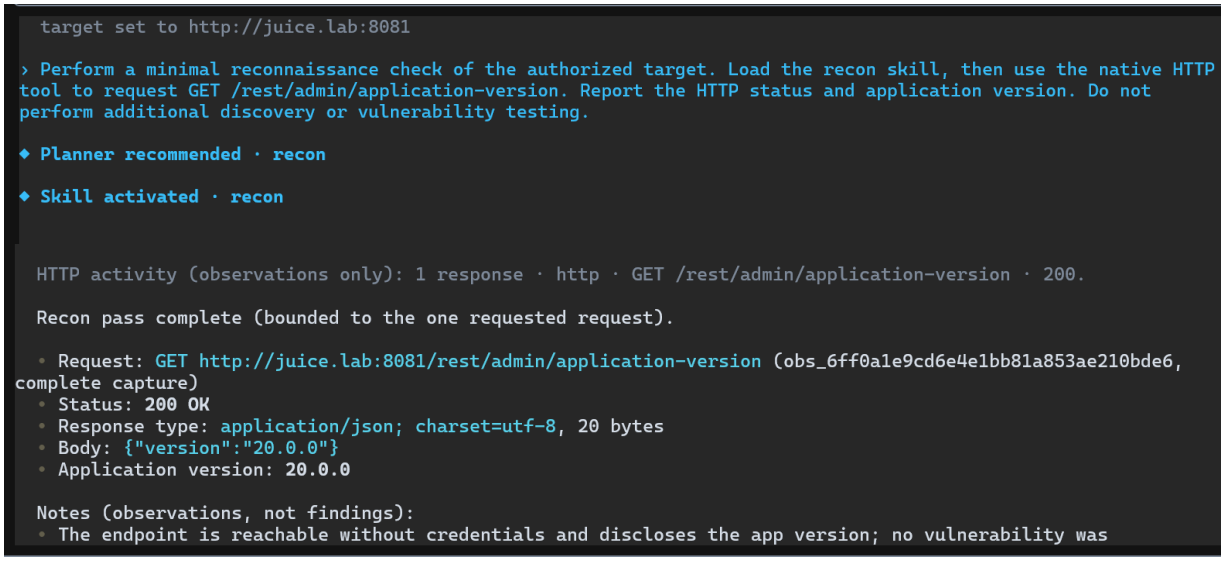
c) Hiện thực cơ chế lựa chọn kỹ năng và lập kế hoạch

Skill Registry quản lý thông tin mô tả các kỹ năng từ tệp SKILL.md. Metadata cho biết chức năng, giai đoạn sử dụng và giới hạn công cụ của mỗi kỹ năng. Hệ thống kiểm tra thông tin này trước khi đưa kỹ năng vào tập khả dụng. Planner sử dụng tập kỹ năng cùng ngữ cảnh kiểm thử để đề xuất hướng xử lý phù hợp.

Với Candidate cần xác minh, Planner ưu tiên Skill chuyên biệt khi xác định được một kỹ năng phù hợp và khả dụng. Nếu có nhiều Skill phù hợp nhưng chưa chọn được, hoặc Skill cần thiết không thể nạp, hệ thống ghi nhận nguyên nhân Candidate chưa thể được kiểm tra tiếp. Nhánh xác minh tổng quát có giới hạn chỉ được xem xét khi không có Skill chuyên biệt và đủ điều kiện thực hiện. Hướng dẫn

của Skill được đưa vào ngữ cảnh LLM sau khi lời gọi load_skill hợp lệ và nạp thành công.

Quá trình nạp kỹ năng và thực hiện yêu cầu HTTP được minh họa trong Hình 3.2.



Hình 3.2. Nạp kỹ năng và thực thi công cụ trên KAgent

Hình 3.2 minh họa quá trình KAgent tiếp nhận yêu cầu kiểm tra phiên bản ứng dụng, đề xuất và nạp Skill recon, sau đó sử dụng công cụ HTTP để thực hiện kiểm tra. Kết quả trả về có mã trạng thái 200 và phiên bản ứng dụng 20.0.0. Ví dụ thể hiện sự phối hợp giữa lựa chọn kỹ năng, thực thi công cụ và tổng hợp kết quả trong một lượt xử lý.

d) Hiện thực cơ chế gọi và kiểm soát công cụ

Lời gọi công cụ do LLM đề xuất được Agent chuyển tới Tool Registry sau khi kiểm tra ràng buộc của kỹ năng và workflow. Mỗi công cụ có tên định danh và mô tả tham số để hệ thống tra cứu đúng chức năng. Registry kiểm tra đối số theo hợp đồng của từng công cụ trước khi chuyển sang bước thực thi. Việc có lời gọi đúng cấu trúc chưa đồng nghĩa thao tác đã được cho phép.

Khi hành động cần phê duyệt, hệ thống chuẩn bị thông tin để người vận hành xem xét qua hộp thoại quyền ở Hình 3.6. Quyết định cho phép có hiệu lực trong phạm vi tương ứng. Chế độ YOLO có thể tự xử lý bước phê duyệt được chính sách hỗ trợ, nhưng không thay thế các kiểm tra về phạm vi, quyền hiện hành và giới hạn thực thi.

Các kiểm soát cụ thể được áp dụng theo loại công cụ. Công cụ HTTP tích hợp kiểm tra nguồn đích, địa chỉ riêng và ngân sách yêu cầu theo cơ chế runtime. Các tích hợp cần bộ thực thi cô lập không được tự chuyển sang thực thi trực tiếp trên

máy vận hành khi bộ thực thi này không khả dụng. Sau khi thao tác kết thúc, kết quả hoặc lỗi được xử lý và trả về Agent. Dữ liệu đưa vào ngữ cảnh được giới hạn dung lượng và xử lý thông tin nhạy cảm theo chính sách của nguồn.

e) Hiện thực quản lý trạng thái và ngữ cảnh phiên

KAgent duy trì lịch sử hội thoại để tiếp tục trao đổi qua nhiều lượt. Khi xử lý một yêu cầu mới, Agent tạo ngữ cảnh làm việc từ lịch sử này và bổ sung chỉ dẫn phù hợp. Bộ nhớ phiên giữ phần thông tin tóm lược cần cho việc tiếp tục nhiệm vụ. Trong khi đó, WorkflowState lưu tiến độ kiểm thử bằng các bản ghi có cấu trúc. Nhờ được quản lý riêng, trạng thái kiểm thử không phụ thuộc hoàn toàn vào nội dung hội thoại.

Session Store lưu các dữ liệu cần thiết để mở lại phiên sau gián đoạn. Khi khôi phục, hệ thống nạp lịch sử, thông tin mục tiêu, bộ nhớ phiên và workflow rồi xây dựng lại chỉ dẫn cho mô hình. Các lượt xác thực dở dang được xử lý theo trạng thái gián đoạn. Việc mở lại phiên không tự khôi phục tiến trình công cụ hoặc quyền thực thi tạm thời. Hành động tiếp theo vẫn phải đáp ứng điều kiện hiện hành.

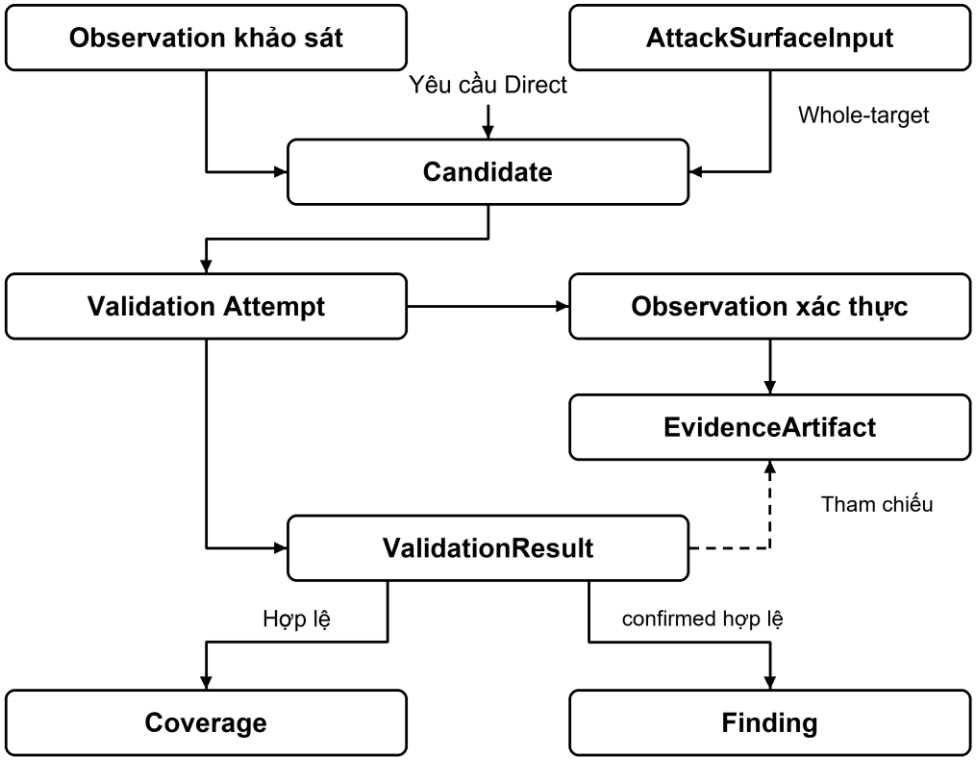
Đối với kết quả công cụ dài từ những nguồn được hỗ trợ, cơ chế giảm nội dung ngữ cảnh giữ bản xem trước và tham chiếu tới phần lưu riêng. Agent có thể đọc lại dữ liệu khi còn đáp ứng điều kiện truy cập của dự án, phiên và thế hệ mục tiêu. Cách lưu này giúp hạn chế lượng nội dung gửi tới LLM mà vẫn giữ khả năng xem lại kết quả.

Khi lịch sử hội thoại tiếp tục tăng, cơ chế thu gọn ngữ cảnh tạo bản tóm lược và kết hợp với phần tương tác gần nhất. Agent so sánh kích thước yêu cầu dự kiến trước và sau thu gọn. Nếu mức giảm chưa đạt ngưỡng quy định, lịch sử cũ được giữ lại. Khi kết quả được chấp nhận, dữ liệu phiên được cập nhật và lưu để tiếp tục xử lý. Các bản ghi workflow vẫn được duy trì riêng, nên một câu tóm tắt về phát hiện không thay thế kết quả xác thực đã ghi nhận.

f) Hiện thực quản lý tiến trình và kết quả kiểm thử

Trong chương trình, Workflow Tool tiếp nhận các yêu cầu cập nhật tiến độ kiểm thử và ghi nhận thay đổi trên WorkflowState. Khi một lượt xác thực được bắt đầu hợp lệ, runtime tạo ngữ cảnh thực thi gắn với Candidate và duy trì ngữ cảnh này trong các thao tác công cụ tiếp theo. Dữ liệu phát sinh được liên kết với lượt kiểm chứng tương ứng để Agent lập hồ sơ và gửi kết quả. Các thành phần điều phối sử dụng trạng thái đã ghi nhận để xác định công việc cần tiếp tục.

Hình 3.3 minh họa các liên kết dữ liệu được sử dụng trong quá trình hiện thực. Các đối tượng và điều kiện sử dụng bằng chứng trong hình được trình bày tại Mục 2.5.5.a.



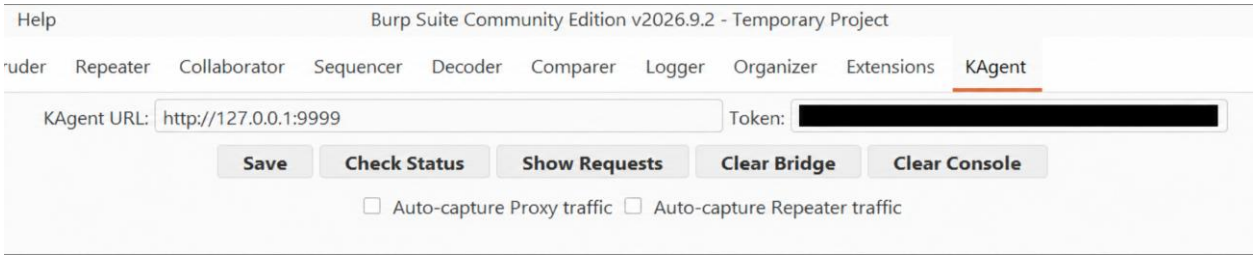
Hình 3.3. Liên kết dữ liệu xác thực và cập nhật kết quả trong KAgent

Hình 3.3 phân biệt dữ liệu thu thập khi khảo sát với bằng chứng dùng để xác minh lỗ hổng. Dữ liệu khảo sát giúp Agent nhận diện điểm nghi vấn và tạo Candidate. Khi kiểm tra một Candidate, hệ thống mở một lượt xác minh (Validation Attempt) và lưu kết quả công cụ thành các bản ghi quan sát (Observation) gắn với lượt đó. Agent sử dụng các bản ghi này để lập tệp bằng chứng (EvidenceArtifact), sau đó gửi kết quả xác minh (ValidationResult) kèm liên kết tới bằng chứng. Đường nét đứt trong hình thể hiện liên kết này, giúp truy xuất dữ liệu làm cơ sở cho kết luận.

Những kết quả đủ điều kiện được Workflow Tool cập nhật vào Coverage để ghi nhận vị trí và loại lỗ hổng đã được kiểm tra. Với kết quả đã xác nhận (confirmed) đáp ứng yêu cầu ghi nhận phát hiện, Agent gọi công cụ confirm_finding để tạo báo cáo. Công cụ kiểm tra kết quả và bằng chứng trước khi lưu báo cáo vào Finding Store. Sau khi lưu thành công và kiểm tra tính nhất quán, hệ thống liên kết Finding với Candidate và kết quả xác minh trong workflow. Nếu thao tác lưu chưa hoàn tất, trạng thái xử lý được giữ lại để có thể tiếp tục.

g) Hiện thực cơ chế tích hợp Burp Suite

Thành phần mở rộng Burp Suite được hiện thực bằng Python trên Jython và giao tiếp với cầu nối HTTP cục bộ của KAgent. Qua API của Burp, thành phần này chuyển các giao dịch được lựa chọn hoặc lưu lượng từ Proxy, Repeater theo cấu hình. KAgent lưu giữ dữ liệu có giới hạn trong bộ nhớ và cung cấp công cụ để Agent truy xuất. Giao diện cấu hình được trình bày trong Hình 3.4.



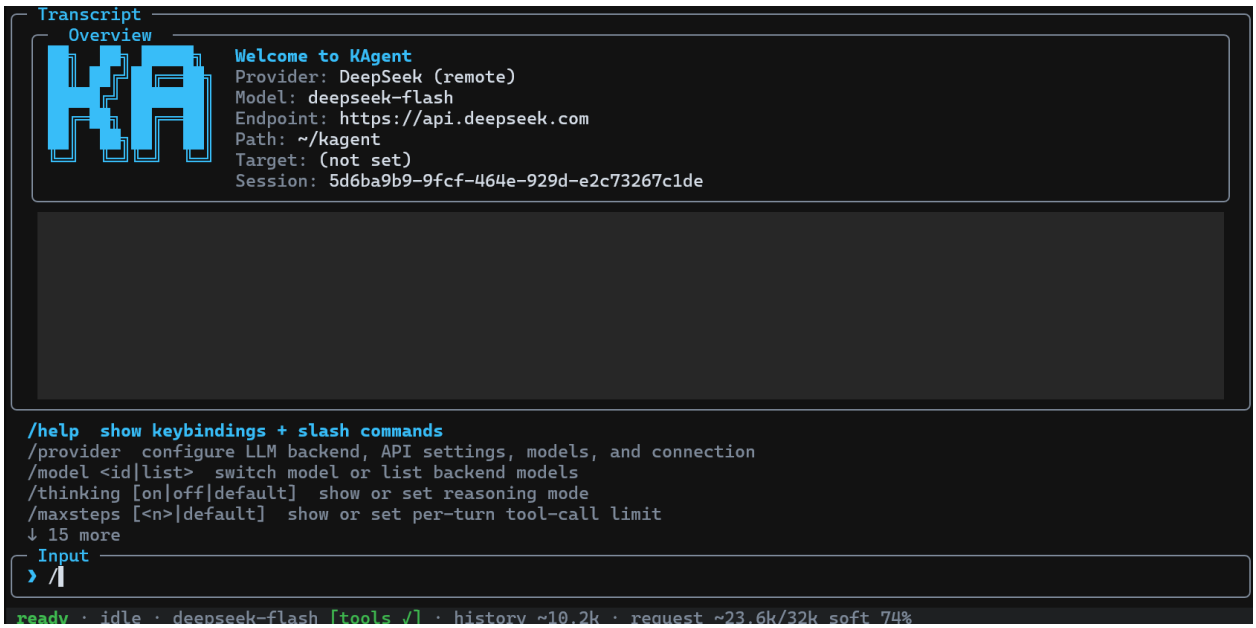
Hình 3.4. Giao diện thành phần mở rộng KAgent trong Burp Suite

Các giao dịch đã tiếp nhận cung cấp cấu trúc yêu cầu và ngữ cảnh cho bước phân tích. Việc chuyển dữ liệu từ Burp không cấp quyền phát lại request hoặc thực hiện kiểm thử. Nếu sử dụng mẫu yêu cầu cho thao tác tiếp theo, công cụ HTTP vẫn phải kiểm tra điều kiện về nguồn, phạm vi và ngữ cảnh xác thực. Cơ chế kiểm soát công cụ tại phần d tiếp tục được áp dụng cho các thao tác này.

Người kiểm thử lựa chọn dữ liệu cung cấp từ Burp, còn KAgent xử lý dữ liệu qua các thành phần điều phối và quản lý trạng thái đã trình bày. Kết quả thực nghiệm tích hợp được phân tích tại Mục 3.3.2.

3.2.3. Giao diện và tương tác với hệ thống

KAgent sử dụng giao diện tương tác trên terminal để người dùng gửi yêu cầu, theo dõi quá trình xử lý và đưa ra quyết định khi cần. Bố cục giao diện được minh họa trong Hình 3.5.



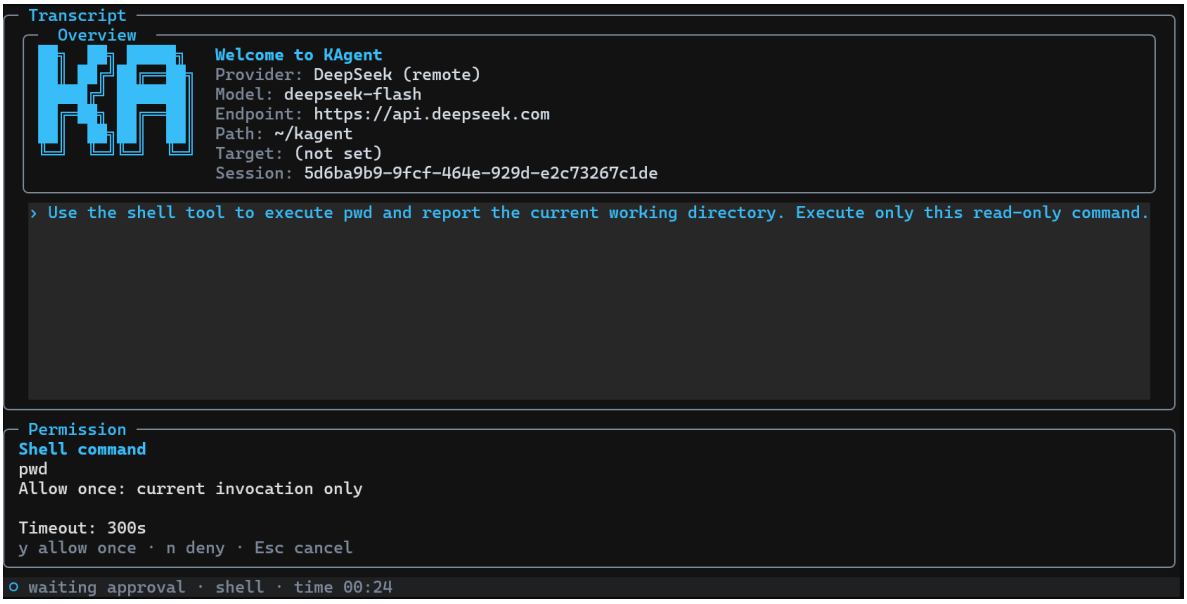
Hình 3.5. Giao diện chính của KAgent

Giao diện gồm bốn khu vực chính. Vùng tổng quan hiển thị cấu hình LLM và thông tin phiên làm việc. Vùng transcript ghi lại yêu cầu của người dùng, phản hồi của Agent, hoạt động công cụ và các thông báo phát sinh. Ô nhập phía dưới cho phép gửi yêu cầu hoặc lệnh điều khiển. Thanh trạng thái cho biết hệ thống đang xử lý hay chờ tương tác, đồng thời cung cấp thông tin về mô hình, mục tiêu kiểm thử và mức sử dụng ngữ cảnh.

Giao diện KAgent còn hỗ trợ tương tác nhiều lượt trong cùng một phiên làm việc. Sau khi Agent hoàn thành một yêu cầu, người dùng có thể xem xét kết quả và tiếp tục đưa ra chỉ dẫn mới để phân tích sâu hơn, thay đổi hướng kiểm tra hoặc yêu cầu tổng hợp thông tin. Chẳng hạn, người dùng có thể yêu cầu KAgent khảo sát một chức năng của ứng dụng, sau đó lựa chọn một đầu vào được phát hiện để tiếp tục xác minh lỗ hổng. Trong quá trình này, KAgent sử dụng ngữ cảnh hội thoại và trạng thái kiểm thử đã lưu để duy trì tính liên tục giữa các lượt, giúp người dùng định hướng công việc mà không cần cung cấp lại toàn bộ thông tin từ đầu.

Ngoài việc gửi yêu cầu thông thường, người dùng có thể nhập một số lệnh trực tiếp trong ô nhập. Chẳng hạn, /provider hỗ trợ thay đổi cấu hình dịch vụ LLM, /target dùng để quản lý mục tiêu kiểm thử và /skills cho phép xem hoặc thay đổi trạng thái bật, tắt của các Skill. Việc quản lý trạng thái Skill trên giao diện chỉ xác định các Skill khả dụng trong hệ thống, còn quá trình sử dụng Skill trong một lượt xử lý được thực hiện theo cơ chế đã trình bày ở Mục 3.2.2. Khi cần thêm thông tin từ người dùng, KAgent có thể hiển thị hộp thoại câu hỏi với các lựa chọn hoặc ô nhập câu trả lời.

Một tương tác quan trọng khác là hộp thoại cấp quyền. Khi một hành động cần người dùng xác nhận, giao diện hiển thị thông tin về công cụ, nội dung hành động và các lựa chọn cho phép hoặc từ chối. Tùy trường hợp, người dùng có thể cho phép hành động một lần hoặc trong phạm vi phiên làm việc.



Hình 3.6. Hộp thoại xác nhận quyền thực thi công cụ

Hộp thoại cấp quyền đặt quyết định của người dùng tại thời điểm Agent chuẩn bị thực hiện một hành động cần xác nhận. Nhờ đó, người dùng có thể tiếp tục theo dõi tiến trình kiểm thử và đồng thời duy trì quyền kiểm soát đối với những thao tác có khả năng tác động tới môi trường mục tiêu. Nhìn chung, giao diện KAgent vừa hỗ trợ quan sát quá trình hoạt động của Agent, vừa cung cấp các điểm tương tác cần thiết trong suốt phiên làm việc.

3.2.4. Cấu hình và triển khai hệ thống

KAgent được xây dựng dưới dạng ứng dụng Python và khởi chạy bằng lệnh kagent. Khi khởi động, chương trình đọc cấu hình, chuẩn bị phiên làm việc và các thành phần cần thiết trước khi mở giao diện TUI để người dùng tương tác.

Các thiết lập chính gồm kết nối LLM, công cụ và Skill, mục tiêu kiểm thử, phiên làm việc, tích hợp Burp Suite và kiểm soát thực thi. Cấu hình được lưu trong tệp ~/.kagent/config.json. Người dùng có thể bổ sung thông tin kết nối LLM qua tham số dòng lệnh hoặc biến môi trường tương ứng với nhà cung cấp. Các nhóm thiết lập được tổng hợp trong Bảng 3.4.

Bảng 3.4. Các nhóm cấu hình phục vụ khởi chạy KAgent

Nhóm cấu hình	Nội dung chính	Điều kiện sử dụng
LLM	Nhà cung cấp, mô hình, địa chỉ dịch vụ và khóa API	Cần thiết trước khi mở TUI
Công cụ và Skill	Công cụ cốt lõi, thư mục Skill và trạng thái bật, tắt	Công cụ cốt lõi được đăng ký khi khởi động. Skill bổ sung là tùy chọn
Target và session	Mục tiêu, phạm vi và dữ liệu phiên	Tạo phiên mới hoặc khôi phục phiên đã lưu
Burp Suite	Cầu nối cục bộ để tiếp nhận yêu cầu HTTP	Chỉ cần khi sử dụng tích hợp Burp Suite
Kiểm soát thực thi	Quyền, phạm vi và giới hạn tài nguyên	Được duy trì cả khi bật YOLO

Thông tin LLM cần được cung cấp trước khi mở TUI. Ở lần khởi chạy đầu tiên, người dùng chỉ định nhà cung cấp và cung cấp khóa API theo dịch vụ sử dụng. Sau đó, menu /provider hỗ trợ lựa chọn mô hình, lưu cấu hình và kiểm tra kết nối. Các công cụ cốt lõi được đăng ký trực tiếp khi khởi động, còn Skill bổ sung được đọc từ những thư mục đã cấu hình.

Từ thư mục mã nguồn và môi trường Python đã chuẩn bị, có thể cài đặt rồi khởi chạy ứng dụng bằng các lệnh sau:

```
python -m pip install .
kagent
```

Khi hoạt động, KAgent duy trì mục tiêu, phạm vi được phép và trạng thái phiên. Người dùng có thể khôi phục phiên đã lưu và theo dõi quyền thực thi qua giao diện. Chế độ YOLO tự chấp thuận các thao tác thuộc chính sách cho phép, đồng thời vẫn chịu các kiểm tra quyền, phạm vi và giới hạn tài nguyên. Quy trình cài đặt, thiết lập cấu hình và khởi chạy hệ thống được trình bày chi tiết tại Phụ lục.

3.3. Thực nghiệm và đánh giá kết quả

Nhằm đánh giá khả năng hoạt động của hệ thống KAgent, đồ án tổ chức hai kịch bản thực nghiệm trong môi trường kiểm thử ứng dụng web được kiểm soát. Kịch bản thứ nhất khảo sát khả năng khám phá, lựa chọn đầu vào và kiểm thử ứng dụng từ địa chỉ mục tiêu ban đầu. Kịch bản thứ hai đánh giá khả năng tiếp nhận yêu cầu HTTP từ Burp Suite và kiểm tra XSS tại đầu vào đã biết. Kết quả được xem xét

qua quá trình điều phối, bằng chứng xác minh, mức độ hoàn tất nhiệm vụ và việc sử dụng công cụ.

3.3.1. Thực nghiệm kiểm thử xâm nhập ứng dụng web

a) Mục tiêu và môi trường thực nghiệm

Thực nghiệm này đánh giá khả năng tự chủ của KAgent khi bắt đầu từ địa chỉ một ứng dụng web. Người dùng giao mục tiêu khảo sát ứng dụng, tìm các đầu vào có liên quan, xác minh SQL Injection và lưu kết quả kèm bằng chứng trong phạm vi được cấp phép. Agent phải sử dụng thông tin thu thập được để lựa chọn vị trí và phương pháp kiểm tra, thay vì nhận sẵn endpoint, tham số hoặc payload.

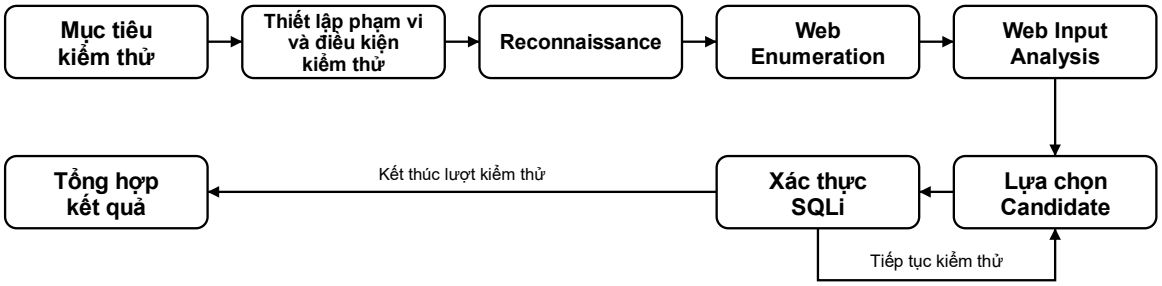
Đối tượng thực nghiệm là OWASP Juice Shop [16] tại http://juice.lab:8081. KAgent sử dụng mô hình deepseek-flash [17] của DeepSeek và không được cung cấp sẵn tài khoản đăng nhập ứng dụng. Nội dung kiểm thử được giới hạn ở SQL Injection. Các kết quả dưới đây được đối chiếu giữa phản hồi HTTP, trạng thái kiểm thử và các hồ sơ đã lưu để phân biệt hoạt động đã thực hiện với nhận định của Agent.

Giới hạn xử lý được đặt ở 200 bước nhằm kiểm soát thời gian và tài nguyên sử dụng, đồng thời tránh để Agent tiếp tục thử nghiệm kéo dài khi chưa tìm được hướng xử lý phù hợp. Chế độ YOLO được bật để tự động chấp thuận các thao tác thuộc chính sách cho phép, giúp Agent thực hiện liên tiếp những công việc đủ điều kiện. Các ràng buộc về mục tiêu, quyền thực thi và tài nguyên vẫn được duy trì. Người dùng tiếp tục tham gia khi cần quyết định về phạm vi hoặc xử lý công việc mà Agent chưa thể tự hoàn tất.

b) Quy trình thực nghiệm

Trước thực nghiệm, người kiểm thử chuẩn bị ứng dụng mục tiêu, khởi tạo phiên KAgent mới và thiết lập phạm vi http://juice.lab:8081. Cấu hình LLM, công cụ, quyền thực thi và giới hạn số bước được ghi nhận trước khi bắt đầu. Agent được yêu cầu tự khảo sát ứng dụng và kiểm thử SQL Injection, không được cung cấp sẵn endpoint, tham số hoặc payload. Người kiểm thử tham gia khi cần quyết định về quyền thực thi hoặc phạm vi, không bổ sung gợi ý về vị trí lỗ hổng.

Quy trình bắt đầu bằng giai đoạn khảo sát để kiểm tra khả năng tiếp cận mục tiêu và thu thập thông tin ban đầu. Agent tiếp tục sử dụng phản hồi HTTP, các đường dẫn và tài liệu API tìm được để liệt kê endpoint cùng tham số đầu vào. Trên cơ sở đó, Agent phân tích chức năng và phản hồi của từng đầu vào để lựa chọn những điểm cần xác minh, ghi nhận thành Candidate. Quy trình tổng quát được minh họa tại Hình 3.7.



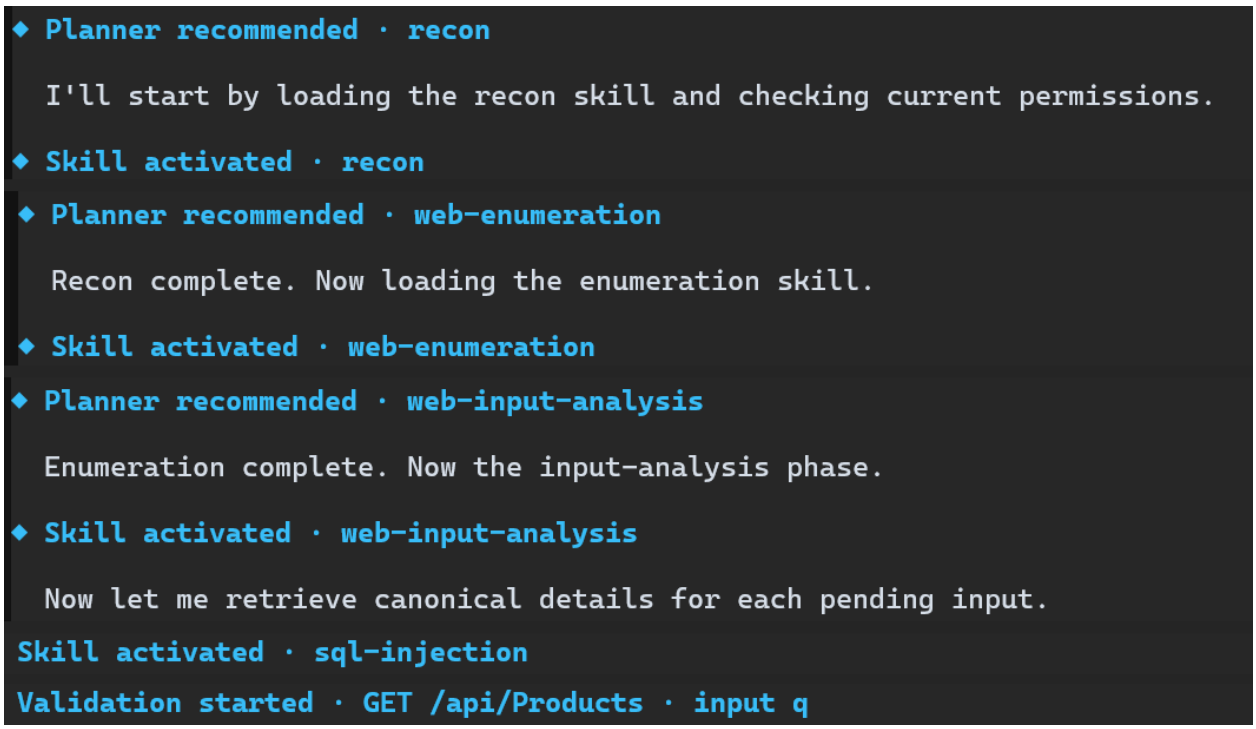
Hình 3.7. Quy trình kiểm thử của KAgent

Trong quá trình kiểm thử, Decision Planner định hướng công việc dựa trên mục tiêu và tiến độ hiện tại. LLM sử dụng dữ liệu quan sát cùng hướng dẫn của Skill để lựa chọn phép thử và công cụ phù hợp. Với các Candidate đã hình thành, Agent sử dụng Skill SQL Injection để so sánh phản hồi đối chứng với phản hồi kiểm thử. Kết quả do công cụ trả về được dùng để đánh giá giả thuyết, điều chỉnh phép thử hoặc bổ sung dữ liệu khi cần.

Workflow duy trì các đầu vào, Candidate, bằng chứng và kết quả xác minh để theo dõi công việc qua từng giai đoạn. Kết quả confirmed đáp ứng các điều kiện về bằng chứng được dùng để tạo Finding. Kết quả not-confirmed chỉ cho biết các phép thử đã thực hiện chưa xác nhận được lỗ hổng. Những trường hợp bị chặn, thiếu bằng chứng hoặc chưa hoàn tất được ghi nhận riêng. Các dữ liệu này là cơ sở để đối chiếu quá trình thực hiện và đánh giá khả năng tự chủ của KAgent.

c) Kết quả thực nghiệm

KAgent thực hiện nhiệm vụ qua bốn giai đoạn gồm khảo sát mục tiêu, liệt kê tài nguyên, phân tích đầu vào và xác minh SQL Injection. Trong quá trình xử lý, Agent lần lượt nạp các Skill recon, web-enumeration, web-input-analysis và sql-injection để sử dụng hướng dẫn kỹ thuật phù hợp với công việc đang thực hiện. Hình 3.8 minh họa các lần nạp Skill trong phiên thực nghiệm.



Hình 3.8. Các Skill được nạp trong quá trình kiểm thử

Ở giai đoạn khảo sát, KAgent gửi yêu cầu tới http://juice.lab:8081 và nhận phản hồi HTTP 200. Agent tiếp tục đọc robots.txt, ghi nhận đường dẫn /ftp trong mục Disallow để khảo sát thêm. Nội dung trang chủ cùng các dấu hiệu điều hướng giúp Agent bước đầu nhận diện OWASP Juice Shop với giao diện ứng dụng một trang.

Với chức năng tìm kiếm, yêu cầu GET /rest/products/search?q=apple trả về JSON chứa ba sản phẩm phù hợp mà không cần thông tin đăng nhập. Yêu cầu GET /rest/user/whoami trả về {"user":{}}, cho thấy yêu cầu hiện tại chưa gắn với người dùng đã xác thực. Các kết quả này giúp Agent xác định những chức năng có thể tiếp tục khảo sát trong điều kiện chưa có tài khoản đăng nhập.

Sang giai đoạn liệt kê tài nguyên, Agent sử dụng nội dung trang chủ, đường dẫn đến các tệp JavaScript và một phần mã đã thu nhận để tìm thêm API. Các thông tin này được bổ sung bằng những yêu cầu thăm dò trực tiếp. Khi đường dẫn dự đoán /api-docs/swagger.json trả về HTML thay vì đặc tả JSON, Agent đọc lại trang Swagger UI và tìm tệp /api-docs/swagger-ui-init.js. Đặc tả OpenAPI được nhúng trong tệp này, nên Agent chuyển sang đọc nguồn đó để tiếp tục tìm hiểu các API.

Đặc tả cung cấp thông tin về thao tác POST /b2b/v2/orders và các trường dữ liệu của yêu cầu đặt hàng. Agent kết hợp nguồn tài liệu này với những đầu vào đã nhận diện qua phản hồi trực tiếp để lập danh sách khảo sát. Kết thúc giai đoạn liệt kê tài nguyên, KAgent ghi nhận tám đầu vào làm cơ sở cho bước phân tích tiếp theo.

Bảng 3.5 tổng hợp Skill, công cụ tiêu biểu và số bước thực hiện theo từng giai đoạn. Một bước tương ứng với một chu kỳ xử lý trong vòng lặp Agent và có thể phát sinh nhiều lời gọi công cụ. Các lượt gọi được phân theo thời điểm nạp Skill chính, bao gồm cả thao tác phải thực hiện lại.

Bảng 3.5. Sử dụng kỹ năng và công cụ trong thực nghiệm

Giai đoạn	Skill và công cụ tiêu biểu	Số bước	Lượt gọi công cụ
Khảo sát	recon permissions_status, http, service_discovery.	6	9
Liệt kê tài nguyên	web-enumeration http, read_discovery_source, content_discovery, workflow.	30	42
Phân tích đầu vào	web-input-analysis http, workflow, ask_user.	17	27
Xác minh SQLi	sql-injection http, read_payloads, workflow, confirm_finding.	57	58

Trong các giai đoạn trên, các Skill cung cấp hướng dẫn kỹ thuật, còn KAgent sử dụng LLM để phân tích dữ liệu quan sát, lựa chọn công cụ và xác định bước xử lý tiếp theo. Công cụ http được dùng xuyên suốt để gửi yêu cầu và thu nhận phản hồi. Các đầu vào, Candidate, tham chiếu bằng chứng và kết quả kiểm thử được ghi nhận qua workflow.

Ở bước khảo sát, Agent tham khảo hướng dẫn của Skill recon, sử dụng permissions_status để xem trạng thái quyền và giới hạn thực thi, đồng thời dùng service_discovery để kiểm tra kết nối đến cổng mục tiêu khi cần. Khi chuyển sang liệt kê tài nguyên theo Skill web-enumeration, Agent sử dụng read_discovery_source để đọc nội dung phản hồi đã lưu và content_discovery để thăm dò các đường dẫn trong phạm vi mục tiêu.

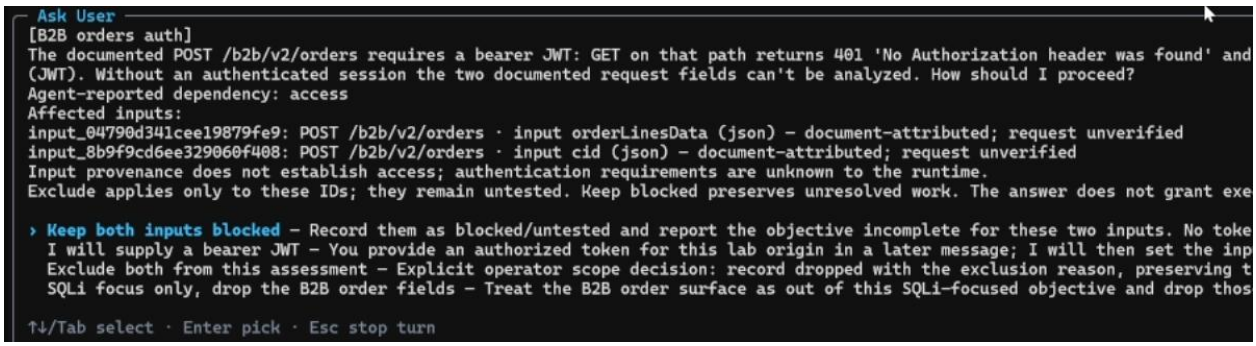
Đối với danh sách đầu vào đã thu thập, Skill web-input-analysis hỗ trợ định hướng phân tích để Agent lựa chọn các Candidate cần xác minh. Những trường hợp cần quyết định của người kiểm thử được làm rõ qua ask_user, như cách xử lý các đầu vào B2B khi thiếu thông tin xác thực.

Với các Candidate SQL Injection, Agent tham khảo hướng dẫn của Skill sql-injection và các mẫu payload qua read_payloads để lựa chọn phép thử, sau đó đánh giá phản hồi thu được. Khi Candidate có kết quả confirmed và đáp ứng điều kiện về bằng chứng, công cụ confirm_finding được sử dụng để lưu báo cáo lỗ hổng đã xác nhận.

Từ danh sách tám đầu vào, Agent xem xét ảnh hưởng của giá trị gửi vào đối với phản hồi. Tại /rest/products/search và /api/Products, giá trị q=apple trả về các sản phẩm phù hợp, còn một giá trị đánh dấu không trùng tên sản phẩm trả về tập rỗng. Agent cũng thử tham số criteria tại /rest/products/search, nhưng phản hồi vẫn là danh sách sản phẩm đầy đủ như khi không gửi tham số. Vì vậy, q được ưu tiên để tiếp tục kiểm tra, còn criteria chưa cho thấy tác dụng lọc dữ liệu trong phép thử đã thực hiện.

Với chức năng đăng nhập, yêu cầu POST /rest/user/login chứa email và mật khẩu không hợp lệ trả HTTP 401 cùng thông báo Invalid email or password. Agent lựa chọn trường email để kiểm tra giả thuyết đầu vào có thể tác động đến truy vấn tìm tài khoản. Hai tham số q và trường email vì vậy được ưu tiên cho bước xác minh SQL Injection. Ở thời điểm này, sự thay đổi kết quả tìm kiếm và phản hồi đăng nhập thất bại mới là cơ sở lựa chọn điểm kiểm tra, chưa xác nhận được lỗ hổng.

Riêng cid và orderLinesData là hai trường trong phần thân JSON của thao tác POST /b2b/v2/orders được mô tả trong OpenAPI. Tài liệu quy định sử dụng bearer token. Trong khi đó, yêu cầu GET đến cùng đường dẫn trả HTTP 401 và thông báo thiếu thông tin xác thực. Phản hồi này ghi nhận hành vi của GET, chưa cho biết cách xử lý yêu cầu POST vì thao tác đó chưa được gửi. Khi chưa có token để kiểm tra trực tiếp các trường đã mô tả, KAgent yêu cầu người dùng xác định phương án xử lý như Hình 3.9.



Hình 3.9. KAgent yêu cầu xác định phương án xử lý các đầu vào B2B

Người kiểm thử quyết định loại cả hai trường B2B khỏi phạm vi xác minh. Agent cập nhật trạng thái và giữ rõ đây là hai đầu vào chưa được kiểm thử, không

phải kết quả âm tính. Kết thúc phân tích, sáu đầu vào được ghi nhận đã phân tích và hai đầu vào bị loại khỏi phạm vi. Agent tạo ba Candidate SQLi tại hai tham số q và trường email. Các đầu vào criteria, password và comment được giữ lại cùng lý do chưa lựa chọn để xác minh.

Ba Candidate được xác minh lần lượt tại q của GET /api/Products, email của POST /rest/user/login và q của GET /rest/products/search. Mỗi Candidate được kiểm tra riêng vì kết quả của một API không xác định hành vi của API còn lại.

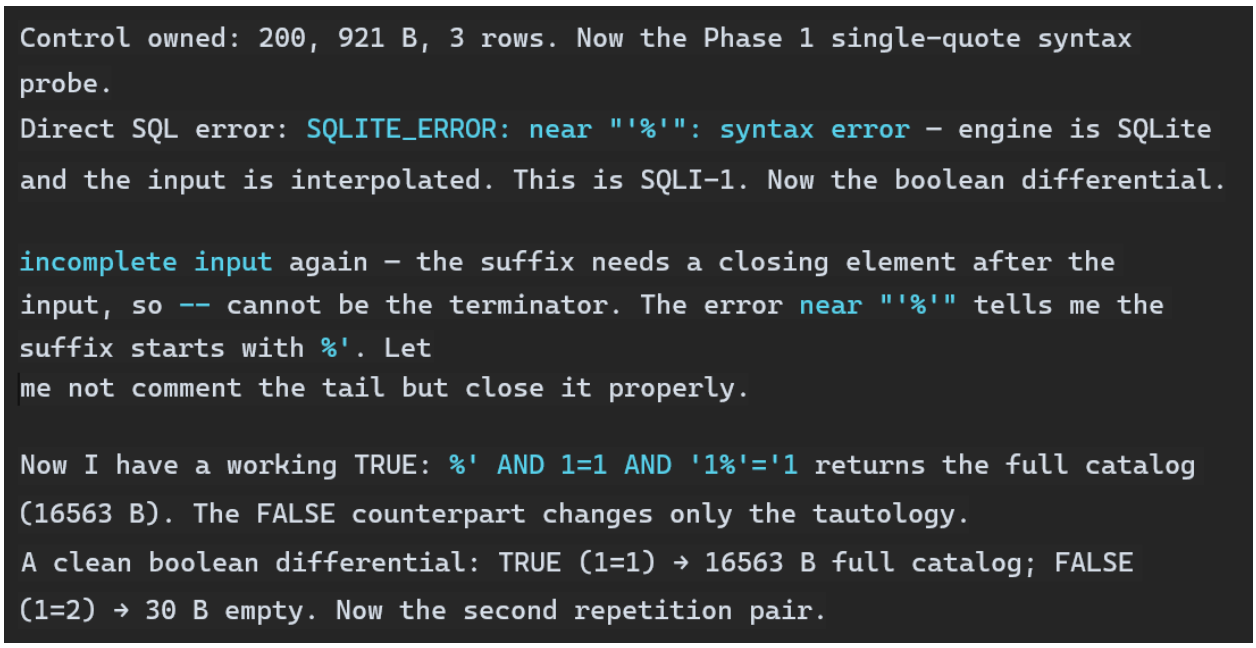
Tại /api/Products, Agent kiểm tra giả thuyết q có thể làm thay đổi điều kiện SQL dùng để tìm sản phẩm. Giá trị apple trả về ba sản phẩm, còn ký tự đại diện % trả về danh sách đầy đủ. Tuy nhiên, apple' và các giá trị chứa dấu nháy đơn kèm biểu thức SQL đều trả HTTP 200 với tập rỗng 30 byte. Các phép thử này không tạo ra lỗi cơ sở dữ liệu hoặc sự khác biệt hỗ trợ giả thuyết SQL Injection.

Agent thu lại phản hồi đối chứng của ký tự % sau khi bản ghi trước bị cắt, rồi ghi nhận kết quả not-confirmed cho Candidate này. Kết quả chỉ cho biết những phép thử đã thực hiện chưa xác nhận được SQL Injection. Phản hồi quan sát được chưa đủ để xác định cơ chế xử lý truy vấn bên trong hoặc kết luận endpoint an toàn.

Tại /rest/user/login, Agent giữ nguyên mật khẩu sai và thay đổi giá trị email trong phần thân JSON. Với ' OR 1=1--, ứng dụng trả HTTP 200 cùng phản hồi 799 byte có token xác thực. Khi chỉ đổi điều kiện thành ' OR 1=2--, ứng dụng trả HTTP 401 cùng phản hồi 26 byte chứa thông báo Invalid email or password. Agent lặp lại cả hai giá trị và thu được kết quả tương ứng ở từng nhánh.

Bốn yêu cầu được thực hiện riêng biệt và có bốn phản hồi đầy đủ, tạo thành hai cặp đối chứng TRUE/FALSE độc lập. Sự khác biệt nhất quán khi chỉ thay điều kiện đúng và sai là căn cứ để KAgent ghi nhận confirmed cho khả năng vượt qua kiểm tra đăng nhập bằng SQL Injection. Thông tin trong phản hồi và token trả về xác định tài khoản admin@juice-sh.op với vai trò admin. Token chưa được dùng để truy cập API khác, nên thực nghiệm chưa xác minh khả năng sử dụng các chức năng quản trị.

Với /rest/products/search, giá trị q=apple' làm ứng dụng trả HTTP 500 và thông báo lỗi SQLite trong phản hồi 994 byte. Agent dựa vào lỗi này để kiểm tra giả thuyết đầu vào có thể làm thay đổi cú pháp truy vấn. Một số biểu thức ban đầu tiếp tục gây lỗi, nên Agent điều chỉnh cách kết thúc giá trị gửi vào cho đến khi thu được phép thử có thể so sánh điều kiện đúng và sai. Hình 3.10 minh họa quá trình điều chỉnh theo phản hồi của ứng dụng.



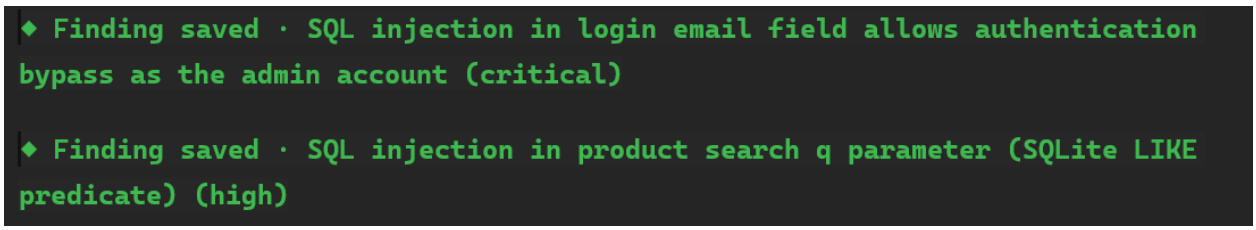
Hình 3.10. Điều chỉnh phép thử SQLi theo phản hồi của ứng dụng

Phép thử với q mang giá trị %' AND 1=1 AND '1%'='1 trả HTTP 200 và danh sách sản phẩm 16.563 byte. Khi giữ nguyên các thành phần khác và chỉ đổi 1=1 thành 1=2, phản hồi vẫn có mã HTTP 200 nhưng chỉ còn tập rỗng 30 byte. Kết hợp với lỗi SQLite ở phép thử dấu nháy đơn, sự thay đổi này cung cấp bằng chứng rằng biểu thức SQL trong đầu vào có thể tác động đến kết quả tìm kiếm.

Cấu trúc truy vấn bên trong vẫn là suy luận từ phản hồi, chưa được kiểm tra qua mã nguồn.

Ở Product Search, một phản hồi TRUE đầy đủ được dùng cho cả hai cặp đối chứng, còn FALSE có hai phản hồi riêng biệt. Một yêu cầu TRUE trước đó chỉ lưu được 4.096 trong tổng số 16.563 byte và bị loại khỏi tập bằng chứng cuối. Vì vậy, dữ liệu chưa thể hiện hai cặp TRUE/FALSE độc lập với đầy đủ nội dung phản hồi, dù kết quả đã được hệ thống chấp nhận ở trạng thái confirmed. Các phép thử cũng chưa kiểm tra việc đọc dữ liệu ngoài danh mục sản phẩm hoặc thay đổi dữ liệu trong cơ sở dữ liệu.

KAgent lưu hai Finding cho chức năng đăng nhập và tìm kiếm sản phẩm như Hình 3.11. Mức độ nghiêm trọng do Agent đánh giá lần lượt là critical và high. Các báo cáo được liên kết với bằng chứng tương ứng.



Hình 3.11. Lưu hai Finding SQL Injection sau xác minh

Bảng 3.6 tổng hợp các kết quả được ghi nhận qua những giai đoạn thực nghiệm.

Bảng 3.6. Tổng hợp kết quả qua các giai đoạn thực nghiệm

Giai đoạn	Kết quả
Liệt kê đầu vào	Ghi nhận 8 đầu vào.
Phân tích đầu vào	Phân tích 6 đầu vào, loại 2 đầu vào B2B theo quyết định của người kiểm thử.
Lựa chọn điểm xác minh	Tạo 3 Candidate SQL Injection.
Xác minh	Ghi nhận 2 kết quả confirmed và 1 kết quả not-confirmed.
Lưu phát hiện	Lưu 2 Finding SQL Injection.

Bảng 3.7 so sánh kết quả xác minh và Finding được lưu tại ba Candidate SQL Injection.

Bảng 3.7. Kết quả xác minh SQL Injection tại ba Candidate

Điểm kiểm tra	Quan sát chính	Kết quả xác minh	Finding
GET /api/Products tham số q	Các giá trị chứa dấu nháy đơn và biểu thức SQL đã thử đều trả về tập rỗng. Chưa có bằng chứng cho thấy biểu thức SQL trong đầu vào được thực thi.	not-confirmed	Không tạo
POST /rest/user/login trường email	Điều kiện đúng trả phản hồi đăng nhập thành công kèm token, điều kiện sai trả thông báo đăng nhập thất bại. Kết quả lặp lại qua hai cặp đối chứng độc lập.	confirmed	Đã lưu, mức critical
GET /rest/products/search tham số q	Ứng dụng trả lỗi SQLite khi thêm dấu nháy đơn. Điều kiện đúng trả danh sách sản phẩm, điều kiện sai trả tập rỗng.	confirmed	Đã lưu, mức high

d) Nhận xét và đánh giá

Thực nghiệm cung cấp bằng chứng về khả năng KAgent tự tổ chức kiểm thử từ thông tin mục tiêu ban đầu. Agent sử dụng phản hồi ứng dụng và tài liệu API để hình thành các giả thuyết, thay vì chỉ tiếp nhận điểm kiểm tra có sẵn. Việc kiểm tra lại nguồn OpenAPI và phân biệt ảnh hưởng của q với criteria thể hiện những quyết định dựa trên dữ liệu. Trong phạm vi này, Planner hỗ trợ giữ định hướng công việc, còn lựa chọn kỹ thuật của LLM được triển khai và kiểm tra qua công cụ.

Giới hạn bao phủ xuất hiện ngay từ cách thu nhận thông tin. Agent chọn mức thu nhận 16 KiB cho tệp JavaScript chính nên chỉ có một phần nội dung để phân tích. Tiêu chí hoàn tất của kỹ năng khảo sát dựa trên danh sách đã ghi nhận cùng các giới hạn được công bố, cho phép chuyển giai đoạn khi ứng dụng chưa được khảo sát đầy đủ. Các bước phân tích sau kế thừa phạm vi thông tin này. Hai trường B2B được loại theo quyết định của người dùng, cần được phân biệt với tài nguyên chưa đọc hết hoặc đầu vào chưa được chọn để xác minh.

Khả năng thích ứng của LLM thể hiện ở việc sửa phép thử theo phản hồi thực tế. Tại Product Search, Agent phải điều chỉnh các biểu thức ban đầu trước khi thu được sự khác biệt giữa điều kiện đúng và sai. Tại /api/Products, phản hồi đối chứng

được thu lại khi bản ghi trước không đầy đủ. Những hoạt động này thể hiện khả năng bổ sung dữ liệu để tiếp tục kiểm tra theo hướng dẫn kỹ thuật. Các lựa chọn cần sửa cũng phản ánh sự phụ thuộc của tiến trình vào chất lượng diễn giải phản hồi và cách tổ chức phép thử.

Việc hệ thống chấp nhận kết quả chưa bảo đảm hai Finding có chất lượng bằng chứng tương đương. Các lần lặp riêng biệt ở chức năng đăng nhập củng cố tính nhất quán của đối chứng. Product Search có dấu hiệu SQL và phản hồi thay đổi theo điều kiện, nhưng thiếu một phản hồi nhánh đúng đầy đủ để tạo cặp lặp độc lập thứ hai. Yêu cầu này đã có trong Skill. Hạn chế quan sát được vì vậy liên quan đến việc áp dụng hướng dẫn và kiểm tra hồ sơ khi kết luận. Kết quả chưa xác nhận ở /api/Products cũng chỉ có giá trị với những phép thử đã thực hiện.

Thực nghiệm cho thấy KAgent có thể duy trì quá trình kiểm thử qua nhiều giai đoạn, nhưng khả năng tự điều phối chưa đồng nghĩa với khả năng tự hoàn tất nhiệm vụ. Việc xác định phạm vi và xác nhận kết thúc vẫn có thể cần người dùng tham gia. Đồng thời, một lượt thực thi trên một ứng dụng chưa đủ để đánh giá tính ổn định, khả năng tái lập hoặc hiệu quả trên các mục tiêu khác. Các kết quả ở đây phản ánh năng lực đã quan sát trong phạm vi thực nghiệm, còn mức độ tin cậy và khả năng khái quát cần được đánh giá qua những lượt chạy lặp có kiểm soát.

3.3.2. Thực nghiệm tích hợp Burp Suite với KAgent

a) Mục tiêu và môi trường thực nghiệm

Kịch bản này đánh giá khả năng phối hợp giữa Burp Suite và KAgent trong việc kiểm tra XSS đối với một yêu cầu HTTP do người dùng lựa chọn. Burp Suite cung cấp yêu cầu đã thu thập làm dữ liệu đầu vào. KAgent sử dụng Skill tương ứng để phân tích phản hồi của ứng dụng, xây dựng giả thuyết và điều phối các công cụ kiểm chứng. Trọng tâm của thực nghiệm là quá trình xử lý từ yêu cầu đầu vào đến kết luận xác minh, kèm theo bằng chứng làm cơ sở cho kết luận.

Đối tượng thực nghiệm là chức năng tìm kiếm bài viết trong một bài lab thuộc PortSwigger Web Security Academy [18]. Giá trị tìm kiếm được đưa vào nội dung HTML và một chuỗi JavaScript. Trong chuỗi này, ứng dụng thêm dấu gạch chéo ngược trước dấu nháy đơn hoặc dấu gạch chéo ngược có trong đầu vào để các ký tự đó được xử lý như dữ liệu của chuỗi. Môi trường thực nghiệm gồm Burp Suite và KAgent sử dụng mô hình deepseek-flash của DeepSeek. Giới hạn xử lý được đặt ở 100 bước nhằm kiểm soát thời gian và tài nguyên sử dụng. Phạm vi kiểm tra được giới hạn trong bài lab này.

Trong quá trình thực nghiệm, người kiểm thử bật chế độ YOLO để KAgent tự động chấp thuận các thao tác thuộc chính sách cho phép. Các điều kiện về phạm vi, quyền thực thi và giới hạn tài nguyên vẫn được duy trì.

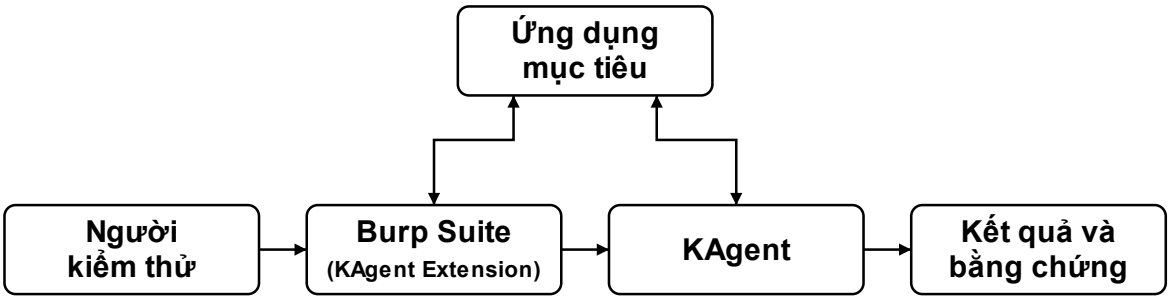
Yêu cầu HTTP được chọn là GET /?search=test, với tham số search là đầu vào cần kiểm tra. Dựa trên dữ liệu do Burp Suite cung cấp, KAgent lựa chọn phép thử và xác minh khả năng khai thác XSS tại tham số này. Kịch bản tập trung đánh giá khả năng phân tích và kiểm chứng một đầu vào đã xác định, không đánh giá khả năng khám phá toàn bộ bề mặt tấn công của ứng dụng.

b) Quy trình thực nghiệm

Ở giai đoạn chuẩn bị, KAgent được khởi tạo với phạm vi kiểm thử giới hạn trong bài lab đã xác định trên PortSwigger Web Security Academy. Người kiểm thử sử dụng Burp Suite để tương tác với chức năng tìm kiếm bài viết và thu thập yêu cầu GET /?search=test. Yêu cầu này sau đó được chuyển sang KAgent bằng chức năng Send to KAgent, cung cấp ngữ cảnh ban đầu cho quá trình phân tích.

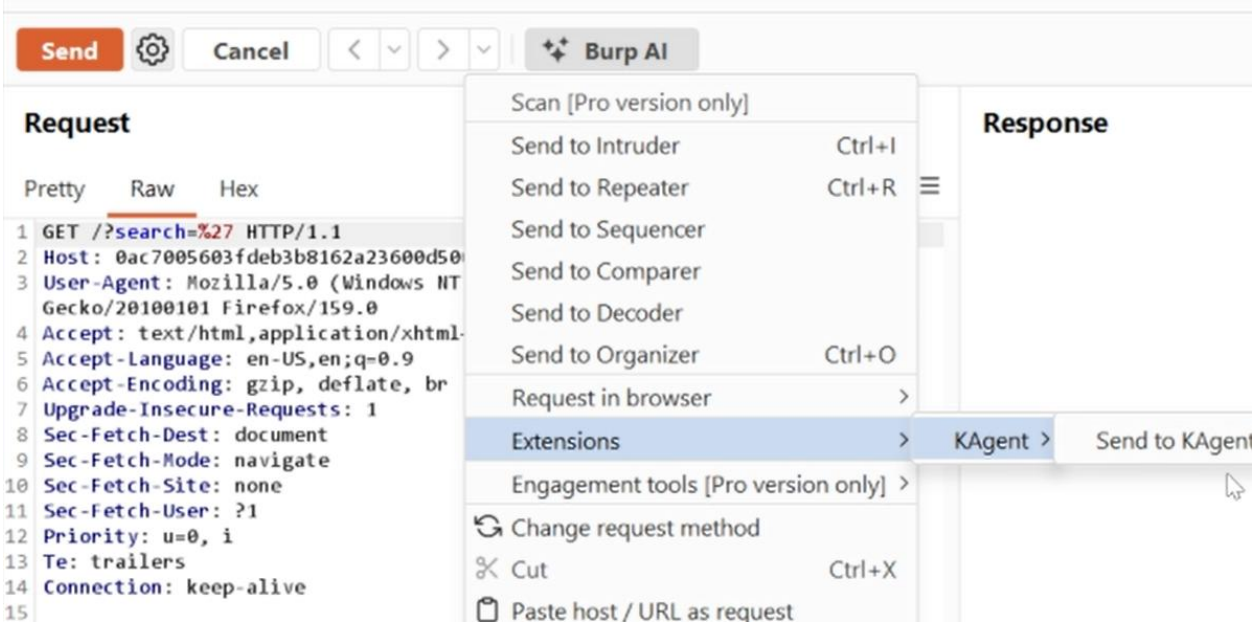
Sau khi dữ liệu được tiếp nhận, người kiểm thử chọn yêu cầu tương ứng trên KAgent và chỉ định kiểm tra XSS tại tham số search. KAgent nạp Skill phù hợp, phân tích vị trí xuất hiện của giá trị tìm kiếm trong phản hồi và xem xét cách ứng dụng xử lý các ký tự đặc biệt. Từ kết quả phân tích, KAgent xây dựng giả thuyết, lựa chọn phép thử và điều phối các công cụ để xác minh. Kết quả kiểm tra được ghi nhận cùng bằng chứng làm cơ sở cho kết luận. Trong quá trình này, Burp Suite cung cấp yêu cầu đầu vào, còn KAgent đảm nhiệm việc phân tích và điều phối các bước kiểm chứng.

Việc chuyển và lựa chọn yêu cầu HTTP cung cấp ngữ cảnh cho KAgent nhưng không đồng thời cấp quyền thực hiện các phép thử. Những hành động tiếp theo vẫn phải tuân thủ phạm vi kiểm thử và cơ chế kiểm soát quyền thực thi của hệ thống. Khi một hành động cần được phê duyệt, người kiểm thử xem xét đề xuất và quyết định cho phép hoặc từ chối. Quy trình phối hợp giữa Burp Suite, KAgent và người kiểm thử được minh họa tại Hình 3.12.



Hình 3.12. Quy trình phối hợp giữa Burp Suite và KAgent

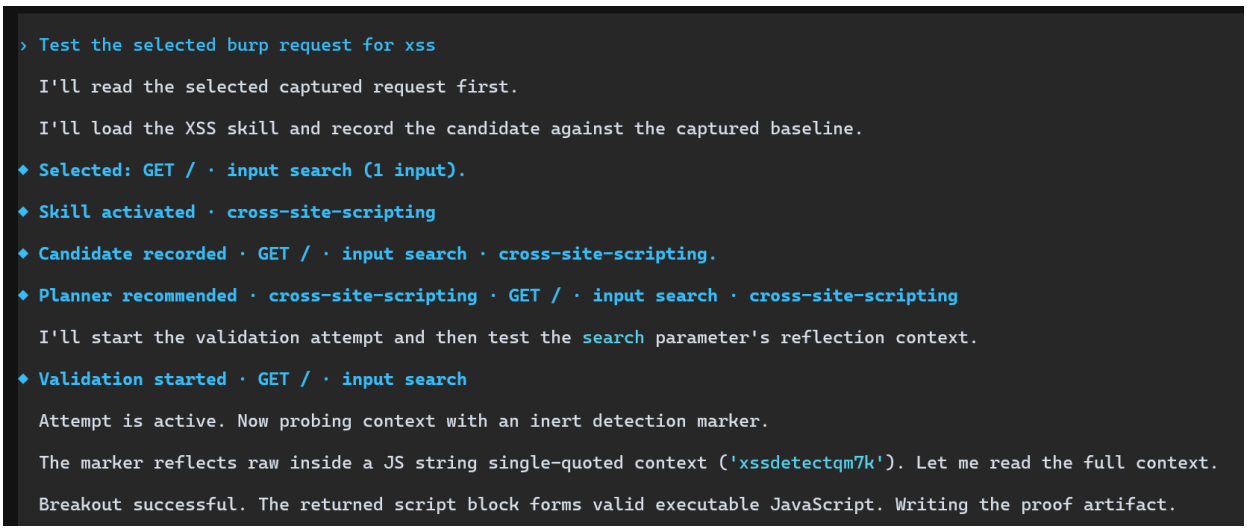
Quy trình trên thể hiện sự phân công giữa thao tác của người kiểm thử và xử lý của hệ thống. Người kiểm thử chủ động lựa chọn request và xác định yêu cầu phân tích, Burp Suite hỗ trợ thu thập và chuyển tiếp dữ liệu HTTP, còn KAgent thực hiện các bước xác minh trực tiếp tới ứng dụng mục tiêu trong phạm vi được phép. Các kết quả thu được được ghi nhận cùng quan sát và bằng chứng tương ứng, làm cơ sở cho việc tổng hợp phát hiện và đánh giá kết quả thực nghiệm.



Hình 3.13. Chuyển yêu cầu tìm kiếm từ Burp Repeater tới KAgent

c) Kết quả thực nghiệm

KAgent tiếp nhận yêu cầu tìm kiếm GET /?search=test do Burp Suite chuyển tới. Người kiểm thử lựa chọn yêu cầu này và chỉ định kiểm tra XSS tại tham số search trong URL. Hình 3.14 minh họa thao tác lựa chọn yêu cầu và kích hoạt Skill xss.



Hình 3.14. Nạp Skill XSS và triển khai xác minh tham số search

KAgent ghi nhận điểm nghi vấn (Candidate) tại tham số search và bắt đầu kiểm tra xác nhận XSS (Validation). Đầu tiên, Agent gửi giá trị đánh dấu xssdetectqm7k và xác định dữ liệu xuất hiện trong phần tử h1 và trong chuỗi JavaScript gán cho biến searchTerms. Từ kết quả này, Agent tiếp tục kiểm tra cách ứng dụng xử lý đầu vào tại hai vị trí.

Khi gửi đầu vào chứa chuỗi sau:

```
</script><script>document.title=731942</script>
```

Agent nhận thấy các ký tự tạo thẻ được mã hóa tại h1 nhưng vẫn được giữ nguyên trong khối script. Từ đó, Agent nhận định rằng chuỗi </script> có thể kết thúc khối script ban đầu, cho phép phần đầu vào tiếp theo tạo một khối script mới chứa lệnh thay đổi tiêu đề trang.

Agent tiếp tục bổ sung dấu nháy đơn trước chuỗi đóng thẻ và sử dụng read_http_observation để đọc lại phản hồi. Kết quả cho thấy dấu nháy đơn được escape thành \', còn chuỗi </script> vẫn không thay đổi. Điều này củng cố nhận định rằng cơ chế xử lý dấu nháy đơn chưa ngăn được việc chèn một khối script mới. Cơ sở xác minh là cấu trúc HTML và mã JavaScript trong phản hồi, còn việc thực thi trên trình duyệt chưa được quan sát trực tiếp.

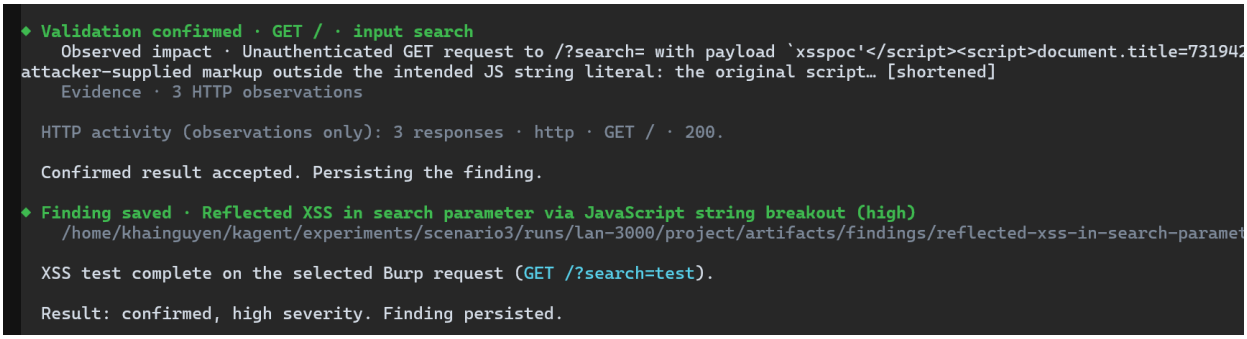
Bảng 3.8 tổng hợp các giá trị đầu vào đã thử, kết quả quan sát từ phản hồi và nhận định của KAgent trong quá trình xác minh XSS tại tham số search.

Bảng 3.8. Các phép thử và kết quả phân tích tại tham số search

Phép thử	Giá trị tham số search	Kết quả quan sát và nhận định
Xác định vị trí phản ánh	xssdetectqm7k	Dữ liệu xuất hiện trong phần tử h1 và chuỗi JavaScript của biến searchTerms. Agent xác định hai vị trí cần kiểm tra.
Kiểm tra khả năng chèn thẻ script	xsspoc</script> <script> document.title=731942 </script>	Các ký tự tạo thẻ được mã hóa tại h1 nhưng được giữ nguyên trong khối script. Agent nhận định đầu vào có thể kết thúc khối script ban đầu và tạo khối script mới.
Kiểm tra ảnh hưởng của dấu nháy đơn	xsspoc'</script> <script> document.title=731942 </script>	Dấu nháy đơn được escape thành \', còn chuỗi </script> vẫn được giữ nguyên. Agent nhận định việc escape dấu nháy đơn chưa ngăn được cách chèn mã này.

Qua các phép thử, KAgent nhận định rằng việc escape dấu nháy đơn chưa ngăn được khả năng chèn mã tại tham số search. Đầu vào vẫn có thể kết thúc khối script ban đầu và tạo một khối script mới chứa lệnh JavaScript hợp lệ. Ba phản hồi HTTP được lưu làm bằng chứng cho quá trình phân tích và kết luận này.

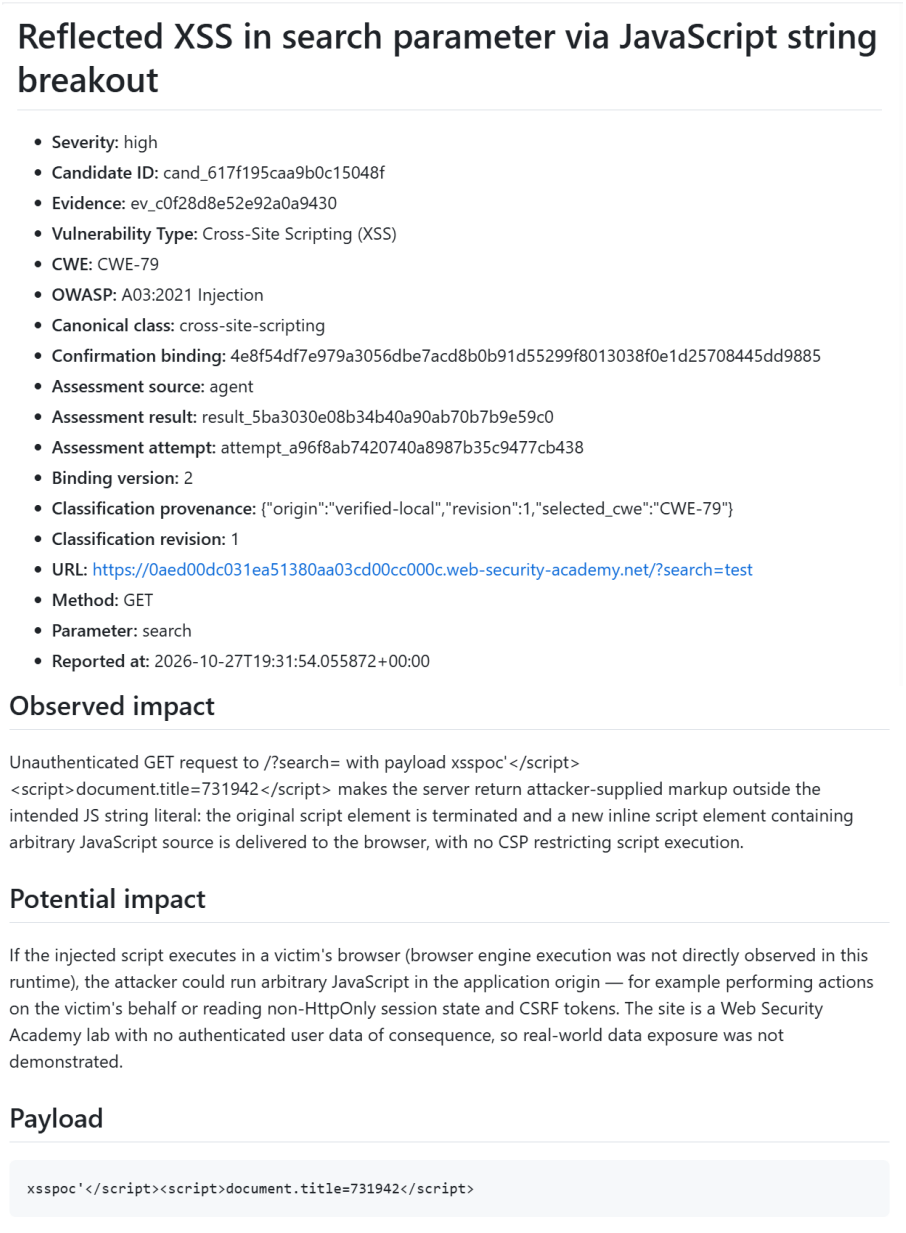
Dựa trên bằng chứng thu được, KAgent ghi nhận kết quả xác minh (ValidationResult) ở trạng thái confirmed, cập nhật Coverage và gọi công cụ confirm_finding để lưu một Finding XSS với mức độ nghiêm trọng high, thuộc CWE-79. Sau khi lưu Finding, hệ thống ghi nhận hoàn tất quy trình xử lý. Hình 3.15 thể hiện kết quả xác minh, thông tin về ba phản hồi HTTP được dùng làm bằng chứng và thông báo lưu Finding.



Hình 3.15. Kết quả xác minh và thông báo lưu Finding XSS

Báo cáo Finding tại Hình 3.16 ghi nhận lỗ hổng XSS ở tham số search và liên kết tới bằng chứng về các phép thử cùng phản hồi tương ứng. Qua nội dung trả về, KAgent xác định đầu vào có thể đóng thẻ script ban đầu và chèn một khối script

mới. Báo cáo cũng nêu rõ kết luận này dựa trên phân tích phản hồi, chưa có quan sát trực tiếp về việc mã JavaScript được thực thi trên trình duyệt.



Reflected XSS in search parameter via JavaScript string breakout

- **Severity:** high
- **Candidate ID:** cand_617f195caa9b0c15048f
- **Evidence:** ev_c0f28d8e52e92a0a9430
- **Vulnerability Type:** Cross-Site Scripting (XSS)
- **CWE:** CWE-79
- **OWASP:** A03:2021 Injection
- **Canonical class:** cross-site-scripting
- **Confirmation binding:** 4e8f54df7e979a3056dbe7acd8b0b91d55299f8013038f0e1d25708445dd9885
- **Assessment source:** agent
- **Assessment result:** result_5ba3030e08b34b40a90ab70b7b9e59c0
- **Assessment attempt:** attempt_a96f8ab7420740a8987b35c9477cb438
- **Binding version:** 2
- **Classification provenance:** {"origin":"verified-local","revision":1,"selected_cwe":"CWE-79"}
- **Classification revision:** 1
- **URL:** https://0aed00dc031ea51380aa03cd00cc000c.web-security-academy.net/?search=test
- **Method:** GET
- **Parameter:** search
- **Reported at:** 2026-10-27T19:31:54.055872+00:00

Observed impact

Unauthenticated GET request to /?search= with payload xsspoc'</script><script>document.title=731942</script> makes the server return attacker-supplied markup outside the intended JS string literal: the original script element is terminated and a new inline script element containing arbitrary JavaScript source is delivered to the browser, with no CSP restricting script execution.

Potential impact

If the injected script executes in a victim's browser (browser engine execution was not directly observed in this runtime), the attacker could run arbitrary JavaScript in the application origin — for example performing actions on the victim's behalf or reading non-HttpOnly session state and CSRF tokens. The site is a Web Security Academy lab with no authenticated user data of consequence, so real-world data exposure was not demonstrated.

Payload

```
xsspoc'</script><script>document.title=731942</script>
```

Hình 3.16. Báo cáo Finding XSS tại tham số search

d) Nhận xét và đánh giá

Thực nghiệm cho thấy Burp Suite và KAgent có thể phối hợp để kiểm tra một yêu cầu HTTP do người kiểm thử lựa chọn. Burp Suite cung cấp dữ liệu đầu vào, còn KAgent sử dụng Skill XSS để phân tích phản hồi và lựa chọn phép thử. Vai trò của LLM thể hiện qua việc nhận diện vị trí xuất hiện của dữ liệu, so sánh cách ứng dụng xử lý đầu vào và dựa vào kết quả quan sát để tiếp tục xác minh.

Điểm đáng chú ý là KAgent phân biệt được cách xử lý cùng một đầu vào tại hai vị trí trong trang. Các ký tự tạo thẻ được mã hóa trong phần tử h1 nhưng vẫn được giữ nguyên trong khối script. Từ đó, Agent tập trung kiểm tra khả năng kết

thúc khối script ban đầu và chèn một khối script mới. Phép thử bổ sung dấu nháy đơn cho thấy cơ chế escape của ứng dụng chưa ngăn được cách chèn mã này. Các phản hồi được lưu và liên kết với kết quả xác minh, giúp người kiểm thử đối chiếu cơ sở của kết luận.

Một hạn chế của KAgent trong môi trường thực nghiệm là chưa có công cụ điều khiển trình duyệt để kiểm tra việc thực thi JavaScript. Agent vì vậy chỉ có thể phân tích nội dung HTML trả về và xác định khả năng chèn một khối script mới, chưa kiểm chứng được lệnh thay đổi tiêu đề trang có thực sự được thực thi hay không. Việc bổ sung công cụ trình duyệt sẽ giúp KAgent tiếp tục kiểm tra hành vi của mã được chèn và củng cố bằng chứng xác minh XSS.

Lượt thực nghiệm sử dụng 17 lần gọi LLM và 17 lượt gọi công cụ, trong đó có ba yêu cầu HTTP mới. KAgent ghi nhận một Candidate, một kết quả xác minh và một Finding có liên kết bằng chứng. Kết quả cho thấy hệ thống đã thực hiện được quá trình từ tiếp nhận yêu cầu qua Burp Suite đến phân tích, xác minh và lưu phát hiện. Phạm vi quan sát của kịch bản giới hạn ở một đầu vào đã được người kiểm thử lựa chọn.

Xét chung hai kịch bản, phạm vi đánh giá của đồ án còn giới hạn ở OWASP Juice Shop và bài lab XSS đã lựa chọn trên Web Security Academy. Các kết quả phản ánh những khả năng và hạn chế quan sát được trong điều kiện thực nghiệm cụ thể, chưa đủ để khái quát tính ổn định, khả năng tái lập hoặc hiệu quả trên các ứng dụng khác. Các chỉ số về số bước và lượt gọi công cụ cũng chưa thay thế phép đo đầy đủ về lượng token, chi phí dịch vụ và tài nguyên hạ tầng khi quy mô kiểm thử tăng lên.

3.4. Hạn chế và hướng phát triển

KAgent được xây dựng theo kiến trúc Single-Agent, kết hợp LLM với Skill, công cụ và cơ chế quản lý trạng thái kiểm thử. Bên cạnh những chức năng đã hiện thực, hệ thống còn có các hạn chế về điều phối, phạm vi và chất lượng kỹ năng, độ tin cậy của kết quả, quản lý ngữ cảnh, chi phí vận hành và mức độ bảo đảm an toàn. Những hạn chế này được xem xét từ thiết kế và phạm vi hiện thực của hệ thống để xác định các hướng phát triển phù hợp.

3.4.1. Hạn chế của hệ thống

Khả năng điều phối của KAgent hiện dựa trên các quy tắc lựa chọn Skill, trạng thái quy trình, mức ưu tiên và bằng chứng đã ghi nhận của Candidate. Những cơ chế này hỗ trợ duy trì tiến độ, trong khi lựa chọn phép thử và diễn giải thông tin mới vẫn

phụ thuộc vào suy luận của LLM. Việc xếp hạng công việc chưa phản ánh đầy đủ mức độ tin cậy của giả thuyết, lượng thông tin dự kiến thu được và chi phí thực hiện từng phép thử. Vì vậy, thứ tự xử lý hợp lệ theo quy trình chưa đủ để bảo đảm các điểm kiểm tra quan trọng được ưu tiên phù hợp.

Phạm vi kiểm thử còn giới hạn bởi Skill và công cụ hiện có. Hệ thống có hướng dẫn xác minh chuyên biệt cho SQL Injection và Cross-Site Scripting, còn những nhóm lỗ hổng khác cần quy trình tổng quát hoặc Skill bổ sung. Chất lượng kỹ năng cũng phụ thuộc vào tính chính xác, mức độ đầy đủ và điều kiện áp dụng của hướng dẫn. Các kiểm tra cấu trúc và điều kiện thực thi chưa đủ để đánh giá hiệu quả thực tế của hướng dẫn đối với việc lựa chọn phép thử và phân tích phản hồi. Bên cạnh đó, khả năng gửi yêu cầu HTTP và tiếp nhận lưu lượng từ Burp Suite chưa thay thế được công cụ điều khiển trình duyệt để tương tác với giao diện phía máy khách.

Độ tin cậy của kết quả kiểm thử phụ thuộc vào phương pháp xác minh, chất lượng bằng chứng và cách LLM phân tích phản hồi từ công cụ. KAgent đã triển khai các cơ chế kiểm tra nguồn gốc, liên kết bằng chứng với lượt xác minh, tính toàn vẹn và trạng thái đầy đủ của dữ liệu thu thập. Tuy nhiên, những kiểm tra này chủ yếu bảo đảm bằng chứng được ghi nhận và sử dụng đúng nguồn, chứ không tự xác định được ý nghĩa bảo mật của mọi phản hồi. Việc đánh giá sự khác biệt giữa các yêu cầu kiểm thử, lựa chọn phép đối chứng và diễn giải kết quả vẫn phụ thuộc vào phương pháp kiểm thử và khả năng suy luận của mô hình. Bên cạnh đó, hiện tượng ảo giác (Hallucination) của LLM có thể dẫn đến những nhận định thiếu căn cứ, diễn giải sai phản hồi hoặc kết luận vượt quá bằng chứng thực tế. Công cụ ghi nhận Finding sử dụng kết quả xác minh đã được chấp nhận cùng bằng chứng liên quan, không tự thực hiện thêm một lượt kiểm chứng độc lập đối với kết luận. Vì vậy, bằng chứng hợp lệ về mặt cấu trúc chưa đủ để bảo đảm kết luận về lỗ hổng luôn chính xác.

Khả năng xử lý nhiệm vụ kéo dài chịu ảnh hưởng từ giới hạn ngữ cảnh và lượng dữ liệu quan sát được lưu giữ. KAgent có cơ chế thu gọn lịch sử, truy hồi Memory và Intelligence, đồng thời duy trì trạng thái quy trình riêng. Tuy nhiên, nội dung tóm lược và dữ liệu thu nhận có giới hạn có thể không giữ đầy đủ chi tiết cần thiết cho bước phân tích sau. Tri thức được sử dụng lại cũng cần phù hợp với trạng thái hiện tại của mục tiêu. Việc khôi phục phiên chỉ nạp lại dữ liệu đã lưu, không khôi phục môi trường ứng dụng, tiến trình công cụ hoặc quyền thực thi tạm thời, nên điều kiện tiếp tục kiểm thử cần được kiểm tra lại.

Chi phí vận hành và hiệu năng của KAgent phụ thuộc vào mô hình, kích thước ngữ cảnh và phương thức triển khai LLM. Với Cloud API, các phiên kiểm thử nhiều

bước có thể làm tăng chi phí xử lý token, đồng thời chịu ảnh hưởng từ kết nối mạng, giới hạn dịch vụ và chính sách của nhà cung cấp. Mô hình cục bộ (Local LLM) giúp chủ động hơn trong quản lý dữ liệu và giảm phụ thuộc vào dịch vụ bên ngoài, nhưng vẫn có chi phí phần cứng, điện năng và vận hành. Phương án này cần được đánh giá thêm về thời gian đáp ứng và khả năng sử dụng công cụ. KAgent đã có cơ chế ghi nhận token và thời gian yêu cầu LLM, song tính đầy đủ của số liệu sử dụng còn phụ thuộc vào phản hồi của nhà cung cấp. Việc đánh giá và tối ưu chi phí tổng thể theo quy mô nhiệm vụ và phương thức triển khai vẫn cần được hoàn thiện.

An toàn bảo mật của KAgent cần được đánh giá thêm trước khi triển khai trên các hệ thống thực tế. Mặc dù đã có cơ chế kiểm soát quyền, phạm vi và xử lý thông tin nhạy cảm, hệ thống vẫn còn những giới hạn trong việc cô lập môi trường thực thi và bảo vệ dữ liệu. Khi sử dụng Cloud API, nội dung phục vụ phân tích có thể được gửi tới nhà cung cấp LLM, đặt ra yêu cầu kiểm soát việc chia sẻ thông tin nội bộ, dữ liệu xác thực và bằng chứng kiểm thử. Bên cạnh đó, các rủi ro đối với ứng dụng LLM được nêu trong OWASP Top 10 for LLM Applications cần được xem xét trong quá trình đánh giá an toàn của KAgent. Phạm vi kiểm chứng của đồ án hiện chưa đủ để kết luận hệ thống có thể vận hành an toàn trong mọi điều kiện, đặc biệt khi tiếp cận dữ liệu nhạy cảm hoặc được cấp quyền thực hiện các thao tác có ảnh hưởng lớn.

3.4.2. Định hướng phát triển hệ thống

Khả năng điều phối cần được cải thiện trên nền tảng Single-Agent hiện có. Bên cạnh trạng thái và mức ưu tiên của công việc, hệ thống cần xem xét mức độ tin cậy của giả thuyết, giá trị thông tin dự kiến và chi phí thực hiện để hỗ trợ lựa chọn phép thử. Cơ sở đề xuất hành động cần gắn với những quan sát, bằng chứng còn thiếu và mục tiêu hiện tại của người kiểm thử. Các cải tiến này cần được đánh giá bằng khả năng hoàn thành nhiệm vụ và chất lượng xác minh, đồng thời duy trì các điều kiện về quyền và phạm vi thực thi.

Việc phát triển Skill cần kết hợp mở rộng phạm vi hỗ trợ với kiểm chứng chất lượng hướng dẫn. Cần tiếp tục rà soát tính chính xác, điều kiện áp dụng, yêu cầu đầu ra và điều kiện dừng của từng kỹ năng. Với Skill xác minh, việc đánh giá cần bao gồm các trường hợp có lỗ hổng, không có lỗ hổng và chưa đủ điều kiện kết luận, cùng chất lượng phép đối chứng và bằng chứng. Hiệu quả của hướng dẫn cũng cần được đối chiếu khi sử dụng những mô hình LLM khác nhau. Các công cụ bổ sung, nhất là tự động hóa trình duyệt, cần cung cấp dữ liệu quan sát có thể liên kết với quy trình kiểm thử.

Đối với kết quả kiểm thử, cần tiếp tục hoàn thiện các tiêu chí đánh giá theo từng lớp lỗ hổng, kế thừa những kiểm tra về nguồn gốc và tính toàn vẹn của bằng chứng. Cần làm rõ mối liên hệ giữa yêu cầu gửi đi, phản hồi, phép đối chứng và tác động được kết luận. Các trường hợp thiếu bằng chứng phải được giữ ở trạng thái phù hợp để kiểm tra bổ sung. Việc thẩm định độc lập các Finding trong quá trình đánh giá hệ thống cũng cần được bổ sung để xem xét độ chính xác của kết luận.

Cơ chế quản lý ngữ cảnh cần được cải thiện theo hướng ưu tiên thông tin cần thiết cho nhiệm vụ, duy trì tham chiếu tới nguồn và đánh giá mức độ mất thông tin sau thu gọn. Memory và Intelligence cần tiếp tục được đối chiếu với công nghệ, trạng thái mục tiêu và điều kiện áp dụng của tri thức. Khi khôi phục phiên, hệ thống cần hỗ trợ kiểm tra những điều kiện có thể đã thay đổi trước khi tiếp tục. Thông tin truy hồi hỗ trợ phân tích, còn quyền thực thi và phạm vi kiểm thử phải được xác định từ trạng thái hiện hành.

Về tài nguyên LLM, cần hoàn thiện việc lưu và tổng hợp số liệu token, thời gian xử lý và chi phí theo phiên, đồng thời phân biệt rõ số liệu do nhà cung cấp trả về với ước tính ngữ cảnh của KAgent. Việc so sánh Cloud API và Local LLM cần xem xét đồng thời chất lượng thực hiện nhiệm vụ, khả năng sử dụng công cụ, độ trễ và tổng chi phí dịch vụ, phần cứng, điện năng và vận hành. Trên cơ sở đó, có thể nghiên cứu kiểm soát ngân sách và lựa chọn cấu hình phù hợp với từng điều kiện triển khai.

Về an toàn bảo mật, cần mở rộng rà soát mã nguồn, kiểm thử tích hợp và kiểm thử các trường hợp biên, lỗi hoặc hủy tác vụ giữa nhiều thành phần. Việc cô lập môi trường thực thi công cụ và giới hạn quyền của tài khoản vận hành cần được nghiên cứu cùng cơ chế kiểm soát hiện có. Đồng thời, cần đánh giá việc bảo vệ khóa API, dữ liệu phiên, bằng chứng và nội dung trao đổi với LLM. Những kịch bản Prompt Injection, tiết lộ thông tin và thao túng việc sử dụng công cụ cần được đưa vào đánh giá an toàn của hệ thống.

Trong dài hạn, kiến trúc Multi-Agent có thể được nghiên cứu để phân chia nhiệm vụ khảo sát, phân tích và thẩm định bằng chứng. Hướng này cần được so sánh với kiến trúc Single-Agent về chất lượng kết quả, chi phí và khả năng phối hợp trạng thái. Việc hoàn thiện chất lượng Skill, độ tin cậy của xác minh và an toàn hệ thống vẫn cần được ưu tiên trước khi mở rộng kiến trúc.

Các hướng phát triển cần được đánh giá bằng những lượt thực nghiệm lặp có điều kiện rõ ràng, theo dõi riêng chất lượng xác minh, mức độ hoàn thành nhiệm vụ,

độ bao phủ và tài nguyên tiêu thụ. Mọi cải tiến tiếp tục tuân thủ nguyên tắc kiểm thử có ủy quyền, giới hạn phạm vi và khả năng truy vết.

3.5. Kết luận chương 3

Chương 3 đã trình bày quá trình xây dựng, triển khai và đánh giá hệ thống KAgent dựa trên kiến trúc và các cơ chế đã thiết kế ở Chương 2. Các nội dung hiện thực cho thấy sự kết hợp giữa mô hình ngôn ngữ lớn, hệ thống kỹ năng, công cụ kiểm thử và cơ chế quản lý trạng thái, bằng chứng nhằm hỗ trợ thực hiện các tác vụ kiểm thử xâm nhập ứng dụng web trong phạm vi được kiểm soát.

Thông qua hai kịch bản thực nghiệm, đồ án đã khảo sát khả năng hoạt động của KAgent trong những điều kiện đầu vào khác nhau. Kịch bản kiểm thử SQL Injection từ địa chỉ mục tiêu ban đầu thể hiện quá trình khám phá ứng dụng, phân tích đầu vào, lựa chọn và xác minh các đối tượng nghi vấn. Kịch bản tích hợp Burp Suite cho thấy khả năng tiếp nhận yêu cầu HTTP, phân tích và xác minh XSS tại đầu vào được lựa chọn. Kết quả thực nghiệm ghi nhận khả năng phối hợp giữa suy luận của LLM và hoạt động thực thi công cụ, điều chỉnh phép thử dựa trên phản hồi, đồng thời lưu trữ kết quả cùng bằng chứng liên quan.

Bên cạnh những kết quả đạt được, quá trình đánh giá cũng cho thấy KAgent vẫn còn hạn chế về khả năng điều phối, mức độ hoàn tất nhiệm vụ, độ tin cậy của kết luận và phạm vi kiểm thử. Những hạn chế này là cơ sở để đề xuất các hướng cải tiến về lựa chọn phép thử, xác minh bằng chứng, mở rộng khả năng quan sát và đánh giá hiệu quả hệ thống. Nhìn chung, kết quả Chương 3 cho thấy tính khả thi của việc ứng dụng AI tạo sinh để hỗ trợ một phần quy trình kiểm thử xâm nhập, đồng thời xác định những vấn đề cần tiếp tục hoàn thiện để nâng cao khả năng hoạt động của KAgent.

KẾT LUẬN

Đề tài "Nghiên cứu ứng dụng AI tạo sinh trong kiểm thử xâm nhập" được thực hiện nhằm khảo sát cách kết hợp mô hình ngôn ngữ lớn với công cụ và trạng thái nhiệm vụ để hỗ trợ kiểm thử ứng dụng web. Từ cơ sở lý thuyết về kiểm thử xâm nhập, LLM và AI Agent, đồ án lựa chọn kiến trúc Single-Agent để nghiên cứu và hiện thực nguyên mẫu KAgent theo chu trình suy luận, hành động và tiếp nhận kết quả.

KAgent đã được thiết kế và xây dựng với sự phối hợp giữa điều phối nhiệm vụ, nạp Skill theo ngữ cảnh, sử dụng công cụ và quản lý workflow. Các kết quả kiểm tra được liên kết với quan sát và bằng chứng để hỗ trợ ghi nhận phát hiện và truy vết kết luận. Người kiểm thử thiết lập mục tiêu, theo dõi tiến trình và tham gia khi cần, trong khi các thao tác thực thi chịu sự kiểm soát về phạm vi, quyền và tài nguyên.

Hai kịch bản thực nghiệm đã khảo sát việc khám phá, lựa chọn đầu vào và kiểm tra SQL Injection từ địa chỉ ứng dụng ban đầu, cùng kiểm tra XSS từ yêu cầu HTTP được chuyển qua Burp Suite. Kết quả cho thấy nguyên mẫu có thể sử dụng phản hồi công cụ để điều chỉnh phép thử và lưu phát hiện kèm bằng chứng trong các trường hợp đã kiểm tra. Các giới hạn về bằng chứng xác minh và nhiệm vụ chưa khép lại cho thấy chất lượng kết luận và mức độ hoàn tất công việc cần được đánh giá riêng.

Trong phạm vi đó, đồ án đã hiện thực được nguyên mẫu và tổ chức đánh giá ban đầu theo mục tiêu đề ra, cung cấp cơ sở xem xét việc ứng dụng AI tạo sinh để hỗ trợ một phần quy trình kiểm thử. Phạm vi kỹ năng, khả năng điều phối, chất lượng bằng chứng và dữ liệu chạy lặp vẫn cần được hoàn thiện. Những ưu tiên tiếp theo là nâng cao độ tin cậy của xác minh, mở rộng nguồn quan sát và đánh giá trên nhiều điều kiện được kiểm soát hơn.

Kết quả của đồ án tạo nền tảng tiếp tục nghiên cứu việc sử dụng tri thức bảo mật theo ngữ cảnh và phối hợp các vai trò chuyên biệt khi quy mô nhiệm vụ mở rộng. Giá trị của những hướng này cần được kiểm chứng bằng thực nghiệm, đồng thời duy trì kiểm thử có ủy quyền và khả năng truy vết. KAgent vì vậy là một nguyên mẫu phục vụ nghiên cứu cách tổ chức sự phối hợp giữa LLM, công cụ và người kiểm thử, với triển vọng phát triển dựa trên chất lượng bằng chứng và kết quả đánh giá thực tế.

TÀI LIỆU THAM KHẢO

[1] PTES Team, "The Penetration Testing Execution Standard." http://www.pentest-standard.org/

[2] NIST, "Technical Guide to Information Security Testing and Assessment," SP 800-115, 2008. https://csrc.nist.gov/pubs/sp/800/115/final

[3] OWASP Foundation, "OWASP Web Security Testing Guide, Version 4.2." https://owasp.org/www-project-web-security-testing-guide/

[4] OWASP Foundation, "OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks," 2021. https://owasp.org/Top10/2021/

[5] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017. https://arxiv.org/abs/1706.03762

[6] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS, 2022. https://arxiv.org/abs/2201.11903

[7] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR, 2023. https://arxiv.org/abs/2210.03629

[8] G. Deng et al., "PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing," 33rd USENIX Security Symposium, 2024. https://www.usenix.org/conference/usenixsecurity24/presentation/deng

[9] A. Happe, A. Kaplan, and J. Cito, "LLMs as Hackers: Autonomous Linux Privilege Escalation Attacks," Empirical Software Engineering, vol. 31, art. 70, 2026. https://doi.org/10.1007/s10664-025-10758-3

[10] K. Desai, "Pentest Copilot Auto Identifies and Exploits XSS," BugBase, 15/08/2025. https://copilot.bugbase.ai/blogs/pentest-copilot-auto-exploits-xss

[11] X. Shen et al., "PentestAgent: Incorporating LLM Agents to Automated Penetration Testing," 20th ACM AsiaCCS, 2025. https://doi.org/10.1145/3708821.3733882

[12] J. Henke, "AutoPentest: Enhancing Vulnerability Management With Autonomous LLM Agents," arXiv:2505.10321, 2025. https://arxiv.org/html/2505.10321v1

[13] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020. https://arxiv.org/abs/2005.11401

[14] MITRE Corporation, “Common Weakness Enumeration (CWE).” https://cwe.mitre.org/

[15] N. V. Khải, “KAgent - AI-assisted Penetration Testing Agent,” GitHub repository. https://github.com/ynvk404/KAgent

[16] OWASP Foundation, “OWASP Juice Shop.” https://owasp-juice.shop/

[17] DeepSeek, “Models & Pricing,” DeepSeek API Docs. https://api-docs.deepseek.com/quick_start/pricing/

[18] PortSwigger, “Lab: Reflected XSS into a JavaScript string with single quote and backslash escaped,” Web Security Academy. https://portswigger.net/web-security/cross-site-scripting/contexts/lab-javascript-string-single-quote-backslash-escaped

[19] PortSwigger, “Installing extensions manually,” Burp Suite Documentation. https://portswigger.net/burp/documentation/desktop/extend-burp/extensions/installing/manual-install

PHỤ LỤC

Chuẩn bị và cài đặt

KAgent yêu cầu Python 3.11 trở lên. Trên Ubuntu, cần kết nối mạng, Git, môi trường ảo Python và quyền sudo để cài các gói hệ thống. Chạy các lệnh sau trong terminal để tải mã nguồn và cài đặt [15].

```
sudo apt update
sudo apt install git python3 python3-venv ca-certificates
mkdir -p ~/kagent-demo
cd ~/kagent-demo
git clone https://github.com/ynvk404/KAgent.git kagent
cd kagentssss
python3 -m venv .venv
source .venv/bin/activate
python -m pip install .
python -m pip check
kagent --version
```

Cài đặt thành công khi pip check không báo xung đột và lệnh cuối hiển thị phiên bản KAgent. Trong terminal mới, vào thư mục ~/kagent-demo/kagent và kích hoạt lại môi trường bằng source .venv/bin/activate.

Cấu hình LLM

KAgent lưu cấu hình mặc định tại ~/.kagent/config.json. Trong terminal, tạo thư mục cấu hình nếu chưa có và mở tệp:

```
umask 077
mkdir -p ~/.kagent
nano ~/.kagent/config.json
```

Với tệp mới, nhập cấu hình cho mô hình deepseek-flash [17] và thay API_KEY_CUA_BAN bằng khóa DeepSeek.

```
{
  "backend": "deepseek", "model": "deepseek-flash",
  "base_url": "", "active_custom_provider_id": null,
  "api_keys": {"deepseek": "API_KEY_CUA_BAN"}
}
```

Nếu tệp đã có cấu hình, chỉ sửa hoặc bổ sung các trường tương ứng và giữ lại những trường khác, kể cả các khóa của nhà cung cấp khác trong api_keys. Trong nano, lưu bằng Ctrl+O, nhấn Enter, sau đó thoát bằng Ctrl+X.

Giới hạn quyền đọc tệp và khởi chạy KAgent:

```
chmod 600 ~/.kagent/config.json
kagent
```

Trong giao diện TUI, nhập /provider, chọn Show current config để xem nhà cung cấp và mô hình, sau đó chọn Test connection để kiểm tra kết nối. Kết quả mong đợi là Connection OK. Tệp cấu hình chứa API key dạng văn bản thuần, vì vậy không chia sẻ tệp hoặc đưa tệp vào kho mã nguồn.

Khởi chạy và sử dụng cơ bản

Trong terminal đã kích hoạt môi trường ảo, khởi chạy với mục tiêu và giới hạn 100 bước như ví dụ sau. Thay địa chỉ bằng ứng dụng đang hoạt động và được phép kiểm thử.

```
kagent --target http://127.0.0.1:3000 --max-steps 100
```

Trong TUI, nhập /skills, chọn Skill phù hợp và nhấn Enter. Nhập yêu cầu, theo dõi kết quả và xem xét phạm vi, hành động trước khi cấp quyền khi được hỏi. Dùng /exit để thoát. Giới hạn bước không phải là giới hạn số yêu cầu HTTP.

Kết nối Burp Suite

Sau khi cài Burp Suite, tải Jython Standalone 2.7.x. Trong Burp, chọn tệp JAR tại Settings → Extensions → Python Environment. Vào Extensions → Installed → Add, chọn Python và tệp integrations/burp/kagent_burp.py trong mã nguồn [19]. Extension nạp thành công sẽ tạo tab KAgent.

Trong TUI KAgent, nhập các lệnh sau để bật cầu nối và xem thông tin kết nối.

```
/burp 8888
/burp credentials
```

Sao chép địa chỉ cầu nối và token vào tab KAgent trong Burp rồi nhấn Save. Không chia sẻ token. Chọn một yêu cầu HTTP thuộc mục tiêu được phép trong Burp và chọn Send to KAgent. Trở lại TUI, liệt kê và chọn yêu cầu bằng mã hiển thị.

```
/burp list
/burp use <id>
```

Kiểm tra yêu cầu và phạm vi, sau đó nhập yêu cầu phân tích cho Agent. Việc chọn yêu cầu không tự cấp quyền thực thi.

Phạm vi kiểm chứng. Báo cáo gốc ghi nhận các bước cài gói, mở giao diện, liệt kê thành phần, lưu và khôi phục phiên cùng thao tác HTTP cục bộ đã được kiểm tra trong môi trường tách biệt. Khi biên soạn bản rút gọn, cú pháp và chức năng được đối chiếu với mã nguồn. Kết nối LLM trực tiếp và tích hợp Burp chưa được kiểm chứng lại trong lượt biên soạn này.